

Centre of n points:

Project Proposal:

My project proposes the user entering an n number of points (otherwise referred to as vertices) on a map and finding the centre of them; otherwise referred to as the point equidistant from every vertex inputted. The central point shall be moved dependent on where a 'logical position' close to it, which will be an accessible location (for example if the centre was in the middle of a lake, then it would need to be moved to a suitable location). Hence this program is trying to deduce the most accessible point on a map, which has near infinite potential to be refined and developed.

The approach could consist of one of two methods: Abstract to Defined, or Accurate to Developed. Both are iterative approaches; however, each follow their own development cycle behind them. The Abstract to Defined approach is to figure out all the mathematical complexities on an abstract and fundamental level – stripped of all the maps and locations – to use simple coordinate geometry to find the centre point with increased accuracy.

However alternatively the Accurate to Developed approach follows a much more industry-based pathway, in which I get the centroids of 2 and 3 vertices, and perfect them to their ideal, and then develop further from that (See more in research).

This project will take a lot of time and effort to perfect, especially with a further development of introducing shortest path algorithms and memory storage for users. Despite the difficulty in the project ahead, I will aim to perfect it to the best that I can achieve in the time frame.

Project Decomposition:

Project Breakdown:

The project aims to find the centre point of any number of vertices entered on a map, in which the centre is equidistant from each vertex. However, this becomes much more difficult past three points due to the nature of a 'centre point'. For two and three vertices there is a very clearly defined centre point, that lies equal distant from every other point, but past that there is no definitive point; it only can be approximated to an extent. Therefore, a good starting point is trying to figure out a method to determine the approximated centre point (or range of points) found in a shape.

The problem gets even more complicated from then though, as what then needs to be considered is 'accessibility'. Not only will some points be completely inaccessible for the user – making the program rather redundant – but it also some points won't actually be the 'centre', due to the annoyance of roads. Whilst it is all good to assume that finding the direct distance from the centre point and the vertices will give the answer I am

looking for, unfortunately our world is constructed in such a way that roads do not follow any linear path. Hence, I will need to account for that when calculating the centre point, using Dijkstra's algorithm to find the shortest path.

Despite the already many levels of complexity, there is an additional issue in optimizing this program. In how I have currently decomposed the program, it assumes that the roads are empty, and it will be smooth driving at the same speed from each vertex when it is very definitely not. This therefore does imply there is no *true* centre, as a large range of factors will change every day's ideal centre point, from the speed of the car to the traffic to the weather. Therefore, I plan on focusing (at least at first) on the hypothetical centre point, in which the centre is very consistent and requires the smallest number of possible approximations, though with many assumptions. Then work will need to be done for the true centre point, in which a variety of heuristics will be implemented into Dijkstra's cooler younger sibling: A* algorithm, however when I get to that phase of development, I will need to take into account the large number of difficulties I will find when compiling so much data.

The program will also need to be user friendly, using a GUI to show exactly where the centre point is and not just giving an address and some coordinates. The GUI may become one of the most difficult aspects due to the programming language I am using. Python has two libraries that are best for GUIs and neither are particularly fun to use. Pygame and Tkinter are both relatively difficult and tedious to use and manipulate, and with something like a map on large scales they could become near impossible to use. On the other hand, maybe more research just needs to be done in finding the optimal library for my GUI, but it is something to consider.

In accessing the coordinate system of the world (latitude and longitude) I will be using the Google Maps API that takes in addresses and locations and converts them into latitude and longitude points, similarly it can do them in reverse to a varying degree of accuracy. As a result of the primarily small range of locations taken initially, the 3D components will be ignored initially, however when going into more and more complex builds of the app, it will be appropriate to factor height and distance traversed whilst in 3D. Therefore, this project has a large scope of potential and improvement, with a constant opportunity to develop a better and more complex program, giving me ample experience for the future.

Methodology:

For my project, it is something heavily reliant on testing, improvements, and development, with a large focus on the user using the program. Therefore, the project lends itself to an Agile software development methodology. An Agile development cycle is the process of repeatedly programming to a certain goal, evaluating it against tests and stakeholders, and then to receive updated requirements for the process to start all over again.

Agile aims to ensure constant stakeholder collaboration, regular reflection and evaluation, and a consistent stream of innovation. This is essential for this project, as the aim to let the user be able to find the midpoints of the points they enter with ease and efficiency; hence making the stakeholder's assessment paramount to its development. On the other hand, it is important in a mathematical sense. As what I am going into is entirely new mathematics for me, and potentially non-existent mathematics further down the line, ensuring that I take an iterative approach will make sure that I don't limit myself to just trying to figure out the mathematics for it, and that I actually produce a product.

Stakeholder:

With so much discussion of the importance of the stakeholder in the methodology section, they are evidently very important. Initially, my stakeholder was to be someone with a background in IT architecture, so that I could better improve my insight into user interfaces and APIs however, as research continued, I decided to make a change to my stakeholder. My mathematics teacher Mr. Edward Banks was my choice as I needed someone with the background in maths itself. As the research went on, the more the project became an effort in showcasing the midpoint on the Earth, which whilst it has its uses for finding the optimal location to meet between some friends and family, its best uses come from its mathematical value. Moreover, due to the amount of maths present in the project, it would be more helpful to have someone with that background to point out mistakes and make suggestions, giving overall more beneficial advice.

Feature's list:

<u>Feature</u>
Display an image of a map pulled from Google Maps API
Display the location of points on that image
Zoom in to the image a suitable amount dependent on what is wanted
To be able to enter any number of points to the program
Using the number of points to find the point equidistant from each point
Checking equidistance and outputting the distance to the centre
Suggesting tourist/meeting locations near that centre point
An option to adjust the centre to remove unreachable points

Research:

Other Solutions:

Before I started my research on the mathematics, I first investigated what programs already existed, to see if there was anything that I could find that mirrored what my project would strive to do. The first one that I found was Whatshalfway.com. This took two postcodes and then found a midpoint venue between them by following the direct route between them on the roads. This is interesting as this is a variation of what I am trying to achieve, however it is something additional that I want to be implemented

alongside my program, and hence it is a program to keep in mind. The next one I found did something similar with the website meetways.com. Instead of giving an individual location or venue, the program spat out several locations that were potential locations to meet. Once again this is like what I want to achieve, but it is focusing on a much more applied setting, not the midpoint as the crow flies. Moreover, it is lacking in being able to do more than two points.

The next website I found had much more of what I wanted, calculating distance, bearing and midpoint between latitude and longitude points. Movable-type.co.uk looks to be a useful website, detailing some of their mathematical processes, as well as giving distance and midpoint as the crow flies, rather than through roads/transport. However, this seems to highlight the general thing missing that centre point calculators can only calculate between two points. The algorithms I have seen online concerning multiple points constantly take the average of latitude and longitudes (and cartesian coordinates?), but I know this does not give a point that is equidistant from all of them, which can be shown with a few counter examples where more points are clustered together.

Circumcentre:

The first stage of my research was modulated into two aspects: abstract and realised. The abstract aspects of my initial research were concerning the mathematics of finding a centre point of an n -sided shape, and the possible methods I could use to find it. On the other hand, my realised research would cover the design and application aspect of the program, looking at other existing examples and learning to manipulate the Google Maps API. My first steps into researching the abstract areas of the problem were how do I find the midpoint of different sided shapes, starting from 1 vertex to as high as I could go. For when there is a single point no calculation is required as the distance from itself is 0, however the more vertices there are the more difficult the mathematics get. Below are the two proofs completed by me in finding the midpoint of a line and a triangle – referred to as the circumcentre.

Circumcenter of a triangle

Points $A(x_1, y_1)$, $B(x_2, y_2)$, $C(x_3, y_3)$ form a triangle

The lines have equations :

$$AB : y - y_1 = \frac{y_2 - y_1}{x_2 - x_1}(x - x_1)$$

$$BC : y - y_2 = \frac{y_3 - y_2}{x_3 - x_2}(x - x_2)$$

$$CA : y - y_3 = \frac{y_1 - y_3}{x_1 - x_3}(x - x_3)$$

$$\Rightarrow y - y_n = \frac{y_{n+1} - y_n}{x_{n+1} - x_n}(x - x_n) \text{ where } (x_4, y_4) = (x_1, y_1)$$

$$\Rightarrow y = \frac{y_{n+1} - y_n}{x_{n+1} - x_n}x - \frac{y_{n+1} - y_n}{x_{n+1} - x_n}x_n + y_n$$

$\therefore$ The general form of the gradient of the lines of the triangles : $-\frac{x_{n+1} - x_n}{y_{n+1} - y_n}$

$$\Rightarrow y - \frac{y_n + y_{n+1}}{2} = -\frac{x_{n+1} - x_n}{y_{n+1} - y_n}\left(x - \frac{x_n + x_{n+1}}{2}\right)$$

$$\Rightarrow y = \frac{y_{n+1} + y_n}{2} + \frac{(x_{n+1} - x_n)(x_{n+1} + x_n)}{2(y_{n+1} - y_n)} - \frac{2(x_{n+1} - x_n)}{2(y_{n+1} - y_n)}$$

$$\Rightarrow y = \frac{y_{n+1} + y_n}{2} + \frac{(x_{n+1} - x_n)(x_{n+1} + x_n)}{2(y_{n+1} - y_n)} - \frac{2(x_{n+1} - x_n)}{2(y_{n+1} - y_n)}x$$

$$\Rightarrow 2(y_{n+1} - y_n)y = (y_{n+1} - y_n)(y_{n+1} + y_n) + (x_{n+1} - x_n)(x_{n+1} + x_n) - 2(x_{n+1} - x_n)x$$

$$\Rightarrow 2(y_{n+1} - y_n)y + 2(x_{n+1} - x_n)x = (x_{n+1} - x_n)(x_{n+1} + x_n) + (y_{n+1} - y_n)(y_{n+1} + y_n)$$

Equations of the lines are :

$$2(y_2 - y_1)y + 2(x_2 - x_1)x = (x_2 - x_1)(x_2 + x_1) + (y_2 - y_1)(y_2 + y_1)$$

$$\text{and } 2(y_3 - y_2)y + 2(x_3 - x_2)x = (x_3 - x_2)(x_3 + x_2) + (y_3 - y_2)(y_3 + y_2)$$

$$\Rightarrow y = \frac{(x_2 - x_1)(x_2 + x_1) + (y_2 - y_1)(y_2 + y_1) - 2(x_2 - x_1)x}{2(y_2 - y_1)}$$

$$\text{and } y = \frac{(x_3 - x_2)(x_3 + x_2) + (y_3 - y_2)(y_3 + y_2) - 2(x_3 - x_2)x}{2(y_3 - y_2)}$$

$$\Rightarrow (y_3 - y_2)[(x_2 - x_1)(x_2 + x_1) + (y_2 - y_1)(y_2 + y_1) - 2(x_2 - x_1)x]$$

$$= (y_2 - y_1)[(x_3 - x_2)(x_3 + x_2) + (y_3 - y_2)(y_3 + y_2) - 2(x_3 - x_2)x]$$

$$\begin{aligned} & \Rightarrow (y_3 - y_2)[(x_2 - x_1)(x_2 + x_1) + (y_2 - y_1)(y_2 + y_1)] - 2(x_2 - x_1)(y_3 - y_2)x \\ & = (y_2 - y_1)[(x_3 - x_2)(x_3 + x_2) + (y_3 - y_2)(y_3 + y_2)] - 2(x_3 - x_2)(y_2 - y_1)x \end{aligned}$$

$$\begin{aligned} & \Rightarrow [2(x_3 - x_2)(y_2 - y_1) - 2(x_2 - x_1)(y_3 - y_2)]x \\ & = (y_2 - y_1)[(x_3 - x_2)(x_3 + x_2) + (y_3 - y_2)(y_3 + y_2)] - (y_3 - y_2)[(x_2 - x_1)(x_2 + x_1) + (y_2 - y_1)(y_2 + y_1)] \end{aligned}$$

$$\begin{aligned} & \Rightarrow [2(x_3 - x_2)(y_2 - y_1) - 2(x_2 - x_1)(y_3 - y_2)]x \\ & = (y_2 - y_1)[x_3^2 - x_2^2 + y_3^2 - y_2^2] - (y_3 - y_2)[x_2^2 - x_1^2 + y_2^2 - y_1^2] \end{aligned}$$

$$\begin{aligned} & \Rightarrow [2(x_3 - x_2)(y_2 - y_1) - 2(x_2 - x_1)(y_3 - y_2)]x \\ & = [(y_2 - y_1)(x_3^2 + y_3^2) - (y_2 - y_1)(x_2^2 + y_2^2)] - [(y_3 - y_2)(x_2^2 + y_2^2) - (y_3 - y_2)(x_1^2 + y_1^2)] \\ & \Rightarrow [2(x_3 - x_2)(y_2 - y_1) - 2(x_2 - x_1)(y_3 - y_2)]x \\ & = (y_2 - y_1)(x_3^2 + y_3^2) + (y_3 - y_2)(x_1^2 + y_1^2) - [(y_2 - y_1)(x_2^2 + y_2^2) + (y_3 - y_2)(x_2^2 + y_2^2)] \end{aligned}$$

$$\begin{aligned} & \Rightarrow [2(x_3 - x_2)(y_2 - y_1) - 2(x_2 - x_1)(y_3 - y_2)]x \\ & = (y_2 - y_1)(x_3^2 + y_3^2) + (y_3 - y_2)(x_1^2 + y_1^2) - [x_2^2(y_3 - y_1) + y_2^2(y_3 - y_1)] \end{aligned}$$

$$\begin{aligned} & \Rightarrow [2(x_3 - x_2)(y_2 - y_1) - 2(x_2 - x_1)(y_3 - y_2)]x \\ & = (x_1^2 + y_1^2)(y_3 - y_2) + (x_2^2 + y_2^2)(y_1 - y_3) + (x_3^2 + y_3^2)(y_2 - y_1) \\ & \Rightarrow x = \frac{1}{2} \left| \frac{(x_1^2 + y_1^2)(y_3 - y_2) + (x_2^2 + y_2^2)(y_1 - y_3) + (x_3^2 + y_3^2)(y_2 - y_1)}{(x_3 - x_2)(y_2 - y_1) - (x_2 - x_1)(y_3 - y_2)} \right| \end{aligned}$$

Through using similar methods the y coordinate was found to be :

$$y = \frac{1}{2} \left| \frac{(x_1^2 + y_1^2)(x_2 - y_3) + (x_2^2 + y_2^2)(y_3 - y_1) + (x_3^2 + y_3^2)(y_1 - y_2)}{(x_3 - x_2)(y_2 - y_1) - (x_2 - x_1)(y_3 - y_2)} \right|$$

With some manipulation the coordinates can tidied up to give :

$$C_x = \frac{1}{2} \left| \frac{\sum_{n=1}^3 (x_n^2 + y_n^2)(y_{n+2} - y_{n+1})}{\sum_{n=1}^3 x_n(y_{n+2} - y_{n+1})} \right|$$

$$C_y = \frac{1}{2} \left| \frac{\sum_{n=1}^3 (x_n^2 + y_n^2)(x_{n+2} - x_{n+1})}{\sum_{n=1}^3 x_n(y_{n+2} - y_{n+1})} \right|$$

Circumcenter of a line

Points $A(x_1, y_1)$, $B(x_2, y_2)$ form a line

Half way in the x and the y directions will give the midpoint :

$$C_x = \frac{x_1 + x_2}{2}$$

$$C_y = \frac{y_1 + y_2}{2}$$

Cyclic formulae:

As seen above, the centre point of a shape gets rather complicated rather quickly, so that is when I turned to the internet and other information regarding the midpoint of a shape. At first, I went to Wikipedia, in researching how to find the centre, in which I found that the formula to find the barycentric coordinates of any shape is:

$$C_x = \frac{1}{6A} \sum_{i=0}^{n-1} (x_i + x_{i+1})(x_i y_{i+1} - x_{i+1} y_i) , \text{ and}$$

$$C_y = \frac{1}{6A} \sum_{i=0}^{n-1} (y_i + y_{i+1})(x_i y_{i+1} - x_{i+1} y_i) ,$$

$$A = \frac{1}{2} \sum_{i=0}^{n-1} (x_i y_{i+1} - x_{i+1} y_i) .$$

With this knowledge I tested it on several shapes and vertex sets to see if it were valid, as not only could I not find this on other websites, but it also originated from Wikipedia which is open to anyone to edit. The results of my written testing were difficult in grasping how the equation worked, with it seeming to only work in some cases, however I was unsure if it was a result of potential arithmetic errors that I made, so I decided to program in the formulas to test them.

```
1. class point:
2.     x = 0
3.     y = 0
4.     i = 0
5.     area = 0
6.     Cx = 0
7.     Cy = 0
8.
9.     points = []
10.
11.     def __init__(self, x, y, i):
12.         self.x = x
13.         self.y = y
14.         self.i = i
15.
16.     def display_coords(self):
17.         print(self.x, self.y)
18.
19.     def list_append(self):
20.         self.points.append([self.x, self.y, self.i])
21.
22.     def signed_area(self):
23.         for i in range(0, n-1):
24.             self.area = self.area + 0.5*((self.points[i][0] * self.p
oints[i+1][1]) - (self.points[i+1][0] * self.points[i][1]))
```

```

25.         return abs(self.area)
26.
27.     def center_x(self):
28.         for i in range(0, n-1):
29.             #print("x: " + str(self.Cx))
30.             self.Cx = self.Cx + ((self.points[i][0] + self.points[i+
1][0])*((self.points[i][0] * self.points[i+1][1]) - (self.points[i+1
][0] * self.points[i][1])))
31.             self.Cx = self.Cx*(1/(6*full_area))
32.         return abs(self.Cx)
33.
34.     def center_y(self):
35.         for i in range(0, n-1):
36.             #print("y: " + str(self.Cy))
37.             self.Cy = self.Cy + ((self.points[i][1] + self.points[i+
1][1])*((self.points[i][0] * self.points[i+1][1]) - (self.points[i+1
][0] * self.points[i][1])))
38.             self.Cy = self.Cy*(1/(6*full_area))
39.         return abs(self.Cy)
40.
41.     def distance_check(self):
42.         for z in range(0, n):
43.             print("From vertex " + str(z) + ": " + str(((self.points
[z][0]-abs(self.Cx))**2 + (self.points[z][1]-
abs(self.Cy))**2)**0.5))
44.
45. n = int(input("Enter how many vertices the shape has: "))
46. for i in range(0, n):
47.     input_x = int(input("Enter the x coordinate of the vertex: "))
48.     input_y = int(input("Enter the y coordinate of the vertex: "))
49.
50.     append_point = point(input_x, input_y, i)
51.     point.list_append(append_point)
52.
53. final_point = point(point.points[0][0], point.points[0][1], point.po
ints[0][2]+n)
54. point.list_append(final_point)
55.
56. full_area = point.signed_area(append_point)
57. center_coords = [point.center_x(append_point), point.center_y(append
_point)]
58. print(center_coords)
59. point.distance_check(append_point)

```

In this testing I gave in vertices with centre points that I knew already from working out beforehand, as well as vertices in which I didn't know how to calculate the centre point. However, it did not matter as I tested the result with the method `distance_check()`. This calculated the distance between the calculated centre point and the vertices, which will be used for all future tests, using the basic distance formula of:

The results from testing the formula found was dissatisfying. As in, it did not work at all. The code was done properly, and there seemed to be no issue in its design, so then I started to consider what was at fault. I didn't figure out the issue until I had gone to my mathematics teacher to ask about the centre of an n-sided shape, and he posed the possibility that potentially all shapes may not have a centre. With this I realised that there may not be a universal formula for the centre point for all shapes, which meant that the equation I found must be achieving something else. What I was looking into was the 'centroid' of shape, which is not the exact centre of a shape. What it is more accurately is the average point of a shape. By average I refer to the distribution points of an object of uniform mass. The centre of mass is a concept in physics which refers to the point the mass concentrates itself around, and this is what that formula calculates, therefore this whole time I was experimenting with a formula that was calculating the wrong thing. However, this formula did provide some insight into vertex geometry with the shoelace formula.

$$A = \frac{1}{2} \sum_{i=0}^{n-1} (x_i y_{i+1} - x_{i+1} y_i).$$

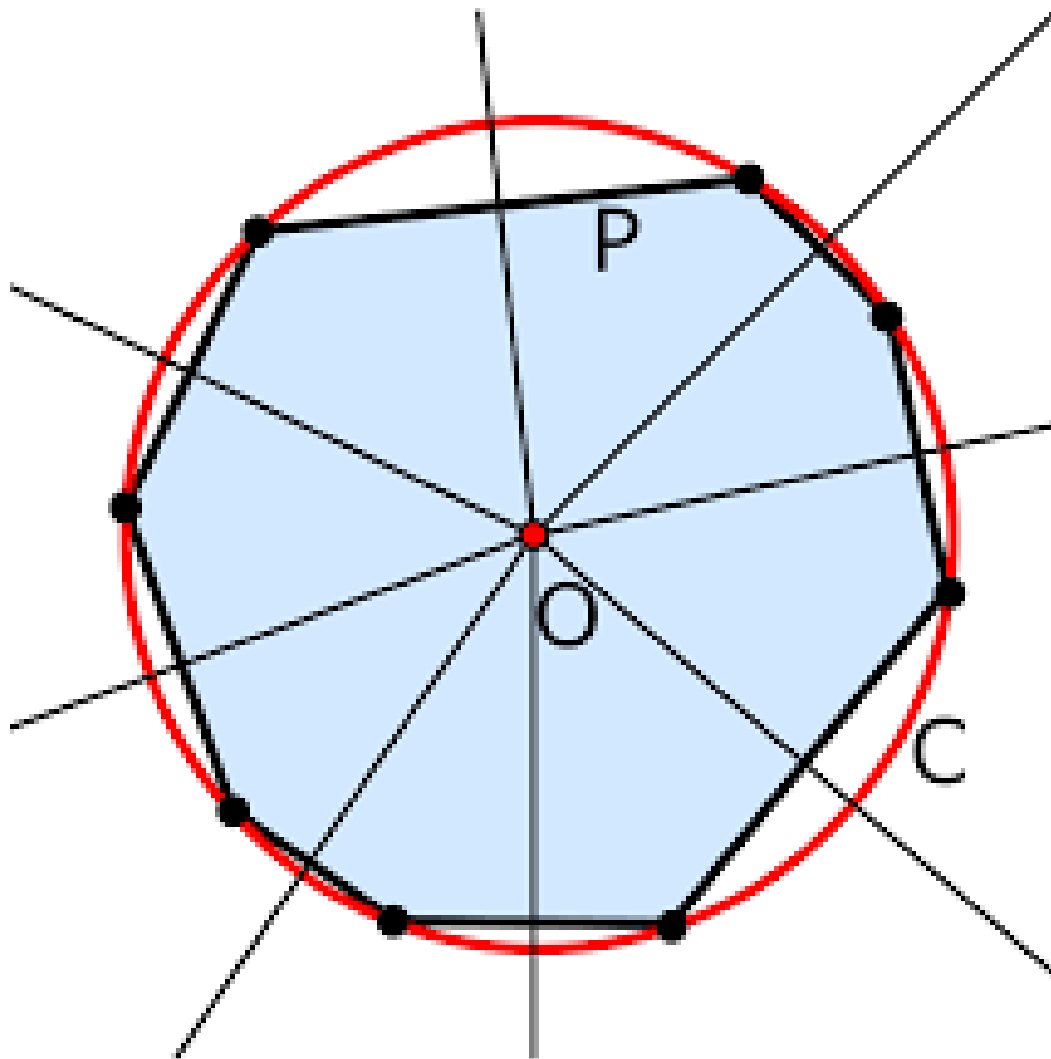
This formula holds true for any non-self-intersecting polygon due to the way that triangles can be formed inside them. The proof which I derived myself is below, and this 'signed area' is useful for how I can use it in future attempts in my goal in finding the centre of a shape.

Evidently, I was at a loss, as I found that not every shape has a specialised centre point, meaning that there was no guarantee that with the coordinates inputted by the user on a map will actually have a defined point. For a while I spent time in experimenting with quadrilaterals, setting up simultaneous equations to find the centre point, however nothing could ever work as of the number unknowns that I had. I had to take another approach, a new way of looking at the problem.

But that was when I noticed what the centre of certain shapes were called; they were referred to by the barycentre, the centroid and then the circumcentre. There had to be a difference in their meanings and there was a rather significant difference. Whilst the centroid of a shape referred to the average of all the coordinates, the circumcentre is the point in which a circumscribed shape's circle's centre lies. This was an important realisation as if you look at the diagram below, the very nature of a circle is that every point on the circle is an equal distance away (the radius) from a specific point. This hence implies that cyclic shapes (shapes in which every vertex lies on a circle) have a definite centre point.

By its very definition, a triangle is a cyclic polygon, as it will always have a definite centre point. However, all polygons with a number of vertices greater than three aren't defined as cyclic. This would logically imply that this information is rather useless, however it is more useful than you may think, as this means if small modifications are made to a noncyclic polygon, to get some condition to tend toward making the shape a cyclic

polygon, it would mean the approximate centre point can be determined. Due to the lateral nature of this project's goals, the exact centre value will not be necessary, rather an approximation that will make travel distance for users to be nearly equal.



The main crux of the project falls upon an algorithm that I will need to create, in making a shape become cyclic. This algorithm will be constantly repeated, translating each vertex each 'pass' so that it increasingly more accurate. Therefore, this algorithm must be programmed, as otherwise it would be too difficult and cumbersome of a task for any human to do in a reasonable time frame. This is mathematically shown by the roots of polynomials. Whilst not rigorously proven, Brahmagupta's and Robbin's formulas show an increasing exponent, which hence creates more and more variables that need altering. Those formulas I referred to are methods in which to confirm whether a shape is cyclic. For a cyclic quadrilateral, Brahmagupta produces the polynomial:

$$K = \sqrt{(s-a)(s-b)(s-c)(s-d)}$$

$$s = \frac{a+b+c+d}{2}$$

Where a , b , c and d are each a side of the quadrilateral. As it can be seen this already produces quartic exponent components of the equation, and hence may already prove difficult. The proof for this already lies on the Wikipedia page, so I won't go into it, however later when developing potential other formulas for cyclic areas, it may be useful in giving some further direction. There are also formulas for a cyclic pentagon and a cyclic hexagon, giving their area as well, but it becomes incredibly complicated, to the point that typing it is a pointless task.

This is where the issues begin though, as when addressing Robbin's formulas, they only hold true for rational numbers, hence ruling out many distances considering that the result of Pythagoras' theorem produces surds that are irrational. Furthermore, there is the limitation of how there is no found formula or proof for the area of a heptagon, or octagon or nonagon or any polygon with sides greater than 6. Despite that, it is still conjectured that mathematically a unique polynomial should exist to describe the area of any n -sided shape, but even that isn't rigorously proven. At this current point in time, the maths simply doesn't exist, which poses a problem for my aims of finding the centre of n sides on a map.

On the other hand, I'm exactly where I need to be, if instead of approaching the problem abstractly, I take a more accurate and developed approach. Originally when working on the problem, I viewed it in an abstract way so that the concepts could work conceptually better for me, however now I believe that I want to produce a minimum viable product. When discussing my computer science project with my dad, who works in IT architecture, we spoke about the approaches to the problem, and he said how on an industrial level, trying to create a working product is more valuable than trying to create a perfect product. For the case of this project, after delving into the maths that is involved, it may turn out more effective to constantly build off previous iterations that serve a real-world use rather than abstract one. This is down to two main reasons: firstly, it works better with the agile methodology that I am taking in this computing project, where I will constantly design, create, evaluate and repeat. If I create a product that has real world use each cycle, it will be easier to evaluate for my stakeholder and so I shall be able to further my project further than if the stakeholder fails to fully understand the mathematics of the abstract side. The other reason is how by developing in a non-abstracted fashion I may open new ways and solutions to difficult mathematical issues that I am struggling in, and therefore I am assured that my efforts to create a viable product will be more successful.

Google Maps API:

Hence, here is where I start working my way into the specifics of the Google Maps API. An API (Application programming interface), is simply put something that tells software components how to act and work together, providing a series of protocols and functions that allows for applications to access features or data of an operating system or an application.

So firstly, I had to create a billing account, with the attainment of an API key to access requests to the API. This was a small necessity I had to first accomplish, in which fortunately it doesn't cost any money due to the small scale and lack of requests that I will be sending the API. Then in order to ensure it worked correctly, I had to import the Python-gmaps library, all of which giving me access, to a large range of details I can now work with.

The Python client for the Google Maps Services has the Directions API, Geocoding API, Roads API, amongst several others, all of which give me great control over finding and manipulating coordinate points and locations. The first most useful one of these APIs, is the Geocoding API, as this lets me have a small bit of code, which lets me convert addresses into latitude and longitude, which on a smaller scale, act like coordinates. Latitude and longitude are a geographic coordinate system which allows for every location on earth to be defined by a set of numbers. If I ever wish to take this project further, I can implement elevation, which hence would make this a three-dimensional problem, however for simplicity's sake I am keeping it at two dimensions – especially since elevation could make it so you are inside of a mountain. They are both measured in degrees, latitude having a domain of $-90^\circ \leq \Phi \leq 90^\circ$, and longitude having a domain of $-180^\circ \leq \lambda \leq 180^\circ$. Why this is so useful, is because relative to latitude (or longitude), one degree represents a certain distance.

Due to the spherical nature of the earth, distances of latitude and longitude will vary, however there are two equations that give the length of one degree when latitude/longitude = φ .

Latitude:

$$lat(\varphi) = 111132.92 - 559.82\cos(2\varphi) + 1.175\cos(4\varphi) - 0.0023\cos(6\varphi)$$

Longitude:

$$lng(\varphi) = 111412.84\cos(\varphi) - 93.5\cos(3\varphi) + 0.118\cos(5\varphi)$$

These equations are accurate to the nearest centimetre, however if need be, they can become more accurate. However, what it means is that if we were looking at the distance to travel one degree upwards along the north-south line, at a longitude of 0° , the distance would be approximately 110574.3 m. This will not only work at large distances but also short distances, hence making it the most appropriate method of trying to define absolute distances (or as the crow flies).

So, first step is to obtain the geocoded coordinates from entered addresses. Fortunately, this is relatively simple. Firstly, there are the small things like setting the API key and importing the API client and specifying the geocoding module.

1. `import gmaps`
2. `from gmaps import Geocoding`
- 3.
4. `with open('API-KEY.txt') as f:`
5. `Key = f.readline()`

6. `f.close()`

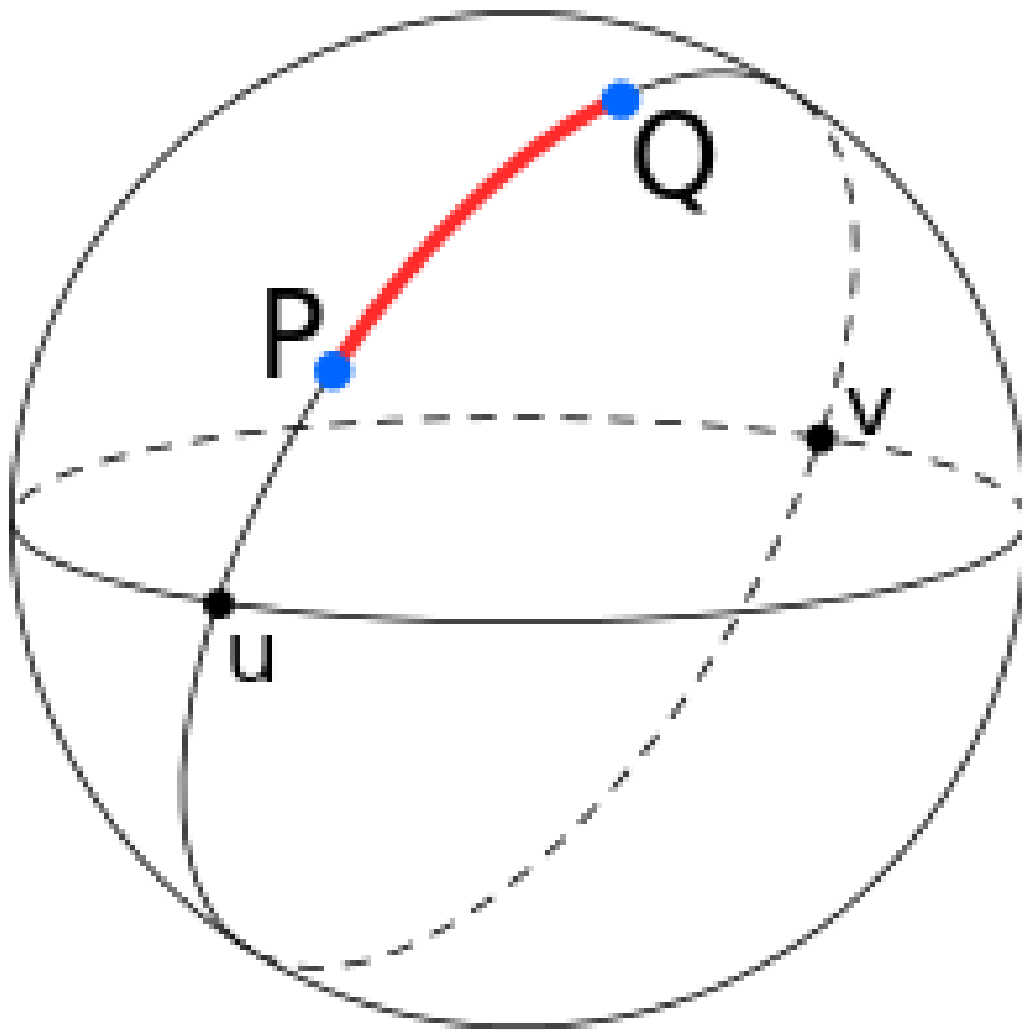
Then, it is as simple as entering an address. The geocode function has several parameters; however, it only actually requires one. The parameters are address, components, region, language, bounds and sensor. The address can be referred to by the street address, the plus code, or the postcode. Optionally you can include components, region etcetera which biases geocode results to certain locations, used in scenarios where addresses have similar if not the same name. For the sake of simplicity and testing though, I am using just the postcode, as it is the easiest to input, and easiest to find. The result of that function is an array, with dictionaries contained inside it. To access what we care about, we access the dictionaries of geometry, then location, and then of latitude and longitude.

```
1. api = Geocoding(api_key = Key)
2. result = api.geocode("Postcode", None, None, None, None, None)
3. lat = result[0]['geometry']['location']['lat']
4. lng = result[0]['geometry']['location']['lng']
5. print(lat, lng)
```

This now in essence means that we have direct access to the coordinate system of the planet earth, meaning we can begin to get more mathematical.

Including curvature:

Upon doing some further research of latitude and longitude, concerning distances and midpoints, we come across a large problem; I've been looking at the problem through the lens of Euclidean geometry, which sets out a large set of rules, that overall boil down to considering everything with straight lines. However, I am looking at spaces with curvature, which changes things drastically.



Above is an image of a sphere, which for the sake of argument we will consider to be the earth. If trying to find the distance between two points, the curvature will greatly impact the results if the angle between the two points from the centre is large enough. Now if we take curvature distance between the two points of P and Q, we can see that it will always be the sector of a great circle, which is one band of the sphere, retaining its radius. This makes the problem of distance to be a question of arc distance which is quite simple being:

$$C = 2\pi r$$

$$\therefore \text{Sector of a circle} = \frac{\theta}{360} \cdot 2\pi r$$

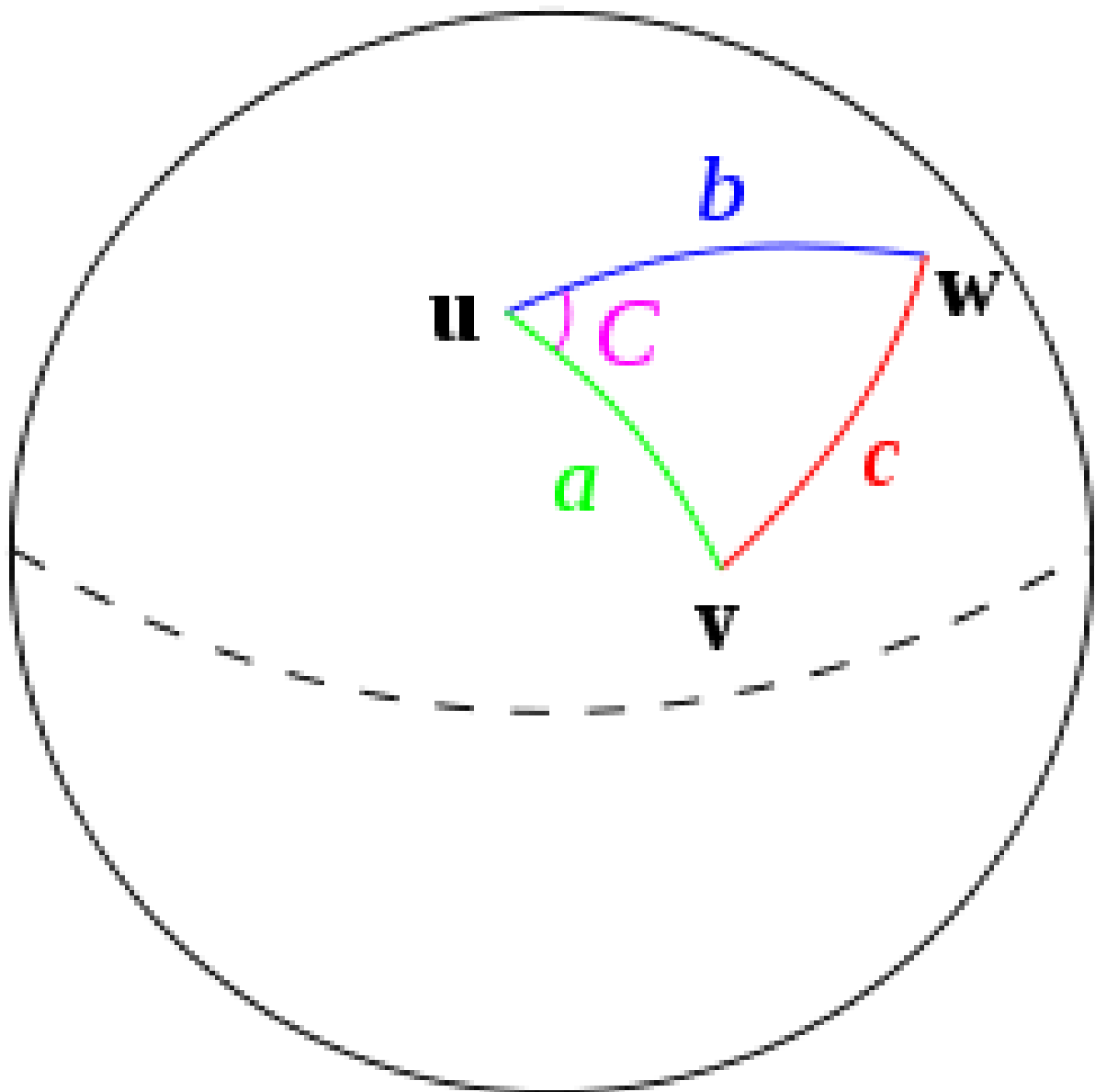
Though the difficulty that arises there is calculating θ . The reason why this is so important to crack is because whilst I can access distance easily, I cannot access the midpoint, which requires precision manipulation of latitudinal and longitudinal points. Now I technically can ignore this for early builds and iterations of my product, because fortunately there is the benefit that this is likely to be used on a much more local level

rather than on a macroscale by my stakeholder, but the limited distance posed on the program certainly makes it less special than other midpoint calculators, and so looking into formulating in spaces of curvature will be something to look toward in the future.

The issue of curvature in spaces, there is a useful bit of mathematics that will greatly help me when looking at distances around a sphere, and that is the haversine formula. What the haversine formula does is it computes the great circle distance with two points, given their longitudes and latitudes. Haversine is the short way of saying half versed sine, which implies how:

$$\text{haversine}(x) = \sin^2\left(\frac{x}{2}\right)$$

However, we can just refer to $\text{haversine}(x)$ as $\text{hav}(x)$ for simplicity's sake. How is this useful though? On its own it's meaningless, but this is where we investigate the spherical laws of cosines. The proof of the law requires rather complex work with vectors, but to try and summarise it, if we take a unit sphere like this:



Then we can manipulate the spherical triangle created by vectors $\mathbf{u}$, $\mathbf{v}$, and $\mathbf{w}$. As the angles and sides do not change as the spherical triangle is rotated, we can position $\mathbf{u}$ at the equivalent of the north pole (spherical coordinates of (0, 0, 0)), then position $\mathbf{v}$ where longitude equals zero (spherical coordinates of (1, a, 0)) and hence $\mathbf{w}$ has it shifted so that it has spherical coordinates of (1, b, C) as seen on the image above. This translates into cartesian coordinates of $\mathbf{v} = (\sin(a), 0, \cos(a))$, and $\mathbf{w} = (\sin(b)\cos(C), \sin(b)\sin(C), \cos(b))$. Then by a lot of vector math which is difficult and confusing you can finally conclude with:

$$\sin(a)\sin(b)\cos(C) = (\mathbf{u} \times \mathbf{w}) \cdot (\mathbf{u} \times \mathbf{v}) \\ = \cos(c) - \cos(a)\cos(b)$$

Which can more neatly be written as:

$$\cos c = \cos b \cos c + \sin a \sin b \cos C$$

Then by making use of trigonometric addition and subtraction formulas and by swapping the side lengths of b and c, the spherical law of cosines can be made to give:

$$\sin^2\left(\frac{d}{2R}\right) = \sin^2\left(\frac{\phi_2 - \phi_1}{2}\right) + \cos \phi_1 \cos \phi_2 \sin^2\left(\frac{\lambda_2 - \lambda_1}{2}\right), \\ \text{haversin}\left(\frac{d}{R}\right) = \text{haversin}(\phi_2 - \phi_1) + \cos \phi_1 \cos \phi_2 \text{haversin}(\lambda_2 - \lambda_1).$$

This is where d = great circle distance, R = the earth's radius, and Φ refers to the latitude of two points, and λ refers to the longitude of the two points. Whilst how this is derived is complicated and difficult for me to explain, its usage isn't, as this gives me a direct gateway for using the geocoded values I can now obtain, as well as accessing spherical distances.

From using the haversine rule, it is possible to derive a formula for the midpoint latitude and longitude, which is a bit more complicated than expected.

$$B_x = \cos \phi_2 \cdot \cos \Delta\lambda$$

$$B_y = \cos \phi_2 \cdot \sin \Delta\lambda$$

$$\phi_m = \text{atan2}(\sin \phi_1 + \sin \phi_2, \sqrt{(\cos \phi_1 + B_x)^2 + B_y^2})$$

$$\lambda_m = \lambda_1 + \text{atan2}(B_y, \cos(\phi_1) + B_x)$$

Where ϕ_m is latitude, and λ_m is longitude (both in radians).

This only works for two points, and if we wanted to visualise it, we could see it as halving the distance between the two points on the great circle, and then finding that middle distance. This opens an opportunity through in the form of finding the solution to the problem of finding the midpoint of three points. See, if we visualise three points on a sphere, and draw great circles between each of them, we can gather that finding a great circle perpendicular to each existing great circle (at the midpoint), the point at which all of them intersect would be the centre of those three points. How to do this though, will require more maths that I currently do not have.

Hence now I can decide upon a plan of action, and what I have figured to do, is to implement a degree of accuracy. With each time n is incremented, there will be a different method utilised to make it so that it can best find the centre location. Each method will have a certain accuracy to them though, and so by finding the percentage of error for each and displaying it to the user, it can indicate how accurate the result is, and if it should be used. This will also be useful to implement for later in the project, for if I have multiple methods to solve for the centre. Each method can be implemented as a function, and the function with the lowest degree of error will be selected to be displayed to the user.

```

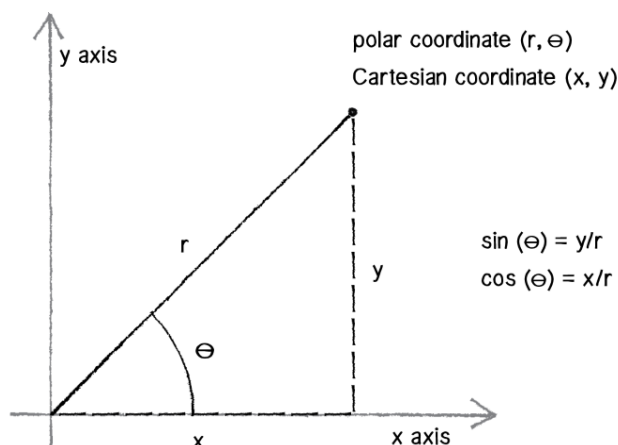
1. def midpoint(lat1, long1, lat2, long2):
2.     Bx = math.cos(lat2) * math.cos(long2 - long1)
3.     By = math.cos(lat2) * math.sin(long2 - long1)
4.     midlat = math.atan2( math.sin(lat1) + math.sin(lat2),
        math.sqrt((math.cos(lat1) + Bx)**2 + By**2))
5.     midlong = long1 + math.atan2(By, math.cos(lat1) + Bx)
6.     return midlat, midlong

```

Above is a small test of the formulae to find the midpoint, making use of the math library in python. The main thing to note is the `math.atan2`, which finds the angle from the positive x axis going counter-clockwise (with the range of $-\pi$ to π).

Polar and Cartesian:

If I was going to stop at only the midpoint of two points, there would be little point in this project, and now I have done enough research, I know how to take the next step. Finding the midpoint of three points on a non-Euclidean plane in theory should be like finding the midpoint of a triangle but using different methods. That was when I learnt that there was a way to convert between polar coordinates (as in latitude and longitude) to cartesian coordinates. This can be best realised in 2 dimensions first.



When looking at the diagram above, you can see that it is a line of length r , with an angle of θ from the positive x-axis. This means that it can be defined by its x and y component

on the cartesian plane. If we setup a triangle shape then, we see we can define x and y in terms of θ and r.

This can similarly be done in a three-dimensional cartesian plane, albeit harder to visualise. Latitude and longitude act as the polar coordinates of the earth, using the average radius of the earth, $R \approx 6371$ km. This means that it is possible to convert between polar and cartesian, opening a wider range of mathematics.

$$\begin{aligned}x &= r \sin \theta \cos \varphi, \\y &= r \sin \theta \sin \varphi, \\z &= r \cos \theta.\end{aligned}$$

Above are the conversions to cartesian, where θ = latitude, and φ = longitude. Similarly, to convert the points back, two other simple formulas need to be known.

$$\begin{aligned}r &= \sqrt{x^2 + y^2 + z^2}, \\ \varphi &= \arctan \frac{y}{x}, \\ \theta &= \arccos \frac{z}{\sqrt{x^2 + y^2 + z^2}} = \arccos \frac{z}{r} = \arctan \frac{\sqrt{x^2 + y^2}}{z}.\end{aligned}$$

Using these I will be able to easily covert back and forth, giving me much more scope for what maths I can apply to the problem. Now that I had this, I was on a roll. See, the process of finding the midpoint of a two-dimensional triangle has two perpendicular bisector lines intersecting; so, I envisioned that in three dimensions on a sphere. It used the idea of great circles from before, where it was the perpendicular intersect of two great circles, but then I realised the nature of a great circle meant that it can be defined as a plane. Say you have three cartesian vectors **O**, **P** and **Q**, defining a plane with those vectors is as simple as preforming the cross product of **OP** and **OQ**. The cross product would avail a vector which each dimension would correspond to a coefficient of the general cartesian equation of a plane: $ax+by+cz=d$.

When looking at our problem, there are three points, but only two that we want to make a plane with, so at first, I was stumped. However, that was when I realised that every great circle is centred around a specific point on the circle: $\langle 0, 0, 0 \rangle$. With that, I had three points that I could create a plane with, and therefore I could begin the next step of finding a perpendicular plane to it.

At this stage we are trying to create a perpendicular plane, where we have access to two points, the midpoint of the great circle and the origin; as well as having access to the equation of a plane perpendicular to it. To find a perpendicular plane with that information, it is relatively simple: you find the line vector between the two points and,

take the cross product of that with the perpendicular vector of the first plane. The new vector produced is the new equation of the perpendicular plane.

If that is done twice, on two sides of the spherical triangle, that gives two intersecting planes that will give two points of intersection for the midpoint. If we were to find the intersections of the planes with the equation of the earth, it would give us a series of infinite answers that wouldn't be that helpful, but if we find the vector equation of the line that follows the intersection of the planes, and then find where that line intersects with the equation of the earth, that should give us the cartesian coordinates of intersection. To solve for the line made by the two planes, we parameterize the two planes by letting $x = t$, and then solving for t in terms of y , and then in terms of z . The result that these outputs can be used if rearranged for in terms t , and then that can be entered into the earth's equation where the goal is solving for t . Once t has been solved, it can be re-entered into the parameterized equations, which can then be converted back to latitude and longitude, finally giving the centre.

Last Points of Research:

That description should be enough to justify why this task is optimized for a computer. To test my maths though, I took some points around the world (which resulted in some easier numbers) and measured the distance between each point and the centre that I found, amazingly finding them to be the same distance, if only out by a small amount (relative to the overall distance). That small amount of error would be a result from the assumed radius of the earth (as it isn't 6371km all around the earth), and due to some rounding and numbers not appearing on the calculator, making the computer even more ideal for the task, able to compute with such large numbers without error. Moreover, my plans to structure the program help play into its advantages, as not only is there a lot of methods repeated with just varying inputs (making functions that would make the program process far more streamlined), but then also using the object-oriented paradigm to define each point with all its necessary details (I.e., cartesian and polar coordinates, address, postcode, etc), would let me access information for each method even easier.

Reaching this point, it becomes a lot clearer why the agile methodology is necessary as it shall let my program keep evolving, improving efficiency, accuracy and ease of use. My current solution is lacking in areas, failing to fully complete what I set out to do originally to find the centre of only n points across the world, as well as the use of Python in IDLE and PyCharm perhaps limiting my GUI, with the imports of Tkinter, Pygame and PyQt maybe not being enough. The solution I've proposed is something that I have found scarce online, that retains mathematical integrity, with a justified and proven method, that can be confirmed by ensuring the equidistance of the points. However, the greatest strength of the proposed solution is how it allows for expansion and development, due to how modular all the processes are.

Fortunately, there are few hardware requirements for my solution, the computer just needing to be able to run PyCharm, and to have access to an internet connection, as a result of needing the Google API. Software requirements on the other hand are a bit different, as the Python IDE must have certain libraries to be able to work. The libraries of gmaps and gmaps geocoding will be required if I am to implement the postcode/address functionality I have planned; and the libraries of math and NumPy to complete the trigonometric functions and cross product of vectors. In addition to them, libraries for a GUI and potentially internet requests for images from google maps; but at this moment, it is hard to determine what is needed for them without the defined code.

Success Criteria:

- Create a GUI that the stakeholder can easily understand and intuitively use
- Create a Point object that can convert between cartesian and polar coordinates with accuracy
- Utilise the Google Maps API successfully, able to consistently convert between latitude and longitude and addresses
- Consistently produce accurate results of the distance between points along the surface of the Earth
- Consistently produce accurate results of the equidistant centre of two and three points
- Able to approximate a centre of four or more points along the surface of the Earth, that are approach equidistance

Pure Mathematics approach:

Throughout the project, I wasn't content to only be able to produce the centre of 2 and 3 points, and I wanted to try to be able to approximate the centre of n points like my initial project suggested, the only problem is, it is degree level mathematics. Whilst currently the approach hasn't gotten me to a solution, it is being included as it not only gave me many helpful insights into mathematics I will need to come to understand, but it will also has helped me in finding improvements in the current algorithm.

First, we take our current model of the Earth, and we know that the arclength along the surface of a sphere can be calculated by:

$$l = r \arctan \frac{|n_1 \times n_2|}{n_1 \cdot n_2}$$

where:

- l is the arclength across the Earth
- n1 and n2 are two points on the surface of the Earth represented as 3D vectors

$$- r \approx 6371$$

From here, I imagined that the attributes of what each vertex and their arclength from the unknown centre may have in relation to each other if they were to be equidistant. This can be best shown with the variance of distances between each vertex and the unknown centre. Variance is defined as:

$$\sigma^2 = \frac{\sum_{i=1}^n (x_i - \mu)^2}{n}$$

But it can be put into a more useful form of:

$$\sigma^2 = \frac{\sum_{i=1}^n x_i^2}{n} - \left(\frac{\sum_{i=1}^n x_i}{n} \right)^2$$

Then, we need to make a minimization formula using a general vertex (V_i) and the unknown centre (C). We can first write arclength between a general vertex and the centre as:

$$\arctan \frac{|V \times C|}{V \cdot C}$$

However, what is important to be noted is that this technically isn't arclength. When comparing to the arclength formula, this has abstracted away the radius. This is because the radius doesn't influence the end result, being pulled out as a factor and made negligible upon optimization. This effectively means minimizing the azimuth angle between vectors, which can be defined at the smallest angle between them from the origin. Next, we need to substitute this azimuth angle formulae into the variance formulae to give:

$$\sigma^2 = \frac{\sum_{i=1}^n \arctan^2 \frac{|V_i \times C|}{V_i \cdot C}}{n} - \left(\frac{\sum_{i=1}^n \arctan \frac{|V_i \times C|}{V_i \cdot C}}{n} \right)^2$$

Then, to minimize this expression, we need to find the derivative of variance with respect to the centre, and then by setting the derivative equal to zero, that would give us the stationary points of the function. This would mean it would give the values of C that are the minimum or maximum variance of the azimuth angle, and its exceedingly difficult to distinguish between the minimum or maximum. Fortunately, the maximum variance of

the azimuth angle in theory should be the point opposite to the minimum variance. By the opposite, this is in reference to the idea of creating a line that passes between the minimum and the origin, and so where that line passes through the sphere again is the point at which the maximum can be found. Then, we could check with the distances from the supposed found centre against the vertices we have, and if greater than the arclength when the angle between the points is 90 degrees, then we take the opposite to be minimum. From here, I had to do the multivariable calculus problem presented to me, and this is where I had to learn a lot of new information.

First, I knew that I was differentiating a scalar (variance) with respect to a vector (unknown centre), which for Vector calculus mean my result would also be a vector. Next, I defined each vertex and the centre as a 3d vector:

$$V_i = [V_{i1}, V_{i2}, V_{i3}]^T$$

$$C = [x, y, z]^T$$

From this point, I wanted to find just the derivative of arclength, as to ensure that I could easily use chain rule in the variance differentiation:

$$l = \arctan(u)$$

$$u = \frac{v}{w}$$

$$v = |V_i \times C|$$

$$w = V_i \cdot C$$

Taking the derivatives of all of these terms had a few steps to them. Starting backwards, taking the derivative of the dot product was the easiest as the process in multivariable calculus is considering the derivative of each step with respect to each dimension, so in this case it would be with respect to x, y and z (with other variables being treated as coefficients when not being differentiated). Hence, it was easy enough to say that:

$$\frac{\partial w}{\partial C} = V_i$$

Taking the derivative of the modulus of the cross product was much harder but was insightful into the mathematics working behind it all. Initially, I found the expanded expression for it, and had to brute force out several derivatives. This resulted in a not very nice-looking vector solution which resembled a system of equations with each dimension

of V_i used as coefficients for the x, y and z values. This was useful as this I had seen before with matrix mathematics. Effectively with matrix multiplication, you can determine that this ends up being a 3x3 matrix consisting of V_i terms (which I am labelling K_i), multiplied by C , as can be seen below:

$$\frac{\partial v}{\partial C} = \frac{1}{|V_i \times C|} K_i C,$$

$$K_i = \begin{pmatrix} V_{i2}^2 + V_{i3}^2 & -V_{i1}V_{i2} & -V_{i1}V_{i3} \\ -V_{i1}V_{i2} & V_{i1}^2 + V_{i3}^2 & -V_{i2}V_{i3} \\ -V_{i1}V_{i3} & -V_{i2}V_{i3} & V_{i1}^2 + V_{i2}^2 \end{pmatrix}$$

These two results allow for me to use the quotient rule to figure out what the derivative of u is, hence, I can find that:

$$\frac{\partial u}{\partial C} = \frac{1}{(V_i \cdot C)^2} \left[\frac{(V_i \cdot C)}{|V_i \times C|} K_i C - V_i |V_i \times C| \right]$$

Then using the known derivative of arctan, and the identity $|V_i|^2 |C|^2 = (V_i \cdot C)^2 + |V_i \times C|^2$, it can be said that:

$$\frac{\partial l}{\partial C} = \frac{1}{|V_i|^2 |C|^2} \left[\frac{(V_i \cdot C)}{|V_i \times C|} K_i C - V_i |V_i \times C| \right]$$

Now with the knowledge that the derivative of arclength can be expressed in terms of V_i , K_i and C , finally, the derivative of variance can be found to be:

$$\frac{\partial \sigma^2}{\partial C} = \frac{2}{n} \sum_{i=1}^n \frac{1}{|V_i|^2 |C|^2} \left[\frac{(V_i \cdot C)}{|V_i \times C|} K_i C - V_i |V_i \times C| \right] \arctan \frac{|V_i \times C|}{V_i \cdot C}$$

$$- \frac{2}{n^2} \left(\sum_{i=1}^n \arctan \frac{|V_i \times C|}{V_i \cdot C} \right) \left(\sum_{i=1}^n \frac{1}{|V_i|^2 |C|^2} \left[\frac{(V_i \cdot C)}{|V_i \times C|} K_i C - V_i |V_i \times C| \right] \right)$$

This beautiful result now should be set to zero, and we can use a similar method to derive the Weiszfeld's algorithm. The way this normally works is by isolating the sum that has C in it, and then pulling it out of the sum (as it is a scalar multiplier), then we can define that sum as an omega matrix, which if we take the inverse of both matrices on both side of the equation, and this results in an iterative solution that will tend toward the centre vector, which should in theory work for n number of points. The problem that arises, is when subbing in an example of values, we attain a $0 = 0$ scenario. Effectively the

inverse matrix and the calculation of the right-hand side of the equation kept giving a 0, which meant that the centre always became the origin. This can be seen in the attempted code for this below. For this, the next method going forward would be using the Newton Raphson method of approximation. The Newton Raphson method that I am familiar with takes place in 2 dimensions, yet very clearly, we are in a 3D plane, so this makes everything much more complicated.

$$\mathbf{x}_{n+1} = \mathbf{x}_n - \mathbf{J}_n^- \mathbf{f}(\mathbf{x}_n)$$

The method is an iterative approach to finding the roots of a function, the values of $\mathbf{x}$ where $\mathbf{f}(\mathbf{x}) = 0$. This is useful as this is what we are trying to do with the derivative, finding the point where the function is zero. Hence, we can implement this solution. The biggest problem found here though, is finding the Jacobian of the function. Due to an inexperience with multivariate calculus, and the function itself being difficult to differentiate, it was not looking likely I would be able to pursue this route. For now, it is on standby, where if the Jacobian can be found, I should be able to implement the solution.

One nice bit of validation of this approach was the minimization of the sum of distances, which is a much nicer algorithm. This tended toward a specific value, not diverging, and consistently when comparing sum of distances relative to other locations, it was by far the smallest. Hence, if I can figure out this method going forward, then the solution to the program will be a lot cleaner and more useful.

One potential solution I have found is Mathematica! Online there is something called Wolfram Cloud which is a way of getting solutions to all kinds of maths. In particular, multivariable calculus. The biggest problem with it, is that I cannot get the simplest form that I need from it, as it returns everything in terms of its 'simplest form' involving everything outside of vector notation. For example, below is a screenshot of the first derivative of just arclength:

```

Vi = {V11, V12, V13}
Centre = {x, y, z}
u = Sqrt[(V12 * z - V13 * y)^2 + (V13 * x - V11 * z)^2 + (V11 * y - V12 * x)^2]
v = V11 x + V12 y + V13 z
l = ArcTan[u/v]
m = FullSimplify[Grad[l, {x, y, z}]]
FullSimplify[Grad[m, {x, y, z}]]

```

Out[139]= {V11, V12, V13}

Out[140]= {x, y, z}

Out[141]= $\sqrt{(-V12 x + V11 y)^2 (V13 x - V11 z)^2 + (-V13 y + V12 z)^2}$

Out[142]= $V11 x + V12 y + V13 z$

Out[143]= $\text{ArcTan}\left[\frac{\sqrt{(-V12 x + V11 y)^2 (V13 x - V11 z)^2 + (-V13 y + V12 z)^2}}{V11 x + V12 y + V13 z}\right]$

Out[144]= $\left\{ \frac{2 (V12 x - V11 y) (V13 x - V11 z) (2 V12 V13 x - V11 V13 y - V11 V12 z) (V11 x + V12 y + V13 z) - 2 V11 ((V12 x - V11 y)^2 (V13 x - V11 z)^2 + (V13 y - V12 z)^2)}{2 (V11 x + V12 y + V13 z)^2 \sqrt{(V12 x - V11 y)^2 (V13 x - V11 z)^2 + (V13 y - V12 z)^2} \left(1 + \frac{(V12 x - V11 y)^2 (V13 x - V11 z)^2 + (V13 y - V12 z)^2}{(V11 x + V12 y + V13 z)^2}\right)}, \right.$

$\frac{(V11 x + V12 y + V13 z) (2 V11 (-V12 x + V11 y) (V13 x - V11 z)^2 + 2 V13 (V13 y - V12 z)) - 2 V12 ((V12 x - V11 y)^2 (V13 x - V11 z)^2 + (V13 y - V12 z)^2)}{2 (V11 x + V12 y + V13 z)^2 \sqrt{(V12 x - V11 y)^2 (V13 x - V11 z)^2 + (V13 y - V12 z)^2} \left(1 + \frac{(V12 x - V11 y)^2 (V13 x - V11 z)^2 + (V13 y - V12 z)^2}{(V11 x + V12 y + V13 z)^2}\right)},$

$\left. \frac{-2 V13 ((V12 x - V11 y)^2 (V13 x - V11 z)^2 + (V13 y - V12 z)^2) + (V11 x + V12 y + V13 z) (2 V11 (V12 x - V11 y)^2 (-V13 x + V11 z) + 2 V12 (-V13 y + V12 z))}{2 (V11 x + V12 y + V13 z)^2 \sqrt{(V12 x - V11 y)^2 (V13 x - V11 z)^2 + (V13 y - V12 z)^2} \left(1 + \frac{(V12 x - V11 y)^2 (V13 x - V11 z)^2 + (V13 y - V12 z)^2}{(V11 x + V12 y + V13 z)^2}\right)} \right\}$

And here is a screenshot of the second derivative:



Yes. As you can see, it only barely fits on the page, even when fully zoomed out, and that's the point. It is entirely unrealistic of me to use this to try and find a formula unless I integrate Mathematica into my program.

Stakeholder Meeting:

At this point in the project, with an issue of where to take my project and what to devote my time into, I went to my stakeholder, my maths teacher Mr Banks, to get an idea on where to go with the project. During our meeting we discussed several facets of the project, involving the design and the next steps, where we came to several useful conclusions.

First, my stakeholder was incredibly supportive of the object-oriented approach in how the points were stored, thinking that the best method to store the data of cartesian and polar coordinates, as well as appreciating the input of addresses to attain locations as well. Furthermore, he took a great deal of interest into the mathematics surrounding the project, as we had lengthy discussions around the degree level minimization approach that I had attempted. He was the one that suggested looking into the Newton-Raphson approach in multiple dimensions after we had recently had a lesson covering the A-Level specification initially. Though most of all the initial and proven to work approach concerning linear algebra methods was the one that was most satisfying to talk through, pulling up large, abstracted diagrams of the Earth, drawing the planes and lines on a whiteboard, and coming to a certain conclusion that approach and method works, with a worked through example of polar (30, 0), (-30, 30), (30, -30) giving (0, 0).

Concerning the next steps, after seeing the mathematical result, me and my stakeholder concluded that the priority should be toward getting a working function, even if the mathematics doesn't yet work. This meant ignoring the pure maths approach at least for the time being. Hence, the project and design going forward will look nearly entirely at

the linear algebra method, the GUI and potentially some additional applications. The stakeholder having a preference to the mathematical side of the project still appreciated the need for more generic usage and concurred with my initial idea of looking in local areas for nearby tourist locations that people can meet at. This will most certainly be accomplished via an API that has a database of nearby locations/tourist attractions if this API can be found.

Therefore, the main take away from this meeting was that until there are working versions, to hold off on the more difficult mathematics, and complete the project to a minimum state, bringing me on to the design.

Design:

Decomposition Diagram:



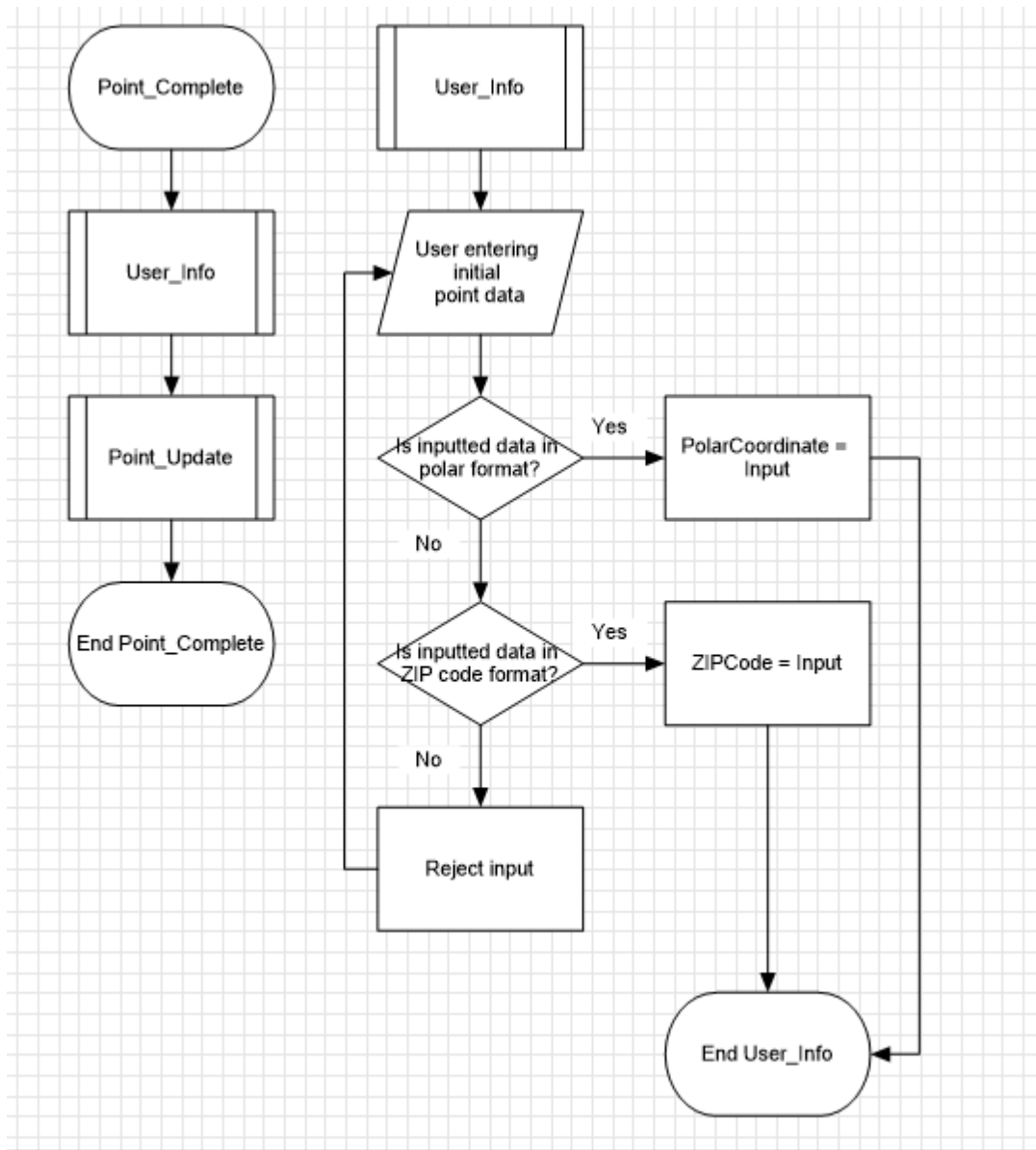
Above is a decomposition diagram, breaking down the program into the necessary components with the OOP approach in mind. This breaks down the major algorithms that I will be using to calculate the centre of up to 3 points. Now when designing each algorithm, I will have to do it in a logical fashion as for example the centre of 3 points requires the algorithm used in the centre of 2 points, which is why I chose to do it in the

object-oriented fashion it will be. For the middle column on the diagram, those are all the functions I am going to write a flowchart for, and then for anything coming after it, pseudocode will be made. This is because a lot of what makes up the left-hand side of the diagram are just using the previously defined functions.

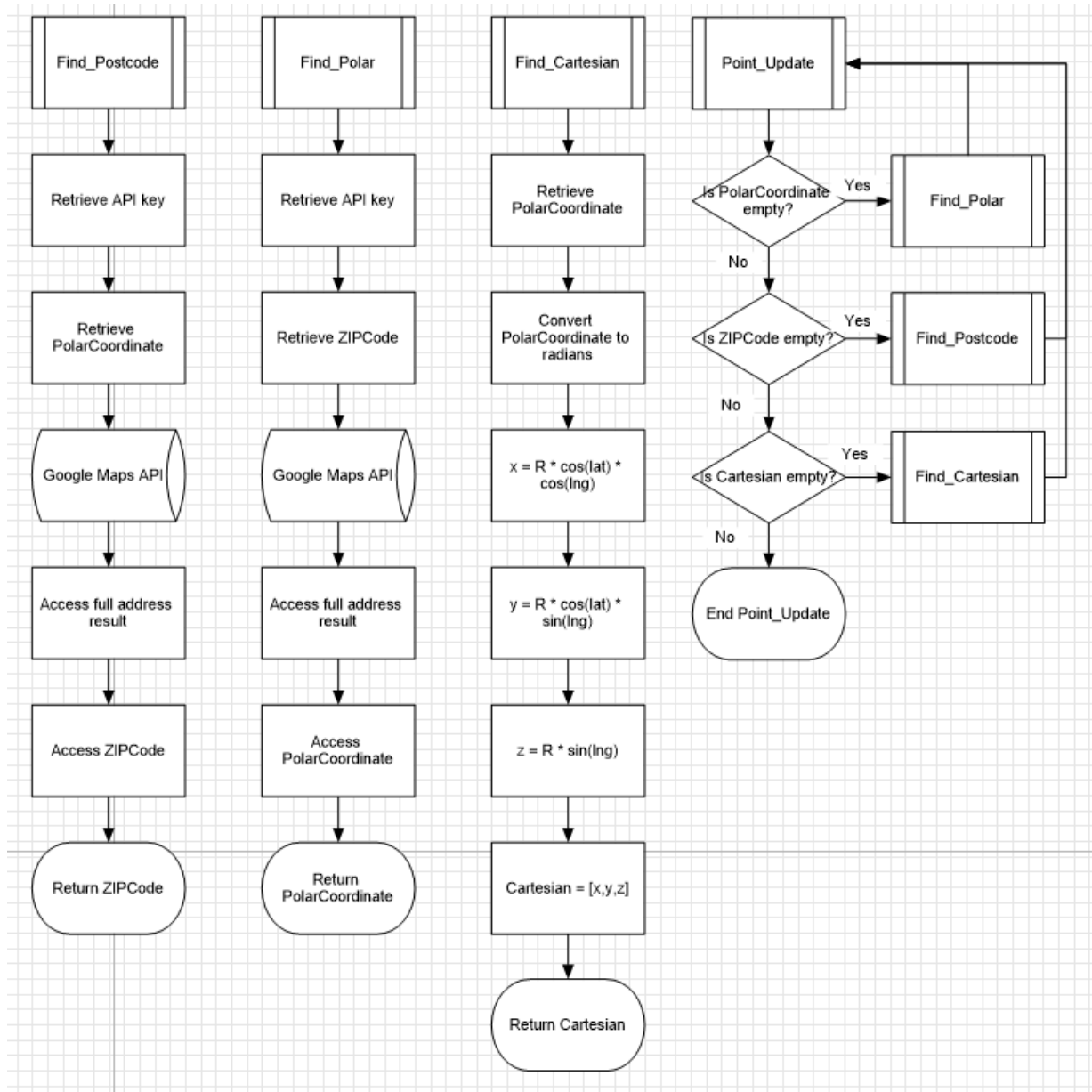
For the pseudocode and flowcharts to be designed below, note that when used in Python, it will be using the libraries of the Google API for geocoding and NumPy for the vector math.

Flowcharts:

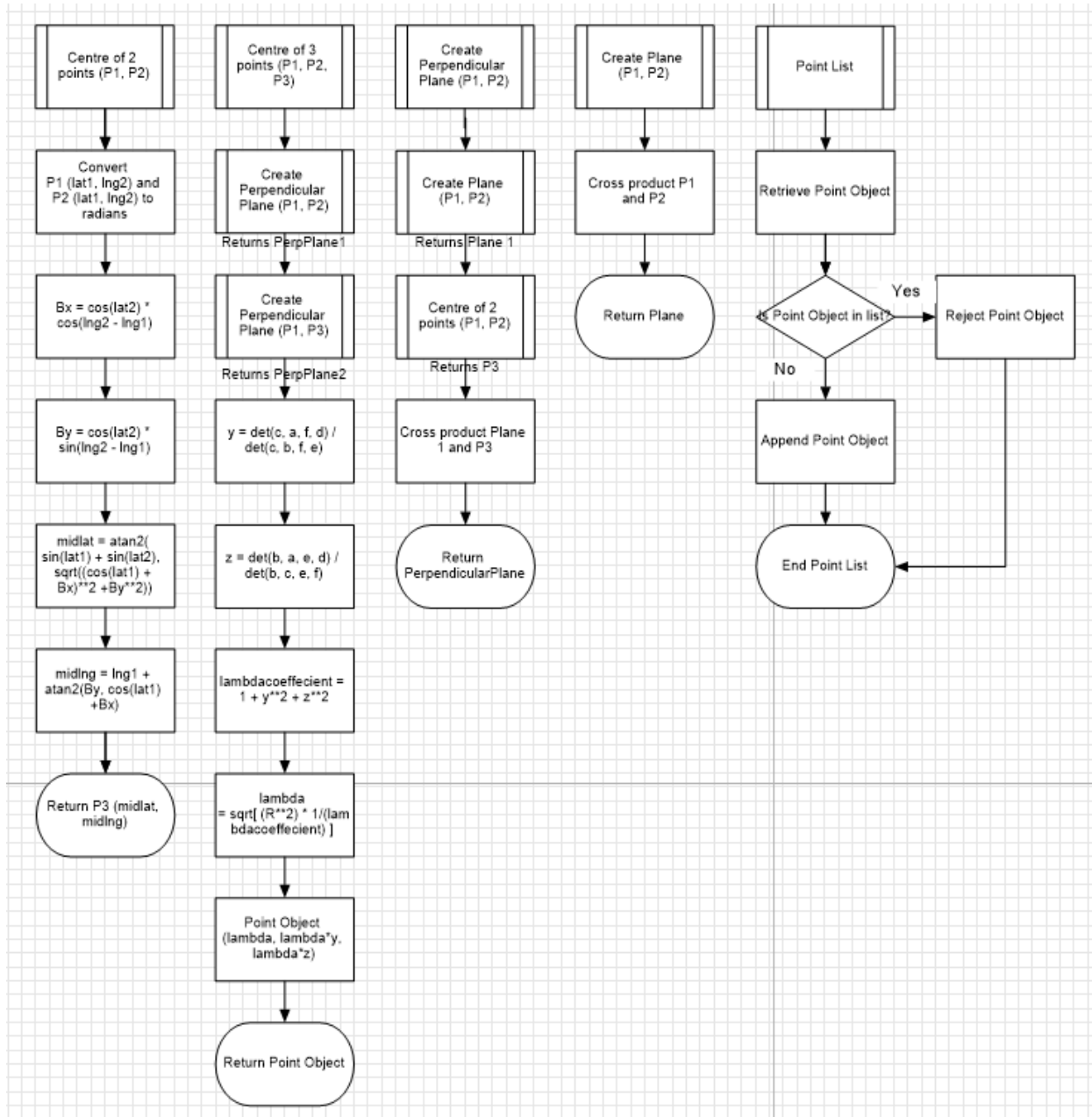
My first plan for designing the project was to define a point and a map object, lending itself well to the problem, reducing a lot of redundancy. This was going to be done by implementing a 'Point_Complete' method. This method would check what data has been inputted into the program and fill out the rest accordingly. For example, if the user inputted in their postcode, the program would identify this and search the latitude and longitude from it. This done so that there can be multiple inputs into the program, such as postcode, or polar coordinates.



From there, the **Point_Update** function would check if each of the attributes of the function to see if they were empty (I.e., false) and hence complete a function to attain the necessary data.



From there, each point object would be defined with all the necessary attributes it would need to be used in the main program. The Map object would have several defined methods which all are used several times throughout more complex programs. This makes it a very modular solution which will help in updating the program to do more than three points in the future.



Initial Pseudocode:

At this point I had optimally written the programs, but that was when I stumbled upon class methods. Class methods are a way of assigning an object certain attributes from a specific input. This was something extremely useful to me as this would be needed for user input and defining the centre of points. So, I scrapped my point update function entirely, and replaced it with 3 class methods that absolve all my issues. The methods: fromCartesian, fromPolar and fromZip do exactly as they say, acting as a constructor from a single input, as seen below, generating Point objects with much less confusion and effort.

1. `class Point`
- 2.
3. `public polar = Vector3[] // [Radius, latitude, longitude]`

```

4.     public cartesian = Vector3[] //[x, y, z]
5.     public zipcode = str()      //I.e B91 3PD
6.
7.     public constructor (userInput)
8.
9.         new cartesian = cartesian
10.        new polar = polar
11.        new zipcode = zipcode
12.
13.
14.
15.    @classmethod
16.    public fromCartesian(cartesian)      //class method
17.
18.        lat = asin(cartesian[2] / R)
19.        lng = atan2(cartesian[1], cartesian[0])
20.
21.        result = api.reverse(lat, lng)    //googlemaps api syntax
22.        api.reverse
23.        zipcode = result[0]['address_components'][7]['long_name']
24.
25.        return (cartesian, [degrees(lat), degrees(lng)], zipcode)
26.
27.
28.    @classmethod
29.    public fromPolar(polar)              //class method
30.
31.        lat = radian(polar[0])
32.        lng = radian(polar[1])
33.
34.        x = R*cos(lat)*cos(lng)
35.        y = R*cos(lat)*sin(lng)
36.        z = R*sin(lat)
37.
38.        result = api.reverse(polar[0], polar[1])    //googlemaps api syntax
39.        api.reverse
40.        zipcode = result[0]['address_components'][7]['long_name']
41.
42.        return ([x, y, z], polar, zipcode)
43.
44.
45.
46.    @classmethod
47.
48.    public fromZip(zipcode)              //class method
49.
50.        result = api.geocode(zipcode)    //googlemaps api syntax
51.        api.geocode
52.
53.        polar = [result[0]['geometry']['location']['lat'],
result[0]['geometry']['location']['lng']]
54.        lat = radian(polar[0])
55.        lng = radian(polar[1])
56.
57.        x = R*cos(lat)*cos(lng)
58.        y = R*cos(lat)*sin(lng)
59.        z = R*sin(lat)
60.

```

61. `return ([x, y, z], polar, zipcode)`

Next to work on were the methods of the core map object, which essentially entailed adding existing points to a list, and then finding the centre of inputted points. This in some ways was simpler to write, as many of the processes are more procedural than anything.

```
1. class Map:
2.
3.     public pointlist = []
4.
5.     public pointlistAdd(point)
6.
7.         pointlist.append(point)
8.
9.         pass
10.
11.    public centre2(point1, point2)
12.
13.        lat1 = radians(point1[0])
14.        lng1 = radians(point1[1])
15.        lat2 = radians(point2[0])
16.        lng2 = radians(point2[1])
17.
18.        Bx = cos(lat2)*cos(lng2-lng1)
19.        By = cos(lat2)*sin(lng2-lng1)
20.
21.        midlat = atan2( sin(lat1) + sin(lat2), sqrt((cos(lat1) + Bx)**2 + By**2))
22.        midlng = lng1 + atan2(By, cos(lat1) + Bx)
23.
24.        midpoint = Point.fromPolar([midlat, midlng])
25.
26.        return midpoint
27.
28.    public centre3(point1, point2, point3)
29.
30.        perp1 = createPerpendicular(point1, point2) //perp1 as in perpendicular1
31.        perp2 = createPerpendicular(point1, point3) //perp2 as in perpendicular2
32.
33.        y_coeffecient = (perp1[2]*perp2[0] - perp1[0]*perp2[2]) /
(perp1[2]*perp2[1] - perp1[1]*perp2[2])
34.        z_coeffecient = (perp1[1]*perp2[0] - perp1[0]*perp2[1]) /
(perp1[1]*perp2[2] - perp1[2]*perp2[1])
35.
36.        t_coeffecient = 1 + y_coeffecient**2 + z_coeffecient**2
37.        t = sqrt(R**2 * (1/t_coeffecient))
38.
39.        //This is the process of solving the simultaneous equations
40.
41.        midpoint = Point.fromCartesian([t, t*y, t*z])
42.
43.        return midpoint
44.
45.    public createPerpendicular(point1, point2)
46.
47.        plane = cross(point1, point2)
48.        centre = centre2(point1, point2)
49.
50.        perpendicular = cross(plane, centre)
```



```

51.
52.         return perpendicular
53.

```

The map class will receive some more detail the further I get into the project, as there are other areas that need to be expanded. First thing to improve is with the way that points are added into the map class, as currently, there isn't a strong argument to make it a class, as it could be a series of functions that pass objects. However, by making it a class I think it will aid in the future with the GUI. For example, the interface could include an image of a map, and then using the pointlist, it would then plot the points.

It was at this point; I took a step back and spoke to my stakeholder. I went to go and explain my algorithm, ensuring that it showed him how the centre point is deduced for each 2 and 3 points, which he agreed with the mathematics. That was when I posed the issue of finding the centre of more than 3 points. There he agreed that they may not defined centre points and suggested two potential methods; one revolving around the minimization of difference of distance (which may use standard deviation as a measure); or using the maximisation of a shape's area, to create a cyclical shape. These will be two decent avenues to explore, but the current program's development is the priority.

The other topic that came up was regarding testing, and this was a big deal as I realised that I couldn't do a basic distance of two points calculator; I would have to implement an arclength calculator. Fortunately, a polar coordinate formula exists for that, making everything far easier.

$$\Delta\sigma = \arccos(\sin\phi_1 \sin\phi_2 + \cos\phi_1 \cos\phi_2 \cos(\Delta\lambda)).$$

$$d = r \Delta\sigma.$$

In the two formulas above, it uses latitude of both points; the change in longitude; and the radius of the Earth, where d is the wanted result of arclength.

```

1.  public arclength(point1, point2)
2.
3.      lat1 = radians(point1[0])
4.      lat2 = radians(point2[0])
5.      deltaLng = radians(point2[1] - point1[1])
6.
7.      centralangle = arccos(sin(lat1)*sin(lat2) + cos(lat1)*cos(lat2)*cos(deltaLng))
8.      arclength = R*centralangle
9.
10.     return arclength

```

This simple method will allow for the checking if the true equidistant value is returned. This would be implemented into the Map class, meaning that at the end of a centre script it can be called for the checking of the accuracy, which may be incredibly useful for testing as well as for improving the approximations for more than 3 points.

In revisiting arclength though, there is a more optimal way to approaching the actual distance and calculating using less computation. This is by using a cartesian form of the

two points:

$$\begin{aligned}\Delta\sigma &= \arccos(\mathbf{n}_1 \cdot \mathbf{n}_2) \\ &= \arcsin|\mathbf{n}_1 \times \mathbf{n}_2| \\ &= \arctan \frac{|\mathbf{n}_1 \times \mathbf{n}_2|}{\mathbf{n}_1 \cdot \mathbf{n}_2}\end{aligned}$$

Class Diagram:

One area I have scarcely touched upon is the interface of the project, which is a lower priority to get working (as it isn't the crux of the project) but it is essential, nonetheless. So, my initial phases of planning it will come in the form of my class diagram. Since a GUI is so dependent on the code you are implementing it around, now that I have generated the pseudocode and flow charts, it is reasonable to think that I can begin to start constructing the interface around it. This first starts in the class diagram below.

The class diagram consists of four classes, where it contains the attributes, methods and the relations between each other (subject to update when the project develops). In blue are the two primary algorithmic classes, that preform most of the mathematical operations to determine the centres, whereas the green classes are the two interface objects that make up the GUI. The MapInterface is what will be showing the location of the points on screen, similarly to Google maps, with the potential to pull images from that website itself, yet currently I haven't done enough research into the GUI to say. However, for the PointInput class, that is a separate textbox that will interpret the users input to see what class method is required to construct the Point object. This will make use of a range of string manipulation techniques, enhanced by pythonic methods. For example, if a string is inputted that resembles an address, then it will use the Point class' fromAddress method. This could be implemented with a try approach, where it tries to get a successful point object from postcode, and if it fails, then it tries polar coordinates (and if that fails it rejects the user input).

Key Variables and Data Structures:

This project consists of a few essential variables that it couldn't function without. Due to most of the methods of other classes revolving around the attributes of the Point objects, it means that the most important variables are found in the Point class. These are polar, cartesian and address. Arguably, cartesian and polar are data structures, taking the form of an array in the program, however this is used to hold the coordinates related to each of those arrays. Therefore, the variables here are x, y, and z (representing directions in 3 dimensions); latitude and longitude (representing two angles around a sphere of 6371km radius centred around the origin); and address, which may only be a nonvalue.

As I experimented with reverse geocoding, seeing what is spat out of the geocoding function when there is no address at a location, I found certain errors would return. When a location only had a place name and no other address, the function returned an out-of-range error, as the index element I was searching for didn't exist. When there was no address at all, there was an error returned that said status = "Error", which was a mystery to solve as the output didn't have a status tag in the dictionary. When researching it some more, the status is something that can be accessed to only when accessing the geocoded result in a difference way, using an online Json file.

Hence, I had to implement a way to pull from the online file and make that my primary method to derive a potential address/latitude and longitude, so that it maintains consistency and ease of understanding when being read. This was done using the requests

and json import, which let me read the website and access the data on the file. The requests library is used to define 'r' which uses requests.get(endpoint) where endpoint is the URL of the website. From there, as the website is a json file, we can use the json library that lets us define json_data with json.loads(r.text). This step is technically unnecessary, as we could navigate the r.text line by line, but it comes with the benefit of cohering to the Google API syntax, as well as making it much easier to traverse through the data. Here, we can pull the status value that we need, with json_data['status']. We can also pull the address or latitude and longitude values with the same method as before but just using json_data rather than result[0].

The status codes of the API are returned with the data, and act as a form of error message. Below is a collection of responses, but ideally, I should only be getting 'OK' and 'ZERO_RESULTS' responses.

Status	Meaning
'OK'	No errors have occurred
'ZERO_RESULTS'	No result was returned, meaning a non-existent address
'OVER_DAILY_LIMIT'	There is an issue with the API key or billing account
'OVER_QUERY_LIMIT'	Over your individual quota
'REQUEST_DENIED'	Request was denied
'INVALID_REQUEST'	Indicates that the query is missing
'UNKNOWN_ERROR'	Request wasn't processed to a server error

When the user is using the program, we don't want the program to error, so if there isn't an address, it must mean that status is giving 'ZERO_RESULT' error. Hence, we can use a quick and easy if statement of the status.

```
60. endpoint = f"{base_url}?latlng={lat}, {lng}&key={Key}"
61. r = requests.get(endpoint)
62. json_data = json.loads(r.text)
63. if json_data['status'] == "OK":
64.     postcode = json_data['results'][0]['address_components'][0]['long_name']
65. else:
66.     postcode = "None"
```

The other major variable that needs to be considered is the user's input. This is quintessential to the project as otherwise the user can't enter the points that they want to. The way I plan on implementing it is with a function that checks the format the input is given in, and from that can pull the necessary data to provide to the point object. There are four conditions it needs to consider: cartesian, polar, address and invalid. Potentially the cartesian condition could be ignored, since nobody would be entering their points with cartesian, and the fromCartesian function is only used in the background by the map functions, however it will be included for testing sake, as so that the cartesian conversion can be demonstrated to my stakeholder.

Originally, I was going to make the detectInput function to be a series of four basic if statements that compares the input to a preexisting format, however that has a large flaw when it comes to addresses. Addresses across the globe have various syntax and trying to account for them all would be near impossible. So instead, the program will assume the input is an address. It will try it in the geocoding function, and if a status of 'OK' is produced, then the assumption is correct. If there isn't an 'OK' status, the method will assume that the input is latitude and longitude, and from there it will check the necessities of the bounds of polar coordinates, and that there are two components. If this returns false, then it will assume the input is cartesian, in which it will do the same thing. If this also returns false, then it will declare the input as invalid and display an invalid message.

There will need to be some testing of same name locations, and work arounds for various inputs of addresses and polar coordinates, such as using NESW for polar; however these are of lower priority to implement in the project, unless the stakeholder enforces its importance.

Test Data:

Testing that the modules of my program work will ensure that the overall sum of components will work for the final product. For now, ignoring the GUI, the main methods that require testing are: fromCartesian, fromPolar, fromZip, centre2, centre3, createPerpendicular, arcLength, and detectInputType. These programs are the ones with the most complexity or the most decision making to do, and therefore testing them against my own calculations/logic diagrams, will be necessary to ensure no errors.

Function	Test data
fromCartesian(cartesian)	[0,0,0] [6371,6371,6371] [6371,0,0] ["a","b","c"]
fromPolar(polar)	[0,0] [30, 30] [30, -30] [90, 180] [91, 181] ["a", "b"]
fromZip(postcode)	School address: B91 3PD City addresses: London, Birmingham, Birmingham UK Landmark addresses: Colosseum, Eiffel Tower, Empire State Building Invalid address: "HGEGEJEG" *note if that is an actual address I will be shocked
centre2(p1, p2)	Two random points: Random number generator; [38, 38], [-11, 171] Home and school: B94 6SA, B91 3PD

	Opposite sides of the earth: [52.353586, -1.781780], [-52.353586, 178.218220] Close together locations: [0, 0], [0.00001, 0]
centre3(p1, p2, p3)	Bermuda, Miami, Puerto Rico Landmark loactions: Colosseum, Eiffel Tower, Empire State Building Invalid locations: I.e “skjdksjsjdkksjj” Close together locations: [0, 0], [0.00001, 0], [0, 0.00001]
createPerpendicular(p1, p2)	Several easy to calculate points Invalid points
arcLength(p1, p2)	Several easy to calculate points: Landmark locations: Colosseum, Eiffel Tower Oppoiste sides of the earth: [52.353586, -1.781780], [-52.353586, 178.218220] Close together locations: [0, 0], [0.00001, 0]
detectInputType(userinput)	Cartesian input Polar input Address input Invalid input

The data that will be tested is shown above, and for some they are currently ‘generic’ examples, as in to say that when it comes to it, it is likely random number generators will be used to get them. This is so that a generic example can be produced. Most methods have a few key test ‘types’: local area input (deals with locations I could actually go to); wide area input (locations that are a much greater distance apart); invalid inputs (for example entering string values into a function using integers); out of bounds inputs (for example a latitude greater than 90 or a longitude greater than 180); non-existent addresses (locations that don’t have an address); close together locations (to test the concerns of floating point logic errors), and opposite sides of the earth (to test the issue of antipodal points).

This testing will be important in refining the program, especially in the arclength function, as some cases such as small distances and antipodal points have formulas that

may be better suited for those cases, such as the haversine formula, or the vector form of the solution.

The agile nature of the project will make deciding on all of the tests at this stage impossible, but these are the core tests for the completion of each major algorithm, and many will remain true for if I manage to develop the centre of 4 or more points. Ensuring that I test small- and large-scale distances, very close locations, antipodal points and invalid locations, will make sure that each calculation-based method will avail the true answer.

Development:

First Iteration:

The first stage in making the program was, very easy.

Libraries and Constants

Since I had done in depth design work with detailed pseudocode, it meant that the transfer of pseudocode to pythonic programming was relatively easy. Using PyCharm as my IDE, my first step was importing all the libraries that I would need. This included: math, NumPy, and Geocoding from gmaps. Next, I initialized the various global variables and constants that would be needed, this included: R, the radius, and API, the permission key so that I could use the gmaps import.

```
67. import math
68. import numpy as np
69. from gmaps import Geocoding
70.
71. #All of the libraries used are at the top of the program
72.
73. with open('API-KEY.txt') as f:
74.     Key = f.readline()
75.     f.close()
76.
77. #The API key is kept in a file so that the sensitive information isn't revealed
   when the project is documented
78.
79. api = Geocoding(api_key = Key)
80. R = 6371
81.
82. #6371 is the rough approximation of the average radius of the Earth in kilometers
```

Point Class:

From there it was implementing the Point function, which in the first iteration only needs to consist of 3 attributes, and 3 method. Its attributes were: cartesian (np.array), polar (array) and postcode (string). The methods bare a similarity to these, each method returning the other values of the object from the one inputted. I.e: fromCartesian takes in cartesian coordinates, and returns polar coordinates and a postcode.

```
83. class Point:
84.     cartesian = np.array([])
85.     polar = []
```

```

86.     postcode = str()
87.
88.     def __init__(self, cartesian, polar, postcode):
89.         self.cartesian = cartesian
90.         self.polar = polar
91.         self.postcode = postcode
92.
93.     #@classmethod is the pythonic way to initialize an object from only one of the
    attributes
94.     #Throughout polar coordinates are in degrees, but before being used by the math
    function they are converted into radian form (as trigonometric functions only take
    radians)
95.     #The result of geocoding gives a large array consisting of several other arrays and
    dictionaries, requiring a long line to traverse through it
96.
97.     @classmethod
98.     def fromPolar(cls, polar):
99.         lat = math.radians(polar[0])
100.        lng = math.radians(polar[1])
101.
102.        x = R * math.cos(lat)*math.cos(lng)
103.        y = R * math.cos(lat)*math.sin(lng)
104.        z = R * math.sin(lat)
105.
106.        result = api.reverse(lat, lng)
107.        api.reverse
108.
109.        zipcode = result[0]['address_components'][7]['long_name']
110.
111.        return cls(np.array([x, y, z]), polar, postcode)
112.
113.     @classmethod
114.     def fromCartesian(cls, cartesian):
115.         lat = math.degrees(math.asin(cartesian[2] / R))
116.         lng = math.degrees(math.atan2(cartesian[1], cartesian[0]))
117.
118.         result = api.reverse(lat, lng)
119.         api.reverse
120.
121.         zipcode = result[0]['address_components'][7]['long_name']
122.
123.         return cls(np.array(cartesian), [lat, lng], postcode)
124.
125.     @classmethod
126.     def fromZip(cls, postcode):
127.
128.         result = api.geocode(postcode)
129.         api.geocode
130.
131.         polar = [result[0]['geometry']['location']['lat'],
    result[0]['geometry']['location']['lng']]
132.         lat = radian(polar[0])
133.         lng = radian(polar[1])
134.
135.         x = R * math.cos(lat) * math.cos(lng)
136.         y = R * math.cos(lat) * math.sin(lng)
137.         z = R * math.sin(lat)
138.

```



```

}          }          },          "place_id" : "GhIJ_____PUAR_____PUA",          "plus_code" : {
"global_code" : "8G2G2222+22"          },          "types" : [ "plus_code" ]          } ],          "status" : "OK"}

```

A lot of this data is abstracted away from my project, my code only retrieving the relevant pieces of information. Formatting remains the same for each result, and we can apply this to the only relevant bits that we need, in this case being the address components, location and status. The best way to access this is through the following lines of code:

```

140.endpoint = f"{base_url}?latlng={lat}, {lng}&key={Key}"
141.r = requests.get(endpoint)
142.json_data = json.loads(r.text)
143.if json_data['status'] == "OK":
144.    postcode = json_data['results'][0]['address_components'][0]['long_name']
145.else:
146.    postcode = "None"

```

What this does is use the requests and json library, and defines a website URL based off of 3 components, the base URL, which is

<https://maps.googleapis.com/maps/api/geocode/json>, then it uses the known latitude and longitude, and my API key, to use the requests library to access the website. As 'r' is an effective json file at this point in its formatting, we can use the json library to load it as json data in which we can easily traverse it to access the status. From there we perform simple logic to see if the address is OK and if it is, we can safely output it to the user, otherwise, we just assume the postcode to be "None" as there is no associated address.

Map Class:

Next the new class of 'Map' was under way. The aim of this function is so that it can house the functions related to multiple points, and in general it is where methods can operate on point objects. At this current stage when I was writing the code a lot of it felt like it could be individual functions not associated with a class, however considering the planned graphical interface it seemed most logical to create it this way, making it easier to work on in later cycles of the project.

```

147.def centre2(self, point1, point2):
148.    lat1 = math.radians(point1.polar[0])
149.    lng1 = math.radians(point1.polar[1])
150.    lat2 = math.radians(point2.polar[0])
151.    lng2 = math.radians(point2.polar[1])
152.
153.    #Latitudes and longitudes are converted into radians and their individual
    coordinate forms as to make the formulae below have far less point1.polar[0], which
    would make it far harder to understand what is actually taking place
154.
155.    Bx = math.cos(lat2) * math.cos(lng2 - lng1)
156.    By = math.cos(lat2) * math.sin(lng2 - lng1)
157.
158.    midlat = math.atan2(math.sin(lat1) + math.sin(lat2), math.sqrt((math.cos(lat1)
    + Bx) ** 2 + By ** 2))
159.    midlng = lng1 + math.atan2(By, math.cos(lat1) + Bx)
160.

```

```

161.     midpoint = Point.fromPolar([math.degrees(midlat), math.degrees(midlng)])
162.     #Another example of trying to keep the standard form of points in degrees.
163.
164.         return midpoint

```

First is the centre of 2 points which worked well with only a few small changes post testing revolving around the changes between degrees and radians.

```

165. def centre3(self, point1, point2, point3):
166.     perpendicular1 = self.createPerpendicular(point1, point2)
167.     perpendicular2 = self.createPerpendicular(point1, point3)
168.
169.     #These perpendiculars are two planes that are perpendicular to the planes
    formed by point1, point2 and the origin; and point1, point3 and the origin.
170.
171.     y_coefficient = -((perpendicular1[0]*perpendicular2[2] -
    perpendicular1[2]*perpendicular2[0]) / (perpendicular1[1]*perpendicular2[2] -
    perpendicular1[2]*perpendicular2[1]))
172.     z_coefficient = -((perpendicular1[0]*perpendicular2[1] -
    perpendicular1[1]*perpendicular2[0]) / (perpendicular1[2]*perpendicular2[1] -
    perpendicular1[1]*perpendicular2[2]))
173.
174.     #This collection of lines is solving the intersection of the two planes,
    parameterising them with the effort of creating the line of intersection in terms
    of x, y and z.
175.
176.     t_coefficient = 1 + y_coefficient**2 + z_coefficient**2
177.     t = math.sqrt((R**2)*(1/t_coefficient))
178.
179.     #These lines then solve the intersection of the parametrized line and the
    sphere, giving a t constant
180.
181.     midpoint = Point.fromCartesian([t, t*y_coefficient, t*z_coefficient])
182.
183.     #This t constant can finally be used to find the cartesian values of the
    midpoint, which can be passed into the fromCartesian function to get the point
    object.
184.
185.     return midpoint
186.
187. def createPerpendicular(self, point1, point2):
188.     plane = np.cross(point1.cartesian, point2.cartesian)
189.     centre = self.centre2(point1, point2)
190.
191.     perpendicular = np.cross(plane, centre.cartesian)
192.     return perpendicular

```

Similarly, due to the extensive planning and preparation from the pseudocode and flow charts the function came out successfully, mostly being linear algebra math using NumPy for the cross product.

```

193. def arcdistance(self, point1, point2):
194.     cross = np.cross(point1.cartesian, point2.cartesian)
195.     dot = np.dot(point1.cartesian, point2.cartesian)
196.     temp = np.linalg.norm(cross) / dot
197.
198.     sigma = math.atan(temp)

```

```

199.     distance = R * sigma
200.
201.     #When getting the arcdistance, the method used was the vector approach, as in
    the end it was less computations for the computer, and worked more consistently
    with more inputs
202.
203.     return distance

```

Next the arcdistance was made, as for a way of testing and debugging. Originally I thought that I would have to implement a modulus formula of a vector, however fortunately NumPy had one readily available in the form of a 'norm' which is supposedly the true mathematical term for the process.

```

204. def distanceCheck(self, pointlist, C):
205.     sum = 0
206.     for i in range(0, len(pointlist)):
207.         sum = sum + self.arcdistance(C, pointlist[i])
208.         print("Distance: " + str(self.arcdistance(C, pointlist[i])))

```

Finally there is this small set of code which is used to do two things: check the the distances from a collection of points to a single point, which in this case is usually the centre, just to check that they are all of equal distance; and to find the sum of distances from that one point to the rest of them. That second feature was to test the success of the pure mathematics approach, which brings me to, the lost programs.

The Lost Programs:

The 'lost' programs are the series of attempts that I made to formulate the centre via the pure mathematics approach. Whilst it feels as if they won't play a large part in the final program, they are still important to include to showcase the efforts made. These programs helped prove certain results which guided me to where I had to go next. The first in particular was my test of finding the center by minimizing the sum of the distances. (To note all functions shown are in the map class).

```

209. def createOmega(self, pointlist, C):
210.     omega = 0
211.     for i in range(0, len(pointlist)):
212.         V = pointlist[i].cartesian
213.         K = np.matrix([[V[1] ** 2 + V[2] ** 2), -1 * V[0] * V[1], -1 * V[0] *
V[2]],
214.                        [-1 * V[0] * V[1], (V[0] ** 2 + V[2] ** 2), -1 * V[1] *
V[2]],
215.                        [-1 * V[0] * V[2], -1 * V[1] * V[2], (V[0] ** 2 + V[1]
** 2)]]])
216.         scalar1 = 1 / ((np.linalg.norm(C) ** 2) * (np.linalg.norm(V) ** 2))
217.         scalar2 = np.dot(V, C) / np.linalg.norm(np.cross(V, C))
218.         omega = omega + scalar1 * scalar2 * K
219.
220.         #This function creates the summation coefficient of C (omega), which we
    will be taking the inverse of
221.
222.     return omega
223.
224. def createDelta(self, pointlist, C):
225.     delta = 0
226.     for i in range(0, len(pointlist)):
227.         V = pointlist[i].cartesian
228.         scalar1 = np.linalg.norm(np.cross(V, C)) / ((np.linalg.norm(C) ** 2) *
(np.linalg.norm(V) ** 2))
229.         delta = delta + scalar1 * V

```

```

230.
231.         #This defines the other sum used, in which it is labelled as delta.
232.
233.         return delta
234.
235. def centreN(self, pointlist, C):
236.     omega = self.createOmega(pointlist, C.cartesian)
237.     delta = self.createDelta(pointlist, C.cartesian)
238.
239.     #Omega and delta are generated separately as to avoid confusion with the
    varying sums taking place
240.
241.     invomega = np.linalg.inv(omega) #This takes the inverse of omega, for the
    matrix calculation
242.     centre = np.dot(invomega, delta).tolist()[0] #This does the dot product of
    inverse omega and delta, then setting it equal to a numpy array.
243.
244.     return Point.fromCartesian(centre)

```

Which is derived from this process: (Taking the partial derivative of l with respect to C)

$$\sum_{i=0}^n \left(\frac{1}{|V_i|^2 |C|^2} \left[\frac{(V_i \cdot C)}{|V_i \times C|} K_i C - V_i |V_i \times C| \right] \right) = 0$$

$$\sum_{i=0}^n \frac{1}{|V_i|^2 |C|^2} \frac{(V_i \cdot C)}{|V_i \times C|} K_i C = \sum_{i=0}^n V_i |V_i \times C|$$

$$\left(\sum_{i=0}^n \frac{1}{|V_i|^2 |C|^2} \frac{(V_i \cdot C)}{|V_i \times C|} K_i \right) C = \sum_{i=0}^n V_i |V_i \times C|$$

$$\Omega C = \delta$$

The reason this bit of code takes such significance is because it managed to produce a result that didn't have an equal distance from all points but did have the smallest sum of distances. It worked accurately where I was certain the point produced was the point that had the smallest distance produced, which meant if something similar for the variance formula could be replicated, I would be on the way to a general solution.

```

245. def s1(self, V, C):
246.     s1 = 1 / ((np.linalg.norm(V.cartesian) ** 2) * (np.linalg.norm(C.cartesian) **
    2))
247.     return s1
248.
249. def s2(self, V, C):
250.     s2 = np.dot(V.cartesian, C.cartesian) / np.linalg.norm(np.cross(V.cartesian,
    C.cartesian))
251.     return s2
252.
253. def s3(self, V, C):
254.     s3 = np.linalg.norm(np.cross(V.cartesian, C.cartesian))
255.     return s3
256.

```

```

257. def K(self, point):
258.     X = point.cartesian
259.     K = np.matrix([[X[1]**2 + X[2]**2, -1 * X[0] * X[1], -1 * X[0] * X[2]],
260.                    [-1 * X[0] * X[1], (X[0]**2 + X[2]**2), -1 * X[1] * X[2]],
261.                    [-1 * X[0] * X[2], -1 * X[1] * X[2], (X[0]**2 + X[1]**2)]])
262.     return K
263.
264. #These first four functions defined each problem to their core roots, where they
    made up each repeatedly used expressions
265.
266. def l(self, V, C):
267.     cross = np.cross(V.cartesian, C.cartesian)
268.     dot = np.dot(V.cartesian, C.cartesian)
269.     temp = np.linalg.norm(cross) / dot
270.
271.     l = math.atan(temp)
272.     return l
273.
274. def sum1(self, pointlist, C):
275.     sum = 0
276.     for i in range(0, len(pointlist)):
277.         V = pointlist[i]
278.         temp = self.l(V, C) * self.s1(V, C) * self.s3(V, C) * V.cartesian
279.         sum = sum + temp
280.
281.     sum = sum * len(pointlist)
282.     return sum
283.
284. def sum2(self, pointlist, C):
285.     sum1 = 0
286.     sum2 = 0
287.     for i in range(0, len(pointlist)):
288.         V = pointlist[i]
289.         temp1 = self.l(V, C)
290.         sum1 = sum1 + temp1
291.
292.     for i in range(0, len(pointlist)):
293.         V = pointlist[i]
294.         temp2 = self.s1(V, C) * self.s3(V, C) * V.cartesian
295.         sum2 = sum2 + temp2
296.
297.     sum = sum1 * sum2
298.     return sum
299.
300. def sum3(self, pointlist, C):
301.     sum = 0
302.     for i in range(0, len(pointlist)):
303.         V = pointlist[i]
304.         K = self.K(V)
305.         temp = self.l(V, C) * self.s1(V, C) * self.s2(V, C) * K
306.         sum = sum + temp
307.
308.     sum = sum * len(pointlist)
309.     return sum
310.
311. def sum4(self, pointlist, C):
312.     sum1 = 0
313.     sum2 = 0

```

```

314.     for i in range(0, len(pointlist)):
315.         V = pointlist[i]
316.         K = self.K(V)
317.         temp1 = self.l(V, C)
318.         temp2 = self.s1(V, C) * self.s2(V, C) * K
319.         sum1 = sum1 + temp1
320.         sum2 = sum2 + temp2
321.
322.     sum = sum1 * sum2
323.     return sum
324.
325. def centreN(self, pointlist, C):
326.     print("s1", self.sum1(pointlist, C))
327.     print("s2", self.sum2(pointlist, C))
328.     print("s3", self.sum3(pointlist, C))
329.     print("s4", self.sum4(pointlist, C))
330.
331.     #In the end I was printing all of the values, as to try and figure out what was
    actually happening in the sum
332.
333.     vector = self.sum1(pointlist, C) - self.sum2(pointlist, C)
334.     omega = self.sum3(pointlist, C) - self.sum4(pointlist, C)
335.     #In the end both vector and omega were zero values, making this function
    unusable
336.     inverse = np.linalg.inv(omega)
337.
338.     centre = np.dot(inverse, vector)
339.     return Point.fromCartesian(centre.tolist())

```

This formula was found in a similar way to the last using the derived variance formula. However, had the unfortunate outcome of a 0=0 result, which suggested that the results cancelled each other out before it could get any further.

$$\Omega C = \delta \frac{V_i \cdot C}{|V_i|^2 |C|^2} V_i |V_i \times C| - \frac{2}{n^2} \left(\sum_{i=1} \arctan \frac{|V_i \times C|}{V_i \cdot C} \right) \left(\sum_{i=1} \frac{1}{|V_i|^2 |C|^2} V_i |V_i \times C| \right)$$

The quality of the code I admit is poor, which is mostly a result of wanting to test that it actually works mathematically, as this wasn't the first iteration of this attempt. As will be seen in the eventual appendix there were three main iterations of this same program, with this one being the clearest and easiest to follow as everything was defined by a unique function.

First Iteration Review:

The goals that I had set out to achieve for the first iteration were rather simple: to implement the computational logic and algorithms that were at the basis of this project. This was to see:

- Point object and all associated methods
- Map object and all associated methods
- Point object can be made from an address, polar coordinates, and cartesian coordinates
- Centre of two points can be found
- Centre of three points can be found

And from speaking to my stakeholder, all of those points were considered a success. My stakeholder went to test several of his own latitude and longitude values, as well as certain address values. Notably he tested for the midpoint of (30, 0), (-30, -30), (-30, 30),

which as expected returned (0, 0), which was a nice visual test of the accuracy. Next there was also a general test to see what was returned when putting in two locations and three locations put in as general addresses. This was done first with the test data of “London” and “Birmingham, UK” where the UK is specified as to ensure that it doesn’t confuse for the Birmingham in the US. This returned the polar coordinates of [52.00008517234239, -0.9994435472382416], which after performing the function of distanceCheck saw that it was equally 81.2524376849831km away from the two previous points inputted. Similarly, when adding in a third point of “Manchester” we get the polar coordinates of [53.3396464785486, 1.051059376818928] and an equal distance of 218.853477479621km. This wasn’t perfect though as Mr Banks pointed out, with the distances shown having small differences of being roughly 1×10^{-13} km out from each other. This means they are off by 1×10^7 mm = 100 picometers, which to put into perspective, atoms are 62-520 picometers in diameter, meaning my program is correct to an atomic level.

With that we concluded that the first iteration was successful, it is dropping off accuracy being a result of the computer’s rounding as unfortunately the values given throughout the algorithm aren’t accurate to infinite decimal places – since it’s an irrational number given. However, my stakeholder didn’t find much issue in inaccuracy of an atom, and so we began discussing the next steps of the project.

There were two major areas that the spoke of; firstly, a way to make the program more interactive and accessible. Currently it consisted of text and having to edit the program’s source code to write the locations and points. This GUI would have to be something more visual so that people had a better idea of where the centre ended up being, so I would have to find a way to load in a manipulatable image so the user can see where they are roughly ending up. Additionally, I would need to have textboxes for one, two or three points entered, each acting accordingly depending on however many are filled. I was also going to need to control what the users entered, ensuring that there isn’t a program crash upon poor inputs, and rather the program halts.

By making the program visual it would improve the experience for the user tremendously, in addition to providing some real-world context and application to the problem, as technically in a way, currently this program only finds the point on the sphere, which is meaningless. That’s where the second major point was brought up, in respect to what the program should try to do, and that is find a way to locate the nearest valid address and location and direct the user there, which upon looking into, was far harder than I anticipated.

Second Iteration:

In completing the first iteration I now have the direction as to where to go with the second major iteration. The main focus here is on the graphical interface in a way which will let the user to more easily visualise the area of the centre. This will need to be able to do a few things:

- To have 3 textboxes that can be typed in
- Once enter has been pressed it adds the number of points entered to the point list of the map
- To load an image of the current location
- To be able to zoom in and out using the mouse scroll
- To be able to change the look of the map from terrain, roads and satellite imaging
- Address recognition function (returns error if doesn’t work)

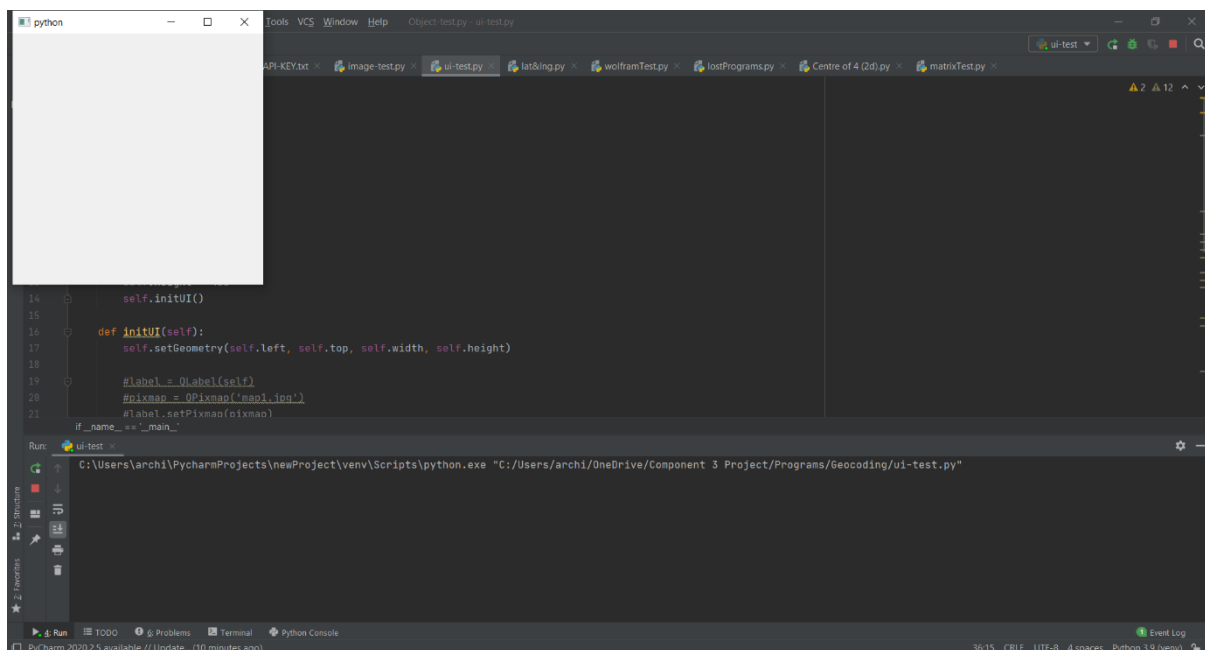
Learning PyQt5

As previously established, I will be using PyQt5 as the library to assist in creating the graphical interface. This was chosen as it had the most intuitive design for what I am aiming to create. As opposed to Pygame or Tkinter, there is far less moving/animated parts to the interface, which means that I am prioritising an easy to manipulate layout and design, which PyQt5 does most successfully. The first step to this was learning how to create a window from PyQt.

```

340. from PyQt5.QtWidgets import *
341. import sys
342. class App(QWidget):
343.
344.     def __init__(self):
345.         super().__init__()
346.         self.left = 0
347.         self.top = 0
348.         #These define where the window will appear on screen
349.         self.width = 400
350.         self.height = 400
351.         #These define the size of the window (in this case a 400 by 400 pixel
352.         square)
353.         self.initUI()
354.
355.     def initUI(self):
356.         self.setGeometry(self.left, self.top, self.width, self.height)
357.         self.show()
358.         #This method is used to instantiate features of the UI, currently there are
359.         none other than the window
360.
361. if __name__ == '__main__':
362.     app = QApplication(sys.argv)
363.     ex = App()
364.     sys.exit(app.exec_())
365. #These final 4 lines are used to execute the program and produce the window

```



Above is the output of the code, and as we see it follows what has been inputted as the appropriate geometry of the program, with a 400 by 400 square window in the top left of the screen, I.e. (0, 0). So, from here we can freely manipulate the size and dimensions of the window and finding the optimal size will be important for improving the use of the program to the user.

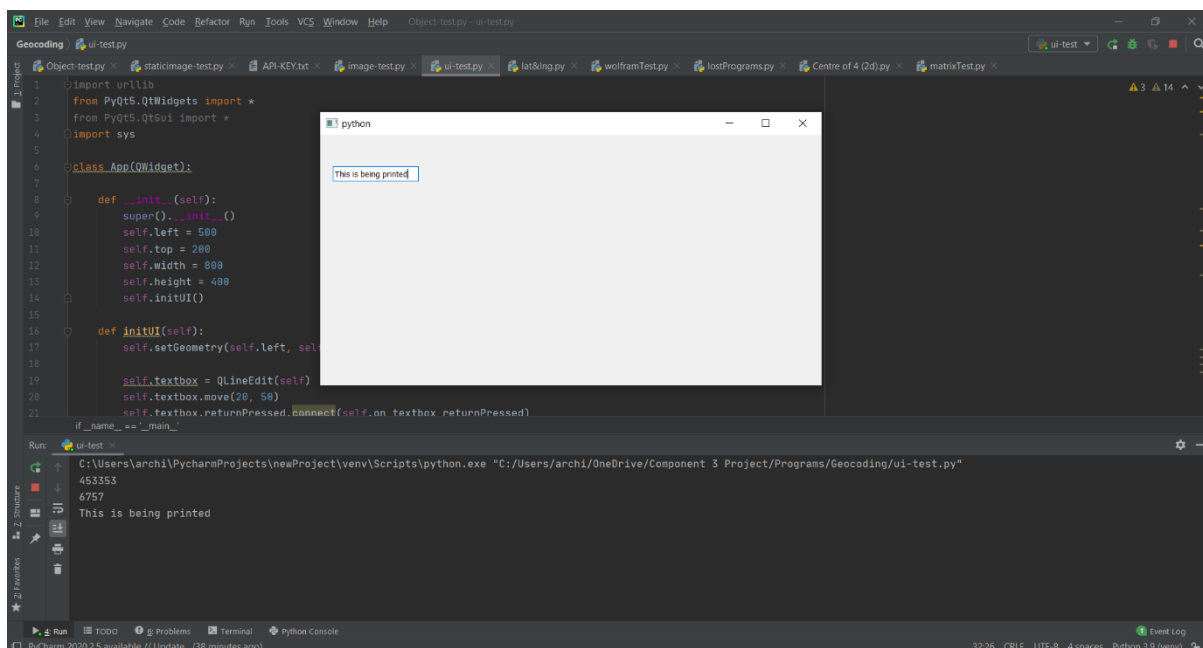
From here I researched how to create a text box which was quite simple in being 3 lines of code in the `initUI` function:

```
366. self.textbox = QLineEdit(self)
367. self.textbox.move(20, 50)
```

The text box itself is named `textbox` and the `move` method is used as by default the `textbox` is present at (0, 0), which means that it is moved to the appropriate position of 20 pixels to the right, and 50 down. What's important to remember is that PyQt assumes down is positive y, and right is positive x, so these must be kept in mind when constructing the interface.

From the `textbox` being generated, we need it to pull the contents of the `textbox` when 'enter' is pressed. For this we want to 'connect' a function to the `textbox`, where the function will contain what we want to happen once enter has been pressed.

```
368. def initUI(self):
369.     self.setGeometry(self.left, self.top, self.width, self.height)
370.
371.     self.textbox = QLineEdit(self)
372.     self.textbox.move(20, 50)
373.     self.textbox.returnPressed.connect(self.on_textbox_returnPressed)
374.
375.     self.show()
376.
377. def on_textbox_returnPressed(self):
378.     print(self.textbox.text())
```



Now the program above dictates that when enter has been pressed, the contents of the `textbox` will be accessed, and will be printed in the python console. This can be seen above in the console with the text 'This is being printed' being in the console and the `textbox` after having pressed enter. Now what we want to do is have the program take the text found and use it to generate a point object from it. The way this will be done will be a mixture of regex, try and except, and a usage of the status feature used in the first iteration.

String Manipulation:

The first step is to check if the input is written in terms of latitude and longitude. This is best done with the library 're', which refers to regex. What regex does is it a tool for string manipulation and identification, with one use being to check if aspects of a string fall between certain parameters. This has us generate a regex expression that if done correctly will allow us to easily identify what to do in the program. Using the website, <https://regex101.com/> it was a good way to practise certain expressions however the beauty of the internet allowed me to pull a solution to the problem I was looking for:

```
^[+-]?([1-8]?\d(\.\d+)?)|90(\.0+)?),\s*^[+-]?(180(\.0+)?|((1[0-7]\d)|([1-9]?\d)))(\.\d+)?)$
```

This expression effectively checks if the entered string consists of xx, xxx, where latitude is between -90 and 90, longitude is between -180 and 180, and when entered the two are separated by a comma. If the string is within the parameters of the expression, it is then accepted using '.match'. This gets rid of any risk of inaccurate values being entered and interpreted, as to not waste the user's or the program's time.

If the input is a 'match' then the program will return a point object using fromPolar. If it isn't then we need to make usage of the fromZip function. Here we assume that if the input isn't polar coordinates, it must be a real-world address, which we should just be able to search. However, if no address is found, how are we able to identify it? For this we can use a cheat of sorts. As fromZip is a function, it means that it can return different things dependent on the situation, which means it doesn't have to return a point object. Hence, I rewrote the fromZip function to return a Boolean, false, if the status is not equal to "OK". I also changed the name to fromAddress because I realised zip is less of universal term and far vaguer compared to address.

```
379. @classmethod
380. def fromAddress(cls, postcode):
381.
382.     endpoint = f"{base_url}?address={postcode}&key={Key}"
383.     r = requests.get(endpoint)
384.     json_data = json.loads(r.text)
385.
386.     if json_data['status'] == "OK":
387.         lat = json_data['results'][0]['geometry']['location']['lat']
388.         lng = json_data['results'][0]['geometry']['location']['lng']
389.         polar = [lat, lng]
390.
391.         x = R * math.cos(lat) * math.cos(lng)
392.         y = R * math.cos(lat) * math.sin(lng)
393.         z = R * math.sin(lat)
394.
395.         return cls(np.array([x, y, z]), polar, postcode)
396.     else:
397.         return False
```

This code takes full use of the json data referenced earlier in the development, and now the full code used in the final program can be seen. Regardless, the important bit here is the return false if json_data['status'] != "OK", which means in the on_textbox_returnPressed function if there is no associated address, it will pick up on it

being false and be able return an error message. For now, this will be a simple output in the console of “Error”, but once the full interface has been made, I will have a small bit of text appear in red saying “No location could be found.” The full code for this can be found below:

```
398. def on_returnPressed(self):
399.     polarCheck = re.compile("^[+-]?([1-8]?\\d(\\.\\d+)?|90(\\.0+)?),\\s*[-+]?([180(\\.0+)?|((1[0-7]\\d)|([1-9]?\\d))(\\.\\d+)?)$")
400.     print(self.textbox1.text())
401.     if polarCheck.match(self.textbox1.text()) != None:
402.         polar = self.textbox1.text().split(", ")
403.         P = Point.fromPolar([int(polar[0]), int(polar[1])])
404.         self.map.pointlist.append(P)
405.         self.map.center = self.map.pointlist[0]
406.         self.newImgLoad()
407.     elif Point.fromAddress(self.textbox1.text()) != False:
408.         self.map.pointlist.append(Point.fromAddress(self.textbox1.text()))
409.         self.map.center = self.map.pointlist[0]
410.         self.newImgLoad()
411.     else:
412.         print("error")
```

Now one bit of this code that I haven’t explained is the newImgLoad and everything happening with the map. This makes more sense with the context of the images.

PyQt Images:

When loading in an image, it makes use of the PyQt widget functionality which will use these lines of code in the initUI function:

```
413. self.label = QLabel(self)
414. pixmap = QPixmap('map1.jpg')
415. self.label.setPixmap(pixmap)
416. self.label.move(500, 0)
```

This uses the widget of QPixmap, where if a jpeg is in the same file, it can be called, and the image can be set. This image is an image we will want to pull from online using the google maps API, by using the static image functionality. The static image functionality of the google API lets you create a link to an image centred at any point you want, with any design you want, with markers placed down. The way the link works is using this base URL: <https://maps.googleapis.com/maps/api/staticmap?parameters> Except we replace parameters with the details of the static map we want. These parameters we want are:

- centre (given in latitude and longitude)
- zoom (integer value)
- size (integer width x height in pixels)
- format (string to determine file type (default jpeg))
- maptype (4 possible strings: roadmap, satellite, hybrid, terrain)
- markers (string with specific formatting determining location and style)
- key (the API key to allow for permissions)

Some examples can be seen below with the respective links, just not including the API key:

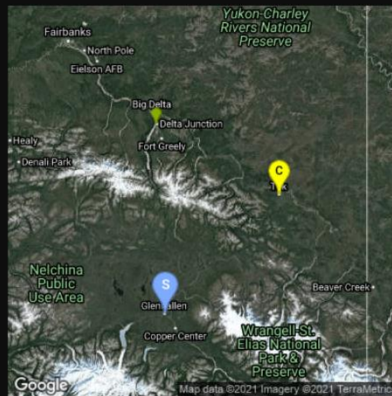
https://maps.googleapis.com/maps/api/staticmap?center=London&zoom=14&size=400x400&key=API_KEY



https://maps.googleapis.com/maps/api/staticmap?center=40.714728,-73.998672&zoom=12&size=800x400&maptype=satellite&key=API_KEY



https://maps.googleapis.com/maps/api/staticmap?center=63.259591,-144.667969&zoom=6&size=400x400&markers=color:blue%7Clabel:S%7C62.107733,-145.541936&markers=size:tiny%7Ccolor:green%7CDelta+Junction,AK&markers=size:mid%7Ccolor:0x000000%7Clabel:C%7CTok,AK%22&maptype=hybrid&key=API_KEY



Now if we define a URL variable to equal the string, we can then use another library to our advantage. The library in question is `urllib`, which allows us to use the procedure: `urlretrieve()`, which takes two parameters: the URL and the name we want to call the file retrieved. In the case of my program, I continue to call it `'map1.jpg'`. The way this procedure works is that it creates a jpeg with the image disclosed in the link and puts it in the same file. If a file with its name is already existing then it overrides it, making it very useful as whenever we want to update the image, we can just call this function with the new link, which is where the function `newImgLoad` comes into play.

```
417. def newImgLoad(self):
418.     self.map.returnImage()
419.     pixmap = QPixmap('map1.jpg')
420.     self.label.setPixmap(pixmap)
421.     self.label.update()
```

Using `.update()` it refreshes the label in the window which contains the new image, so now we can consistently update the image however we want. This will be incredibly useful for the user interface as I can keep updating so the user can use this more than once without having to close anything down. However, it also plays a big role in how the zoom feature will work.

Markers:

So far, we have implemented a lot of the features planned for the second iteration, however we need to include these features in an effective and computational way, which will reduce unnecessary complications in the future. The way this will be done is by adding to the map class. Before this point the map class only contained a single attribute of a list of the points, however this is where we will now store the details of the image rendered as well. The main features that will change on the map will be the centre on the image, which will be a point object, the `maptype` which will just be a string, and the zoom of the image. Since we want to change the zoom of the image easily using the

mouse, we want to make it an easy to change variable, which can be done quite easily. If we now implement a function that will return an image based on the map object's attributes, we can easily repeatedly call that function as we change the object's attributes. Additionally, we can make use of another function of addMarker, which will add another marker of a point we designate on to the URL, which is used below to add all of the points from the pointlist.

```
422. def returnImage(self):
423.     if len(self.pointlist) == 0: #no markers
424.         self.url =
            f"http://maps.googleapis.com/maps/api/staticmap?center={str(self.center.polar[0])},
            {str(self.center.polar[1])}&size=600x600&key={Key}&zoom={str(self.zoom)}&maptype={s
            elf.maptype}&sensor=false"
425.     else:
426.         self.url =
            f"http://maps.googleapis.com/maps/api/staticmap?center={str(self.center.polar[0])},
            {str(self.center.polar[1])}&size=600x600&key={Key}&zoom={str(self.zoom)}&maptype={s
            elf.maptype}&markers=color:blue%7Clabel:S%7C{str(self.center.polar[0])},{str(self.c
            enter.polar[1])}&sensor=false"
427.     for V in self.pointlist:
428.         self.addMarker(V)
429.
430.     urllib.request.urlretrieve(self.url, "map1.jpg")
```

Zooming:

This procedure has all of the details in one place, so that when I will optimize the look of the UI, I can easily change some of these attributes of the image. More importantly though there is a distinction for when the pointlist is empty, as that is when we don't want any markers on the map and want to return to a default, which is centred at (0, 0). Now, we can find ways on the UI to manipulate the attributes of the map object. The first one I went about doing was the zoom function. This would allow for the zoom to be changed with a scroll wheel, which can be accessed via the wheel event that PyQt5 uses.

```
431. def wheelEvent(self, event: QWheelEvent):
432.     delta = event.angleDelta().y()/120
433.     if self.map.zoom > 0:
434.         self.map.zoom = int(self.map.zoom + delta)
435.     elif self.map.zoom == 0 and delta == 1:
436.         self.map.zoom = int(self.map.zoom + delta)
437.     elif self.map.zoom < 0:
438.         self.map.zoom = 0
439.     self.newImgLoad()
```

Keep in mind that when instantiating the Map object in the initUI function, we simply call it map, which is what is referred to here. This function uses the 'event' of the change of the angle of the mouse scroll. We access this angle which is called delta, and we divide it by 120 to give either 1 or -1, where positive one is scrolling upwards, and negative one is scrolling downwards. This is useful as we can then do the zoom + delta, as zoom is an integer. This works out very nicely however we need to put some limits on it. If the zoom is greater than 0 (where zoom = 0 is the full map) then we can just add delta. If the zoom is equal to zero, we want to be able to zoom in but not zoom out anymore (as it puts the

program into a failed state where you cannot zoom in again). Finally, there was one extra circumstance programmed in for when zoom is less than zero. This shouldn't happen often, as delta is only in increments of 1 and -1, however due to the way that the wheel event works, it means you can somehow step the if statement if the wheel is moved down too fast. So, to counter this if the zoom is ever less than 0, the program will automatically set it to zero, as to prevent invalid inputs.

This is difficult to demonstrate in a single image, as the purpose is that the zoom feature is a current image changing, however it is implemented, and it does work. It's incredibly useful in truth as it lets the user see where all the points are regardless of how spaced out, they are.

Maptypes:

Next to deal with is the different map types that makes the program feel way more accessible and in the real world. PyQt has a widget which are checkboxes, which can be ticked in the UI, and the function `.isChecked()` can be used to identify if it has been ticked. This is perfect for letting the user have control over the 4 map types available. The maps that the API offers are:

- Satellite: Actual images of the surroundings, including houses, trees, etc.
- Roadmap: The default showing just roads and everything else abstracted away
- Hybrid: A combination of satellite and roadmap
- Terrain: Satellite images without the surroundings

This suits itself to checkboxes as it can be seen as two Boolean conditions: satellite map is on or off, roadmap is on or off. So, if we make two checkboxes that are Satellite and Roadmap, we can have these four states:

- Both off: Terrain map
- Satellite on and Roadmap off: Satellite map
- Satellite off and Roadmap on: Roadmap map
- Both on: Hybrid map

```
440. self.satellite = QCheckBox('Satellite', self)
441. self.satellite.move(20, 150)
442. self.satellite.stateChanged.connect(self.state_changed)
443. self.roadmap = QCheckBox('Roadmap', self)
444. self.roadmap.move(20, 170)
445. self.roadmap.stateChanged.connect(self.state_changed)
```

This above is in the `initUI` function and simply instantiates the widgets of the checkboxes, as well as connecting the function `state_changed`, as that will reload the image with the new conditions:

```
446. def state_changed(self):
447.     if self.satellite.isChecked() and not self.roadmap.isChecked():
448.         self.map.maptype = "satellite"
449.     elif self.roadmap.isChecked() and not self.satellite.isChecked():
450.         self.map.maptype = "roadmap"
```



```

451.     elif self.roadmap.isChecked() and self.satellite.isChecked():
452.         self.map.maptype = "hybrid"
453.     elif not self.roadmap.isChecked() and not self.satellite.isChecked():
454.         self.map.maptype = "terrain"
455.
456.     self.newImgLoad()

```

This function demonstrates the combinations of the check boxes, and with this it has my interface's intractability somewhat resemble the website Google maps, which is what I'm trying to invoke somewhat, the clear difference being is that this will have its functionality centred around finding the centre, with the rest of the program helping to accommodate someone in the real world using it.

Centre of 3:

The last bit that needs to be implemented is the process of there being 3 textboxes and when pressing enter it adds however many textboxes have a valid string inside them to the pointlist, to which it will then find the centre. Pressing enter on any of the boxes should really have the same result, so therefore I am attaching the on_returnPressed function. This will have the effect that if you enter only two points it will produce a centre midpoint of two and ignore the other blank box. This will be done by going through all of the textboxes, and if they are blank, they won't be added to a temporary array, if they are, then they will and then the processes can go as expected.

```

457. def on_returnPressed(self):
458.     temp = []
459.     temp.append(self.textbox1.text())
460.     temp.append(self.textbox2.text())
461.     temp.append(self.textbox3.text())
462.
463.     self.map.pointlist.clear()
464.     polarCheck = re.compile("^[+-]?([1-8]?[d(\.d+)?|90(\.0+)?],\s*[-+]?([180(\.0+)?|((1[0-7]d)|([1-9]?[d]))(\.d+)?])$" ) #regex command line
465.
466.     for s in temp: #s for string
467.         if s != "":
468.             if polarCheck.match(s) != None: #If it matches the polar coordinate
system
469.                 polar = s.split(", ")
470.                 P = Point.fromPolar([int(polar[0]), int(polar[1])])
471.                 self.map.pointlist.append(P)
472.             elif Point.fromAddress(s) != False: #If it produces a valid email
address
473.                 self.map.pointlist.append(Point.fromAddress(s))
474.             else:
475.                 print("error") #temporary error message
476.
477.         if len(self.map.pointlist) == 1:
478.             self.map.map.centre = self.map.pointlist[0]
479.             #When there is only 1 point, there is only one possible midpoint
480.
481.         elif len(self.map.pointlist) == 2:

```



```

482.         self.map.centre = self.map.centre2(self.map.pointlist[0],
self.map.pointlist[1])
483.         #When there are two points it must find the centre of 2
484.
485.     elif len(self.map.pointlist) == 3:
486.         self.map.centre = self.map.centre3()
487.         #When there are three points it must find the centre of 3
488.
489.     self.newImgLoad()

```

This code above works nicely to produce the intended outcome, however in writing it I did realise that the mark doesn't say the actual coordinates, so I added on a label widget that wrote: "The centre of these points are:" and then added on these lines of code to the end of return pressed:

```

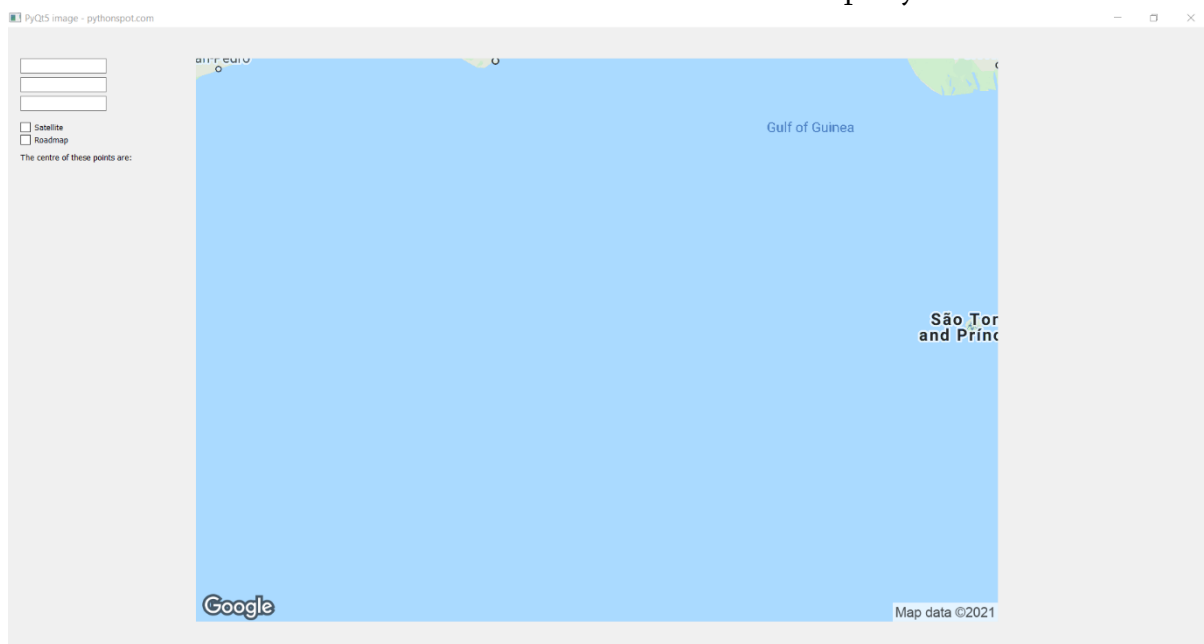
490. roundedLat = "{:.5f}".format(self.map.centre.polar[0])
491. roundedLog = "{:.5f}".format(self.map.centre.polar[1])
492. self.centreText.setText(f"The centre of these points are: {roundedLat},
{roundedLog}")

```

What is important to note is that the .format function is used to take the first 5 characters past the decimal place, so it is almost like rounding to 5 decimal places. This number will likely be increased however it worked best for the current size formatting, as will be seen below.

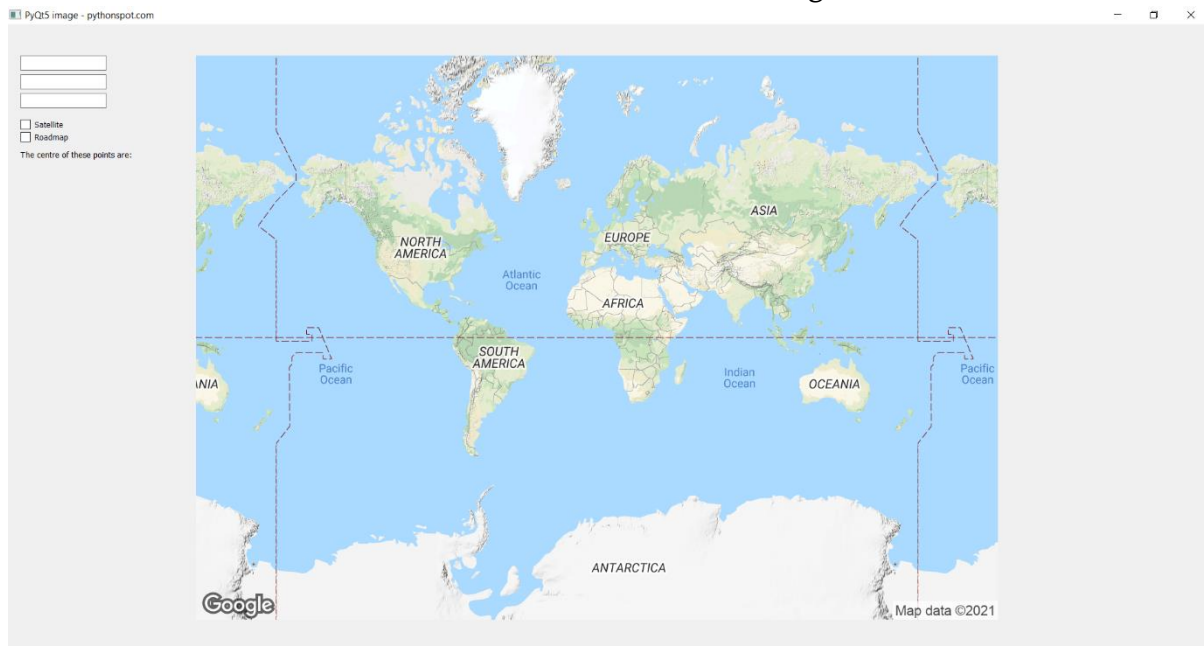
Showing off the UI:

This is the first screen upon loading. As you can see, I gave the window the width of 1920x1080 as to fill the screen. The point is focused on centre [0,0] with a zoom of 6, instantiated in initUI. The image gotten is in respect to the ratio 16:9 as to fit with the screen boundaries however it is smaller than it as to not overlap any textboxes.



This next one is the same screen however I have scrolled down as much as I can, leading

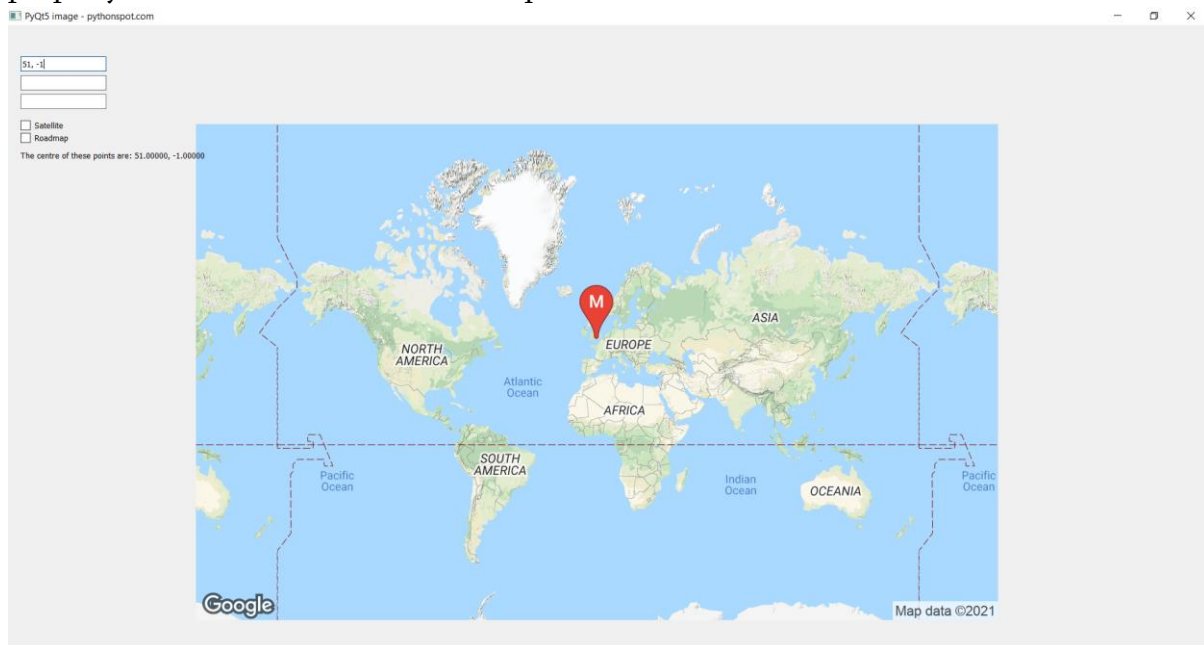
to the minimum zoom, now being 1 as it ends up looking better with the cropping rather than 0. This demonstrates how the zoom function is working.



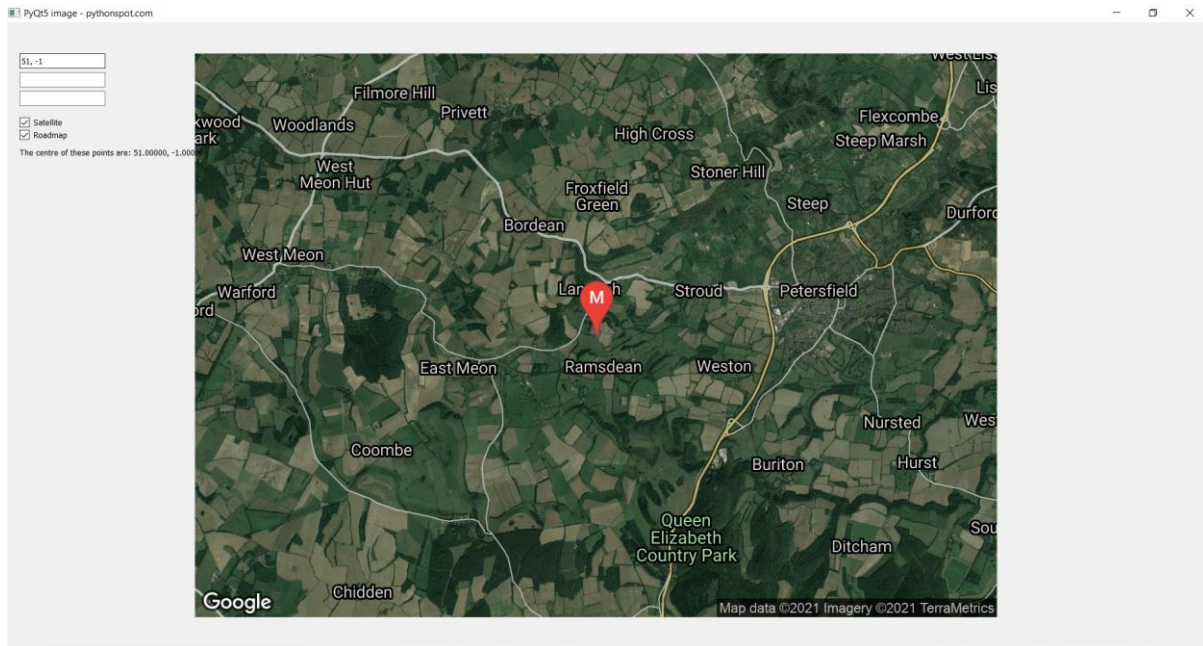
This screenshot shows the placement of the marker, which for the centre has been defined as:

“markers=size:mid%color:red%7Clabel:M%7C{latitude},{longitude}”

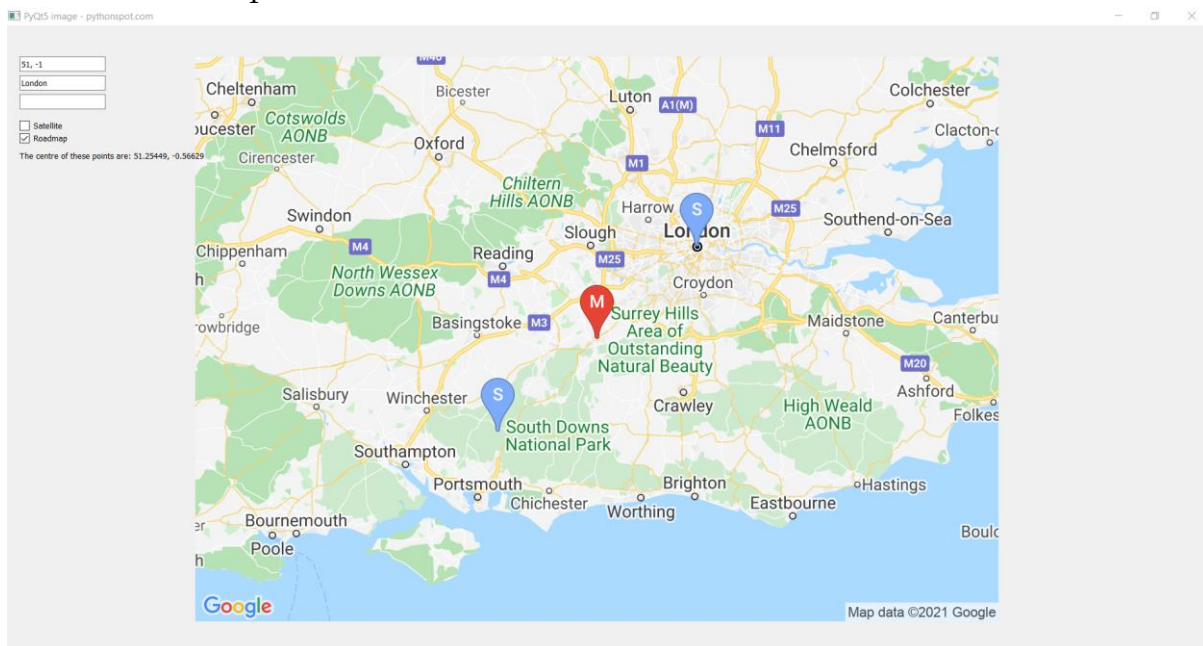
As well as showing an example of a polar coordinate being entered and interpreted properly, as we can see the centre to equal 51, -1.



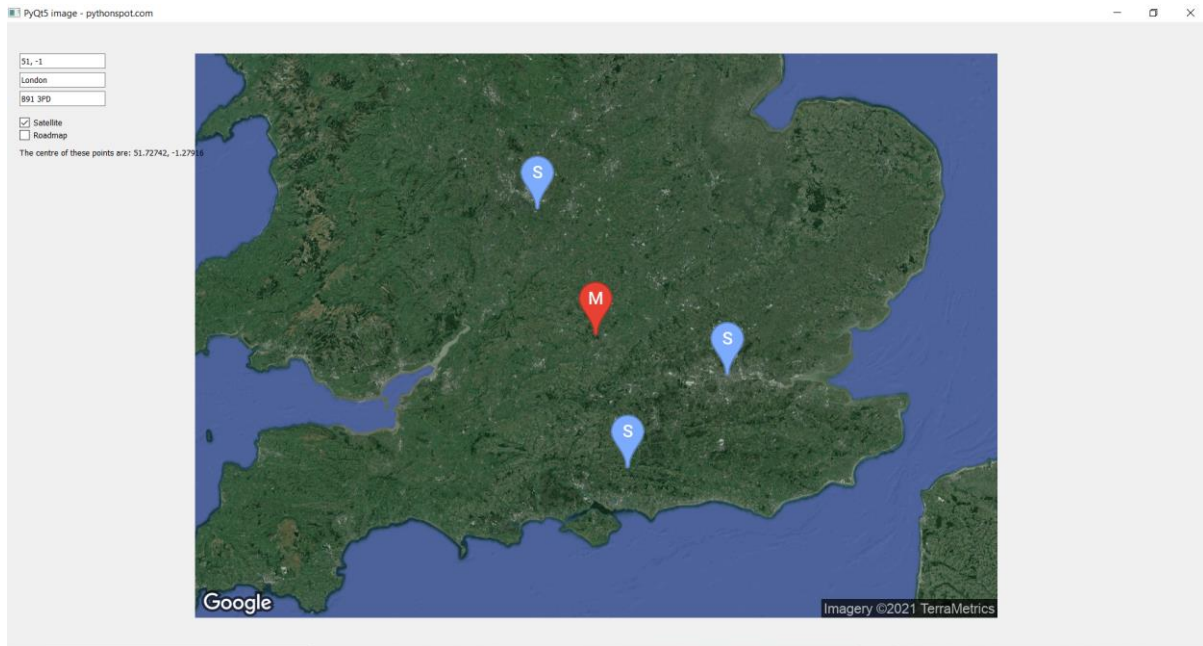
This next one follows on and zooms in, with the satellite and roadmap checkboxes checked, showing the effects of both in the hybrid map.



Next, I demonstrate inputting a second point, and inputting an address, generically London, which works as well, with both points being represented by a blue S marker, and the centre by a red M marker like before. It also shows just the roadmap as well, and a more zoomed out point.



This next and final screenshot is used to showcase how it works for 3 points, taking a postcode. It also shows the satellite map on its own and all 3 points present. However, it should be noted the satellite map is far more detailed the more zoomed in you go.



```

493. class App(QWidget):
494.
495.     def __init__(self):
496.         super().__init__()
497.         self.title = 'PyQt5 image - pythonspot.com'
498.         self.left = 0
499.         self.top = 0
500.         self.width = 1920
501.         self.height = 1080
502.         self.map = ""
503.         self.text = ""
504.         self.initUI()
505.
506.     def initUI(self):
507.         self.setWindowTitle(self.title)
508.         self.setGeometry(self.left, self.top, self.width, self.height)
509.
510.         self.map = Map(6, "terrain", Point.fromPolar([0,0]), [])
511.
512.         self.map.returnImage()
513.
514.         self.label = QLabel(self)
515.         pixmap = QPixmap('map1.jpg')
516.         self.label.setPixmap(pixmap)
517.         self.label.move(300, 50)
518.
519.         self.centreText = QLabel("The centre of these points are:
", self)
520.         self.centreText.move(20, 200)
521.
522.         self.satellite = QCheckBox('Satellite', self)
523.         self.satellite.move(20, 150)
524.         self.satellite.stateChanged.connect(self.state_changed)
525.         self.roadmap = QCheckBox('Roadmap', self)
526.         self.roadmap.move(20, 170)
527.         self.roadmap.stateChanged.connect(self.state_changed)
528.

```

```

529.         self.textbox1 = QLineEdit(self)
530.         self.textbox1.move(20, 50)
531.         self.textbox1.returnPressed.connect(self.on_returnPressed)
532.
533.         self.textbox2= QLineEdit(self)
534.         self.textbox2.move(20, 80)
535.         self.textbox2.returnPressed.connect(self.on_returnPressed)
536.
537.         self.textbox3 = QLineEdit(self)
538.         self.textbox3.move(20, 110)
539.         self.textbox3.returnPressed.connect(self.on_returnPressed)
540.
541.         self.show()
542.
543.     def newImgLoad(self):
544.         self.map.returnImage()
545.         pixmap = QPixmap('map1.jpg')
546.         self.label.setPixmap(pixmap)
547.         self.label.update()
548.
549.     def on_returnPressed(self):
550.         temp = []
551.         temp.append(self.textbox1.text())
552.         temp.append(self.textbox2.text())
553.         temp.append(self.textbox3.text())
554.
555.         self.map.pointlist.clear()
556.         polarCheck = re.compile("^[-+]?([1-8]?\\d(\\.\\d+)??|90(\\.0+)?),\\s*[-+]?([180(\\.0+)?|((1[0-7]\\d)|([1-9]?\\d))(\\.\\d+)?)$")
557.
558.         for s in temp:
559.             if s != "":
560.                 if polarCheck.match(s) != None:
561.                     polar = s.split(", ")
562.                     P = Point.fromPolar([int(polar[0]), int(polar[1])])
563.                     self.map.pointlist.append(P)
564.                 elif Point.fromAddress(s) != False:
565.                     self.map.pointlist.append(Point.fromAddress(s))
566.                 else:
567.                     print("error")
568.
569.                 if len(self.map.pointlist) == 1:
570.                     self.map.centre = self.map.pointlist[0]
571.
572.                 elif len(self.map.pointlist) == 2:
573.                     self.map.centre = self.map.centre2(self.map.pointlist[0],
self.map.pointlist[1])
574.
575.                 elif len(self.map.pointlist) == 3:
576.                     self.map.centre = self.map.centre3()
577.
578.         self.newImgLoad()
579.         roundedLat = "{:.5f}".format(self.map.centre.polar[0])
580.         roundedLog = "{:.5f}".format(self.map.centre.polar[1])
581.         self.centreText.setText(f"The centre of these points are: {roundedLat},
{roundedLog}")
582.
583.     def state_changed(self):

```



```

584.         if self.satellite.isChecked() and not self.roadmap.isChecked():
585.             self.map.maptype = "satellite"
586.         elif self.roadmap.isChecked() and not self.satellite.isChecked():
587.             self.map.maptype = "roadmap"
588.         elif self.roadmap.isChecked() and self.satellite.isChecked():
589.             self.map.maptype = "hybrid"
590.         elif not self.roadmap.isChecked() and not self.satellite.isChecked():
591.             self.map.maptype = "terrain"
592.
593.         self.newImgLoad()
594.
595.     def wheelEvent(self, event: QWheelEvent):
596.         delta = event.angleDelta().y()/120
597.         if self.map.zoom > 1:
598.             self.map.zoom = int(self.map.zoom + delta)
599.         elif self.map.zoom == 1 and delta == 1:
600.             self.map.zoom = int(self.map.zoom + delta)
601.         elif self.map.zoom < 1:
602.             self.map.zoom = 1
603.         self.newImgLoad()
604.
605. if __name__ == '__main__':
606.     app = QApplication(sys.argv)
607.     ex = App()
608.     sys.exit(app.exec_())

```

Second Iteration Review:

Of my goals that me and my stakeholder set out for the second iteration, they were all met with flying colours, yet it isn't the most optimized state. When going to bring my laptop in to show off the GUI there were areas which were far more successful than others. The first thing that was an instant favourite was the use of the textboxes, and how that automatically let the image change position appropriately to locate where it where the latitude and longitude points were. Additionally, the ability to zoom in and out was greatly enjoyed, giving a sense of scope, as well as allowing us to see where all the points were, but there was something that was appropriately noted and that was how slow each transaction seemingly was. See, when each image was gathered, it had a lengthy process of:

1. Analysing the text input
2. Retrieving a point's data from online Json data
3. Pulling a specialised image from the internet
4. Replacing the previous image with the new image
5. Updating the window

All of these steps show that not only is the program slow at points, but it feels very staggered; as well as the program's speed being very dependent on the internet. Moreover, the smoothness of transitions wasn't helped by the lack of frames. When comparing the GUI to what is effectively aspiring to be, Google Maps, then we can see that whenever a new point is entered, rather than cutting straight to that point, the

camera moves across from the current point it focusing on, to look at the new point, moving by a degree each frame as to make it look like a fast and smooth transition, whereas mine cuts immediately. This is most obvious in the zoom feature which stems from the lack of control that the static image API gives in the zoom levels. In an ideal world I would be able to change zoom by 0.1 places each scroll, instead its by 1 as zoom has to be an integer, creating an uglier and less fluid zoom in zoom out process.

Despite these flaws though, my stakeholder said how it worked for the project as I wasn't trying to make Google Maps. Instead, I was trying to show the centre of two, and centre of three, and that was done incredibly well by using the satellite imaging checkbox, as being able to see houses and streets on the ground level created a whole new level of immersion and realism for him.

From here though, I wasn't sure where I was going to update it if not for improving the GUI. I had found that I couldn't ascertain the minimum point of variance, meaning I could never implement a solution for n points. It was a slightly depressing realisation for both me and my stakeholder, who had briefly seen the maths behind it and thought it was interesting, but we agreed it was a loose end that couldn't be tied, and we would just have to move on, to having our maths lesson that was just after our talk. In this maths lesson we were covering an all too familiar topic of iteration, the method I used previously to try and isolate C . In the lesson we learnt about the converging and diverging formulae formed from equations, and how that different roots could be found with different formulas... WAIT! During the lesson I had a realisation. When looking back at the minimum variance result that returned a $0 = 0$ answer, I only tried one formula derived from the values. The way I originally went about it, was that I would gather every value of C , as to get that as close to the answer as possible; but instead, I wanted to keep them separate! By doing these two different formulae can be made which each will tend to one of two answers: the maximum and the minimum possible variance.

At the end of the lesson, I disclosed this realisation, and shared a laugh at how this felt preordained somehow, too perfectly written. Regardless, I then had the new goal of minimizing the variance of the distances, as well as an additional one. Before I went, I was asked about finding the minimum total distance. This would be good as it would be finding the point that had the smallest distance for people to travel in total, which would mean less money spent on travel costs. Both of these challenges were something I was sure I could do, now armed with my new knowledge.

Third Iteration:

In receiving the feedback and the new approach to the older problem, I now have a new aim, and new focus for the algorithm, which will produce the original endgame of the

project I had in mind. This iteration of the program will need to do a few more things which will be interesting to implement:

- To have an algorithm that finds the point of minimum variance (I.e., closest to equidistance) of n number of points
- To have an algorithm that has the smallest sum of lengths (I.e., shortest total distance travelled) of n number of points
- To output and display the changes in variance and the sum of distances
- To have a user interface that allows n number of points to be entered
- To have a button that changes between smallest total distance and minimum variance
- To show each iteration of the program as it gets closer as a marker on the map

This will be the hardest section yet as it brings together everything from the previous sections into one large successful bundle.

The Algorithm Proofs:

As discussed with my stakeholder, there were two things that I needed to find a method to calculate, both of which I am very close to having proven already. Firstly, there is my minimum variance formula which I thought that I would have to use the second derivative for the Newton-Raphson iterative approach, however now with the new knowledge of iterative formulas I have another solution.

First, we start with the derivative set equal to zero as we did last time:

$$\begin{aligned} & \frac{2}{n} \sum_{i=1}^n \frac{1}{|V_i|^2 |C|^2} \left[\frac{(V_i \cdot C)}{|V_i \times C|} K_i C - V_i |V_i \times C| \right] \arctan \frac{|V_i \times C|}{V_i \cdot C} \\ &= \frac{2}{n^2} \left(\sum_{i=1}^n \arctan \frac{|V_i \times C|}{V_i \cdot C} \right) \left(\sum_{i=1}^n \frac{1}{|V_i|^2 |C|^2} \left[\frac{(V_i \cdot C)}{|V_i \times C|} K_i C - V_i |V_i \times C| \right] \right) \end{aligned}$$

From here, we are going to now separate each sum into 2 components, however first we are going to rewrite it in a series of letters which are easier to identify.

$$s_1 = \frac{1}{|V_i|^2 |C|^2}$$

$$s_2 = \frac{(V_i \cdot C)}{|V_i \times C|}$$

$$s_3 = |V_i \times C|$$

$$l = \arctan \frac{|V_i \times C|}{(V_i \cdot C)}$$

This does a good job of transforming the function into something that fits on to a single line, which means we can rearrange it into a useful form, splitting it into 4 parts. Note that the names s1, s2, s3 all represent scalar1, scalar2, scalar3.

$$\sum_{i=1}^n l s_1 (s_2 K_i C - s_3 V_i) = \frac{1}{n} \sum_{i=1}^n l \sum_{i=1}^n s_1 (s_2 K_i C - s_3 V_i)$$

$$\sum_{i=1}^n l s_1 s_2 K_i C - \sum_{i=1}^n l s_1 s_3 V_i = \frac{1}{n} \sum_{i=1}^n l \sum_{i=1}^n s_1 s_2 K_i C - \frac{1}{n} \sum_{i=1}^n l \sum_{i=1}^n s_1 s_3 V_i$$

$$\Omega_1 C - \delta_1 = \Omega_2 C - \delta_2$$

As you can see above, we have all the sums then written as an omega or a delta, the omega representing matrices, and the delta written as vectors. This is useful because then we can rearrange this into two new formula:

$$C_{n+1} = \Omega_1^{-1} (\Omega_2 C_n + \delta_1 - \delta_2)$$

$$C_{n+1} = \Omega_2^{-1} (\Omega_1 C_n + \delta_2 - \delta_1)$$

Unlike its predecessor this formula won't give a zero = zero scenario as we aren't pulling both C values out of the equation and are rather creating ourselves two scenarios. Due to the nature of iterative formulas, if there are multiple answers then it will tend to either one dependent on if the starting formula is increasing or decreasing. As there should only be two answers for the cases, we give it aside from some more extreme examples, this means we can use these formulae above give us the answer that we want.

Programming the Formula:

Fortunately, due to how mathematical this approach is, and how abstracted away all the components are, we can program this in rather quickly. Especially since many of the base functions were coded in the initial lost programs.

```

609. def s1(self, V, C): #scalar1
610.     s1 = 1 / ((np.linalg.norm(V.cartesian) ** 2) * (np.linalg.norm(C.cartesian) **
611.         2))
612.     #s1 = 1/|V|^2|C|^2
613.     return s1
614. def s2(self, V, C): #scalar2
615.     s2 = np.dot(V.cartesian, C.cartesian) / np.linalg.norm(np.cross(V.cartesian,
616.         C.cartesian))
617.     #s2 = V.C/|VxC|
618.     return s2
619. def s3(self, V, C): #scalar3
620.     s3 = np.linalg.norm(np.cross(V.cartesian, C.cartesian))
621.     #s3 = |VxC|
622.     return s3
623.
624. def l(self, V, C): #arclength
625.     cross = np.cross(V.cartesian, C.cartesian)
626.     dot = np.dot(V.cartesian, C.cartesian)
627.     temp = np.linalg.norm(cross) / dot
628.
629.     l = math.atan(temp)
630.     #l = atan(|VxC|/V.C)
631.     return l
632.
633. def K(self, V): #K matrix
634.     X = V.cartesian
635.     K = np.array([[X[1] ** 2 + X[2] ** 2), -1 * X[0] * X[1], -1 * X[0] * X[2]],
636.         [-1 * X[0] * X[1], (X[0] ** 2 + X[2] ** 2), -1 * X[1] * X[2]],
637.         [-1 * X[0] * X[2], -1 * X[1] * X[2], (X[0] ** 2 + X[1] ** 2)])
638.     return K

```

Similarly put in the Map class, these base functions give the components to make the rest of the functions in the formulae.

```

639. def createOmega1(self, C): #Matrix
640.     omega = np.array([[0, 0, 0],
641.         [0, 0, 0],
642.         [0, 0, 0]])
643.     for V in self.pointlist:
644.         l = self.l(V, C)
645.         s1 = self.s1(V, C)
646.         s2 = self.s2(V, C)
647.         K = self.K(V)
648.         omega = omega + l * s1 * s2 * K
649.
650.     return omega
651.
652. def createOmega2(self, C): #Matrix
653.     omega = np.array([[0, 0, 0],
654.         [0, 0, 0],
655.         [0, 0, 0]])
656.     n = len(self.pointlist)

```

```

657.         lsum = 0
658.         for V in self.pointlist:
659.             l = self.l(V, C)
660.             lsum = lsum + l
661.
662.         for V in self.pointlist:
663.             s1 = self.s1(V, C)
664.             s2 = self.s2(V, C)
665.             K = self.K(V)
666.             omega = omega + s1 * s2 * K
667.
668.         return (lsum / n) * omega
669.
670.     def createDelta1(self, C): #Vector
671.         delta = np.array([0, 0, 0])
672.         for V in self.pointlist:
673.             l = self.l(V, C)
674.             s1 = self.s1(V, C)
675.             s3 = self.s3(V, C)
676.             delta = delta + l * s1 * s3 * V.cartesian
677.
678.         return delta
679.
680.     def createDelta2(self, C): #Vector
681.         delta = np.array([0, 0, 0])
682.         n = len(self.pointlist)
683.         lsum = 0
684.         for V in self.pointlist:
685.             l = self.l(V, C)
686.             lsum = lsum + l
687.         for V in self.pointlist:
688.             s1 = self.s1(V, C)
689.             s3 = self.s3(V, C)
690.             delta = delta + s1 * s3 * V.cartesian
691.
692.         return (lsum / n) * delta
693.
694. #These functions all take use of the pointlist in the map, as in it doesn't take in
        individual points unlike the centre2 function. This is because these need to cycle
        through multiple items always, whereas 2 points is needed in the centre of 3.

```

One thing to note before showing the final two functions of this is that these functions are very broken down and modular, purely because it's so easy to make mistakes with this. There are so many similar terms and values it would be too risky to do otherwise, plus it helps in the understandability of the code when reading it through, assuming you understand the formula presented.

```

695. def centreN1(self, C):
696.     omega1 = self.createOmega1(C)
697.     omega2 = self.createOmega2(C)
698.     delta1 = self.createDelta1(C)
699.     delta2 = self.createDelta2(C)
700.     #omegas and deltas assigned early and clearly as for no mistakes in code below
701.
702.     invomega = np.linalg.inv(omega1) #inverse matrix

```

```

703.     temp = np.matmul(omega2, C.cartesian) - delta2 + delta1 #matmul is matrix
      multiplication of numpy arrays. This is valid here as 3x3 matrix can multiply a 3x1
      matrix
704.     M = np.matmul(invomega, temp)
705.
706.     t_coeffecient = R / np.linalg.norm(M)
707.     C = t_coeffecient * M #This takes from the previous function to make a vector
      to the sphere, done so that it can return a Point rather than cartesian
      coordinates.
708.
709.     return Point.fromCartesian(C)
710.
711. def centreN2(self, C):
712.     omega1 = self.createOmega1(C)
713.     omega2 = self.createOmega2(C)
714.     delta1 = self.createDelta1(C)
715.     delta2 = self.createDelta2(C)
716.     #omegas and deltas assigned early and clearly as for no mistakes in code below
717.
718.     invomega = np.linalg.inv(omega2)
719.     temp = np.matmul(omega1, C.cartesian) - delta1 + delta2
720.     M = np.matmul(invomega, temp)
721.
722.     t_coeffecient = R / np.linalg.norm(M)
723.     C = t_coeffecient * M
724.
725.     return Point.fromCartesian(C)

```

Finally, this program is the one I wanted to create all along. Using centreN1 and centreN2, we can find some way to see what the program is tending toward when we repeatedly use this formula again and again iteratively. Now the real test for it, is to put in the correct value as a starting value, hence using this for three points. The way I expect this to work is that for 1 of the functions it will change greatly, whilst the other will stay nearly the exact same, one tending toward maximum variance and the other tending toward minimum variance. That is the small risk that is accompanied with using the first derivative, however since we have both of these algorithms, I think that there aren't any cases which either algorithm will both give the maximum variance, however that'll be something that will come up in testing. The big issue is that due to the approximations the computer has to make, whether that be removing some decimals from the irrational numbers or the simplifications made in the multiplications, it is clear that small changes are occurring each turn. Therefore, we will need to implement some level of intelligence for the program, as to know when a good point is to stop. See when putting in the actual centre of 3 points, many times it remains around that value for 10-30 iterations, only for it to spiral off out of control, and when looking at it with the variance function as written below, we find that variance is increasing dramatically, which shows to me the reason it sometimes spirals out like that is it is tending toward the point of maximum variance. Therefore, we need the computer to know if variance is increasing or decreasing. If it is increasing, then we either revert, or take the smallest one available to us. For this we definitely need a way to look at variance, so we can use this function:

```

726. def varianceCheck(self, C):
727.     mean = 0
728.     variance = 0
729.     for V in self.pointlist:
730.         mean = mean + self.arcdistance(C, V)
731.     mean = mean/len(self.pointlist)
732.     for V in self.pointlist:
733.         variance = variance + (self.arcdistance(C, V) - mean)**2
734.     variance = variance/len(self.pointlist)
735.
736.     return variance

```

Using this variance function, the computer will be able to see how the points are changing each iteration. If method 1 causes variance to increase, then it should try method 2. If method 2 causes an increase in variance, then an error message needs to be displayed saying there is no exact centre. If either have a decrease, then they should follow the same procedure x number of times for it to properly centre in on the solution, perhaps it can be implemented in such a way that it will take the first 7-10 characters and compare them against each other, and when they are equal it is enough to conclude it has centred in on the point.

This isn't the only algorithm to look at though, as we still have the minimum sum of distances to consider. The minimum sum algorithm is something I actually tackled in the first section of the lost programs, as it was a test for programming minimum variance. However, after a conversation with my stakeholder and the realisation of the other potential application, finding the sum of the minimum amount of points is also necessary as it could help save on overall travel prices for a meetup. For this I looked back on my original code and saw how it could be implemented with the current code far easier.

$$\left(\sum_{i=0}^n \frac{1}{|V_i|^2 |C|^2} \frac{(V_i \cdot C)}{|V_i \times C|} K_i \right) C = \sum_{i=0}^n V_i |V_i \times C|$$

Here we see the formula I derived in the end from taking the derivative of the sum of l, and these should be two vary familiar sums. The first is the sum of s1, s2 and Ki, and the second is the sum of s3 and Vi. This can more than easily be written up into a function for the Map class.

```

737. def minimumDistanceTravelled(self, C):
738.     omega = np.array([[0, 0, 0],
739.                       [0, 0, 0],
740.                       [0, 0, 0]])
741.     delta = np.array([0, 0, 0])
742.     for V in self.pointlist:
743.         s1 = self.s1(V, C)
744.         s2 = self.s2(V, C)
745.         K = self.K(V)
746.         s3 = self.s3(V, C)
747.
748.         omega = omega + s1 * s2 * K
749.         delta = delta + s3 * V.cartesian
750.

```

```

751.     invomega = np.linalg.inv(omega)
752.     M = np.matmul(invomega, delta)
753.
754.     t_coeffecient = R / np.linalg.norm(M)
755.     C = t_coeffecient * M
756.
757.     return Point.fromCartesian(C)

```

As you can see it's a very similar function to the previous ones, using similar elements, however this one can be nicely and easily condensed into this smaller and more manageable form because it's so much simpler. Who would have thought that the more useful process was simpler. There is a huge disadvantage though, and that is that is a single formula. There are no other ways to extract an answer to this, so if the sum of distances tends toward a maximum, then there will be no way to overcome it.

Back to the GUI:

Unfortunately, that is most of the algorithmic work done, which means it's time to implement this in a way that would be useful to users. The problem is currently, that there are only 3 textboxes, and if we want to add variable textboxes, we need to add a method to do so. Originally, I was going to tackle this in a similar way to Google Maps, that is whenever enter is pressed, a new text box is added in; and all empty textboxes are removed. This, was a mistake. Not only did it end up an algorithmic mess with a series of nonsensical arrays and lists, but it also didn't really work, not properly, not in a way that would be useful to the user. This was first done by having a class that would be used for textbox objects, but that failed because PyQt got really annoyed whenever I tried to do something like that. So the next thing I did was I implemented a new text box function which added a textbox 50 pixels down, and an additional $30 \cdot n$, where n is the number of textboxes there are. Alongside this was the function of `updateTextBox` which tried to go through every text box individually, and then if it was empty, remove it, and if it had text in it, it would store the text in an array, and then recreate new textboxes for each text stored in the array, putting the text back in it. This didn't work as every time this was done it gave an unexpected and unwanted result of the textboxes in positions that weren't accessible, or that overlapped the image.

```

758.def newTextbox(self, pretext, n):
759.    self.textbox = QLineEdit(pretext, self)
760.    self.textbox.move(20, 50 + n*30)
761.    self.textbox.returnPressed.connect(self.on_returnPressed)
762.    self.textboxarray.append(self.textbox)
763.    print(self.textboxarray)
764.    self.textbox.show()
765.
766.def updateTextbox(self):
767.    textlist = []
768.    print(textlist)
769.    for box in self.textboxarray:
770.        print(box.text())
771.        if box.text() != "":
772.            textlist.append(box.text())
773.            box.deleteLater()
774.            self.textboxarray.remove(box)
775.    print(textlist)
776.    for text in textlist:
777.        self.newTextbox(f"{text}", textlist.index(text))
778.        self.newTextbox("", len(textlist)+1)

```

So, in the face of failure, what do you do? Give up and try another solution! Here I made the silly effort of trying to add more textboxes based off of another input box placed at

the top of the screen, and really that just added in another series of unfortunate logic errors, and it would not be useful to have the user try and enter the number they want as that's a whole other input that has to be monitored. The code for it is below.

```
779. def create_N_Boxes(self):
780.     n = int(self.pointNumber.text())
781.     for i in range(0, n):
782.         self.textbox = QLineEdit(pretext, self)
783.         self.textbox.move(20, 50 + i * 30)
784.         self.textbox.returnPressed.connect(self.on_returnPressed)
785.         self.textboxarray.append(self.textbox)
786.     self.textbox.show()
```

However, all of these failed functions eventually made me realise what I actually had to do, and that was to create buttons. See, rather than implementing an overcomplicated method to add in textboxes, I decided to use what works, buttons, where a + button would have an add textbox feature, and a – button which would remove textboxes. So first I implemented the buttons without a feature, using the geometry function to make them small and less obstructive to the whole page.

```
787. self.widget = QWidget(self)
788. self.addb = QPushButton(self.widget)
789. self.addb.setText("+")
790. self.addb.setGeometry(160, 20, 24, 24)
791. self.addb.clicked.connect(self.addbox)
792. self.subb = QPushButton(self.widget)
793. self.subb.setText("-")
794. self.subb.setGeometry(185, 20, 24, 24)
795. self.subb.clicked.connect(self.removebox)
```

Then I proceeded to connect two functions, addbox and removebox. The first function adds a textbox, and the second one removes a textbox. The way this is done is by adding a textbox into an array, and when removing a textbox it removes it from the array, using the deleteLater feature. This is effective as I can use an updated traversal method inside of the returnPressed function, by going through each textbox in the array, checking if its text value is not equal to "", and it meets the format wanted (I.e it's a valid and existing point). There is also a condition so that we can always have one textbox available, the length of the textbox list always having to be greater than or equal to 1, which can be seen as an if statement in removebox function.

```
796. def addbox(self):
797.     self.textbox = QLineEdit(self)
798.     self.textbox.move(20, 50 + (self.n - 1)*30)
799.     self.textbox.returnPressed.connect(self.on_returnPressed)
800.     self.textbox.show()
801.     self.textboxarray.append(self.textbox)
802.     self.n = self.n + 1
803.
804. def removebox(self):
805.     if self.n > 1:
806.         self.textboxarray[self.n - 1].deleteLater()
807.         del self.textboxarray[self.n - 1]
808.         self.n = self.n - 1
809.     self.show()
```

As you can also see below, this allows for the addition of any number of textboxes I want to add in, however there is the limitation found in the screen dimensions. The maximum number of textboxes that currently fit on the screen is a number I think that is greater than actually necessary, however this problem can potentially be solved by adding in a slider/scrollbar in a semi-separate window, still attached to the current window, similar to Google Maps, but this will be discussed with the stakeholder to see if it is necessary, as

it is extremely unlikely anybody will need to enter more than 30 points (more specifically 33 is the maximum before the textboxes appear off of screen.

Centre of N and iterations

From here, I now need to add in some better functionality, and a way for the program to find the centre of n number of points. Whilst I have two programs to calculate this centre, I need the program to be able to determine which is the correct one to use. To do this, I need to observe increasing and decreasing variance, which is the perfect use for my varianceCheck function. Since this calculates the variance of a dataset given a centre, I can run each function 5 times; if one has variance increase and one has variance decrease, I know which formula to choose. The only problem is deciding that initial value of C. Currently I have decided for 0, 0 to be the starting latitude and longitude for C, but that may not be the most optimal starting condition for all of the formulas, meaning there is definitely room to improve.

```
810. def centreMethod(self):
811.     original = 0
812.     temp1 = 0
813.     temp2 = 0
814.
815.     M1 = []
816.     M2 = []
817.
818.     C = Point.fromPolar([0, 0])
819.     original = self.varianceCheck(C)
820.     for i in range(5):
821.         C = self.centreN1(C)
822.         temp1 = self.varianceCheck(C)
823.
824.     C = Point.fromPolar([0, 0])
825.     for i in range(5):
826.         C = self.centreN2(C)
827.         temp2 = self.varianceCheck(C)
828.
829.
830.     C = Point.fromPolar([0, 0])
831.
832.     if temp1 <= original and temp2 >= original:
833.         return 1
834.     elif temp2 <= original and temp1 >= original:
835.         return 2
836.     else:
837.         print("Error")
```

It is also important to note that for cases where they both decrease or both increases, I'm not yet sure what the program should do. I think returning an error message makes the most sense, and unfortunately this will likely be more common than not for certain points. Ones that have some semblance of a centre work really well, but ones that are extremely unequal and spread out often don't really have a point where variance is small, and this could be communicated to the user on the side in a small text aside.

Before implementing this centreMethod function into the returnPressed function, there was something that would work best alongside it, and that is showing the iterations it uses to approach the solution given. These would be done in the form of small green

markers, and could be added by a process in the returnImage function that always adds markers of points put into a list, as can be seen below:

```
838. for I in self.iterationlist:
839.     self.url = self.url +
        f"&markers=size:tiny%7Ccolor:green%7C{str(I.polar[0])},{str(I.polar[1])}"
```

This works out well for when few iterations are performed! For ones with hundreds of iterations though, it results in a slow and laggy process with the screen bloated with markers. The solution to this is quite simple, and that is to limit the number of iterations that are shown. We can still have x number of iterations of the formula, but the iterations only reflect the major changes by showing the first 10, and then every 5th iteration. This may be changed or optimize, but when putting in values it felt like a nice compromise between speed and detail. Additionally, it was decided to make the number of iterations to be an attribute of the App, in the form of accuracy, where the accuracy = number of iterations. In considering though that some may not care about the iterations being shown, I added in a checkbox which if checked acts as a permission for future inputs to show iterations, the way to refresh being pressing enter on already existing textboxes.

However, when making this feature, I also saw it to be quite easy to implement in another feature I had planned, that was the minimization of the total distance. This fortunately didn't have two methods (meaning if it went toward maximum total distance, it would fail horribly) but it made programming pretty simple, adding in a checkbox for that feature, and then making an if statement do that iterative process if it was selected.

```
840. def on_returnPressed(self):
841.     self.map.iterationlist.clear()
842.     self.map.pointlist.clear()
843.     polarCheck = re.compile("[^+]?([1-8]?d(\.d+)?|90(\.0+)?),\s*[-+]?([180(\.0+)?|((1[0-7]d)|([1-9]?d))(\.d+)?])$")
844.
845.     for box in self.textboxarray:
846.         text = box.text()
847.         if text != "":
848.             if polarCheck.match(text) != None:
849.                 polar = text.split(", ")
850.                 P = Point.fromPolar([int(polar[0]), int(polar[1])])
851.                 self.map.pointlist.append(P)
852.             elif Point.fromAddress(text) != False:
853.                 self.map.pointlist.append(Point.fromAddress(text))
854.             else:
855.                 print("error")
856.
857.         if self.minimization.isChecked():
858.             C = Point.fromPolar([0, 0])
859.             for i in range(self.accuracy):
860.                 print(C.polar)
861.                 print(self.map.summationCheck(C))
862.                 C = self.map.minimumDistanceTravelled(C)
863.                 if self.iteration.isChecked():
864.                     if i < 10:
865.                         self.iterationImage(C)
866.                 else:
```

```

867.             if (i / 10).is_integer():
868.                 self.iterationImage(C)
869.         self.map.centre = C
870.
871.         elif not self.minimization.isChecked():
872.
873.             if len(self.map.pointlist) == 1:
874.                 self.map.centre = self.map.pointlist[0]
875.
876.             elif len(self.map.pointlist) == 2:
877.                 self.map.centre = self.map.centre2(self.map.pointlist[0],
self.map.pointlist[1])
878.
879.             elif len(self.map.pointlist) == 3:
880.                 self.map.centre = self.map.centre3()
881.
882.             else:
883.                 C = Point.fromPolar([0, 0])
884.                 if self.map.centreMethod() == 1:
885.                     for i in range(self.accuracy):
886.                         print(C.polar)
887.                         C = self.map.centreN1(C)
888.                         if self.iteration.isChecked():
889.                             if i < 10:
890.                                 self.iterationImage(C)
891.                             else:
892.                                 if (i/10).is_integer():
893.                                     self.iterationImage(C)
894.
895.                 self.map.centre = C
896.                 elif self.map.centreMethod() == 2:
897.                     for i in range(self.accuracy):
898.                         print(C.polar)
899.                         C = self.map.centreN2(C)
900.                         if self.iteration.isChecked():
901.                             if i < 10:
902.                                 self.iterationImage(C)
903.                             else:
904.                                 if (i/10).is_integer():
905.                                     self.iterationImage(C)
906.
907.                 self.map.centre = C
908.                 else:
909.                     print("Error")
910.
911.         self.newImgLoad()
912.         roundedLat = "{:.10f}".format(self.map.centre.polar[0])
913.         roundedLog = "{:.10f}".format(self.map.centre.polar[1])
914.         self.centreText.setText(f"The centre of these points are: {roundedLat},
{roundedLog}")
915.
916.         self.update()

```

As you can see however, there is a lot of repetition found in that code, so that can easily be alleviated by creating an iteration function. This will take the functions of centreN1, centreN2 and minimumDistanceTravelled and then apply the iterations over and over

and over again. This is the most effective as it means that repetition can be called all in one function. With this completed though, I have achieved what I set out to do, with a formula that finds the centre of N points, all put into code form, showing the approximations each time, the only thing left to do is to add some additional smaller quality of life features, which will be discussed with my stakeholder, in the review of this iteration.

Additionally, another function is added in, which takes in other functions. This method function takes in differing parameters which vary on the function that it wants to use. This is useful to reduce the amount of repeated code, as this does the same process for any method of centreN1, centreN2 and minimumDistanceTravelled, as well as the detail parameter being used to refer to any function of the detail we want to watch be minimized, so the variance and summation functions. This will be really useful in debugging as well as showing off the effect to my stakeholder, in some cases the variances decreasing from the millions to hundreds.

```
1. def iterations(self, method, detail, C):
2.     if self.iteration.isChecked():
3.         self.iterationImage(C)
4.     for i in range(self.accuracy):
5.         print(C.polar)
6.         print(detail(C))
7.         C = method(C)
8.         if self.iteration.isChecked():
9.             if i < 10:
10.                self.iterationImage(C)
11.            else:
12.                if (i / 10).is_integer():
13.                    self.iterationImage(C)
14.     return C
```

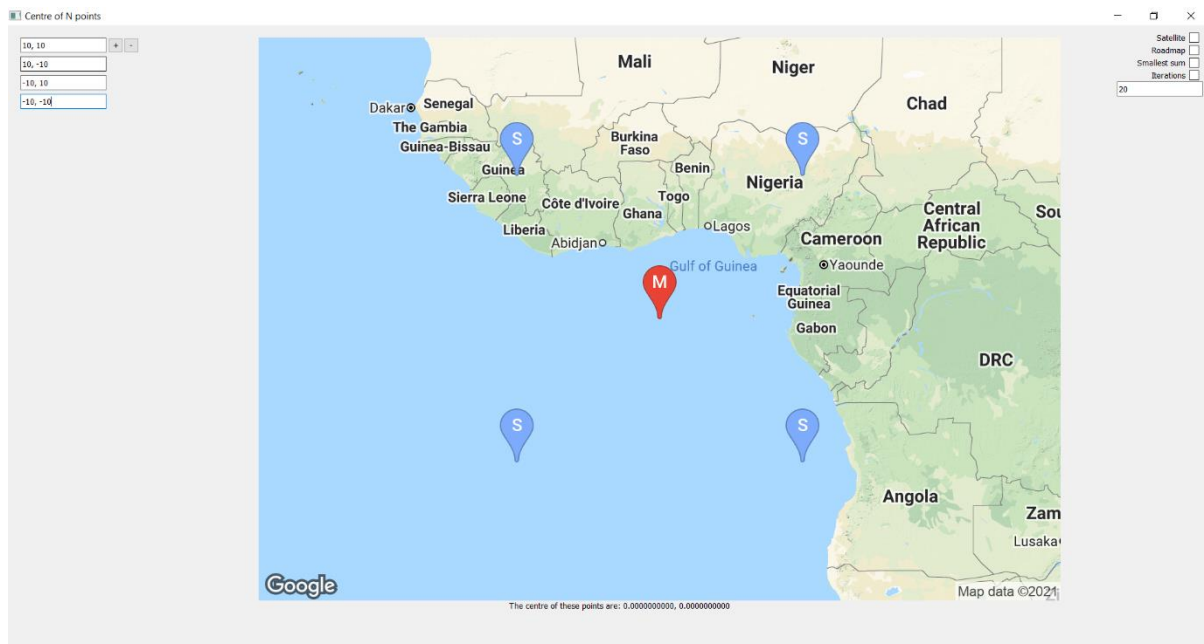
Finally, the last feature that became a must as I was debugging was to implement a feature that allowed me and the user to alter the accuracy. The accuracy was the number of times that the iteration programs would repeat to centre in on the point, the more times it ran the closer it got to the solution (in most cases). This was useful to make variable in the program, as it meant when something seemingly was still being centred in on, I could increase the accuracy accordingly. As we will see from my testing, I thoroughly look into how accurate the program needs to be, for now though I had it set as a default of 100 if the textbox was left blank or non-integer. The way this was done was with a simple if statement at the start of the on_returnPressed function, where it accessed the text of the textbox, and made the decision off of that.

Examples:

So, before going on, it would be best to demonstrate the new functionality of the project with some examples. First, we want an example we know the centre of; this can be done

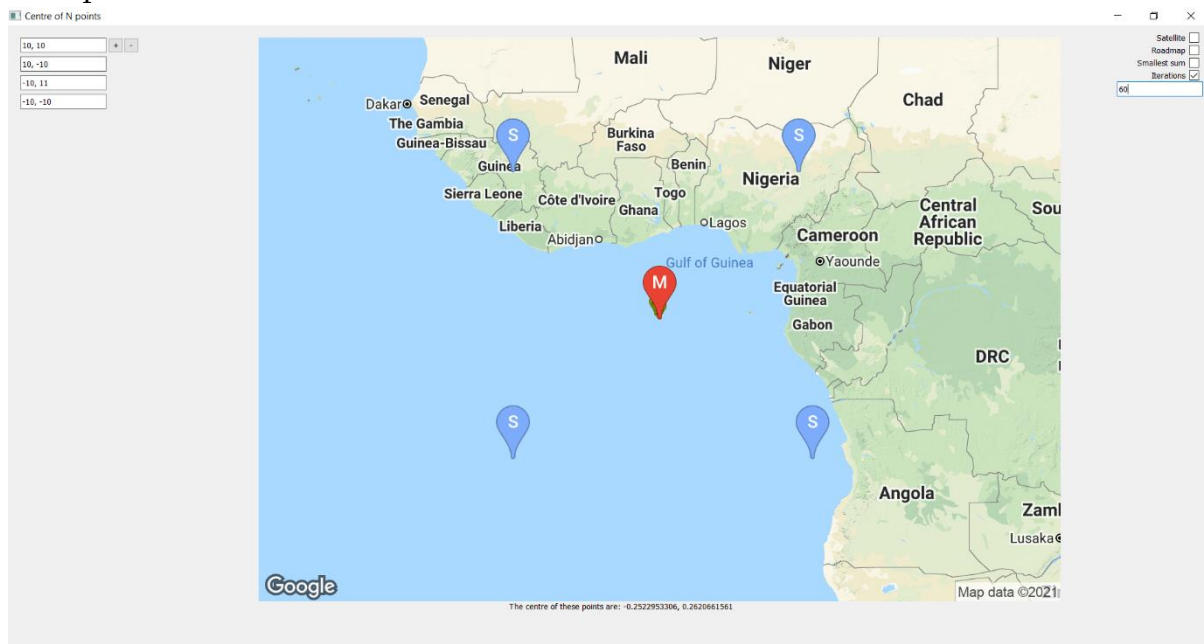
with symmetry, having 4 points of plus or minus of any number of degrees; the points being entered in polar coordinate form.

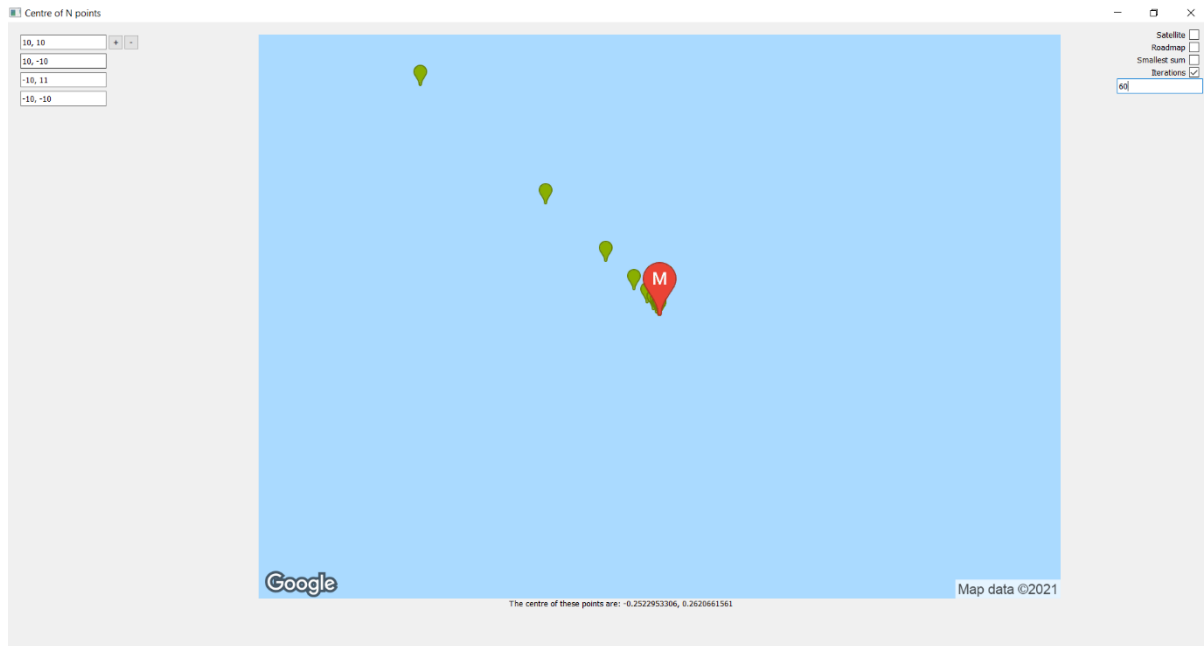
Example 1:



As we can see above this example has four points that centre exactly at 0, 0. These points are obviously equal distance away from each other, and we can see that as the azimuth angles between them are 10, 10 always.

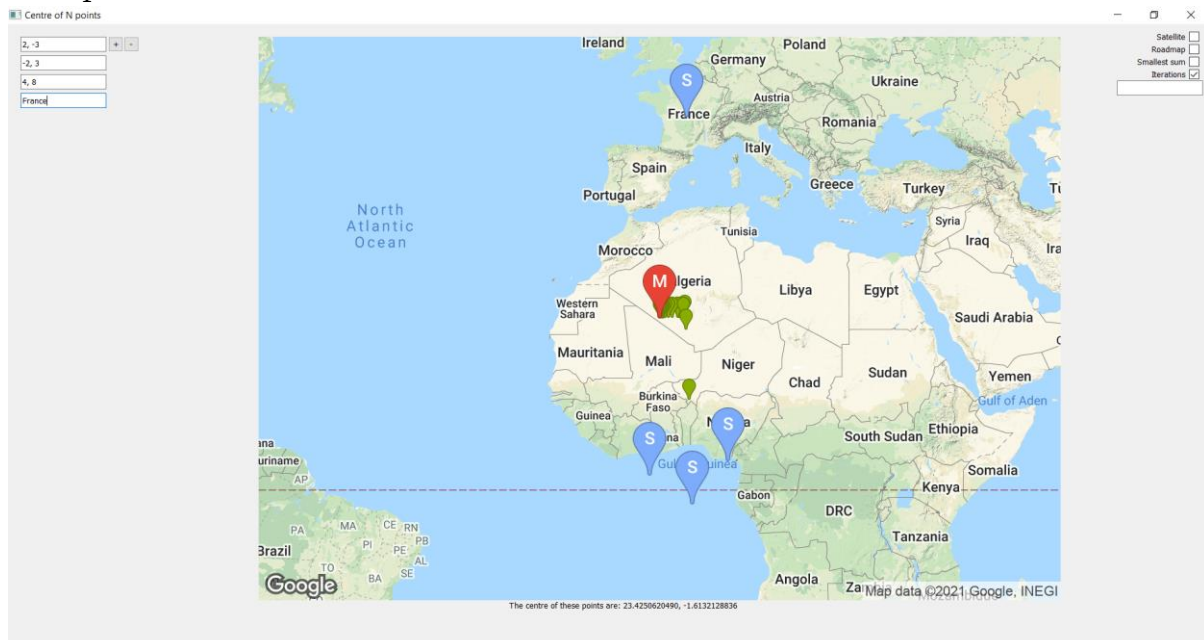
Example 2:

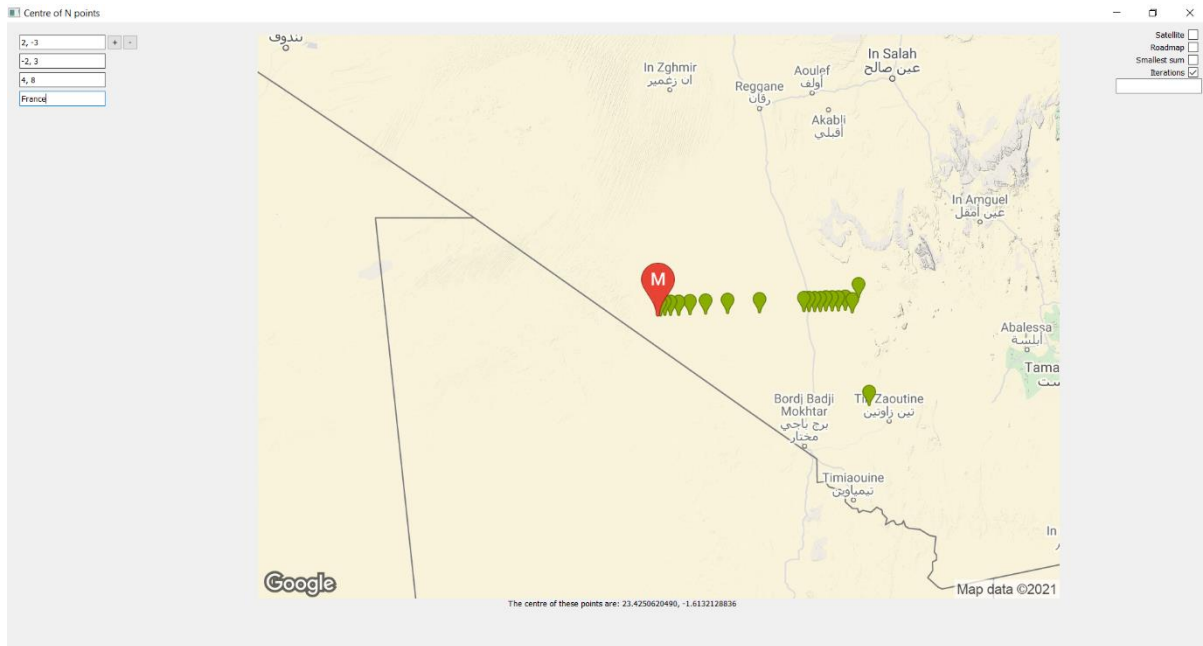




When looking at this next example it is a small variation of the previous one, by making one of the angles 11. This creates a skew in the answer, that can be seen in the small variation of 0, 0 to -0.2522953305623975, 0.26206615605396005. The iterations are shown by the green markers, and the first one centres at 0, 0 and iterates toward those points with an accuracy of 60 (any more it becomes a negligible change in decimal places).

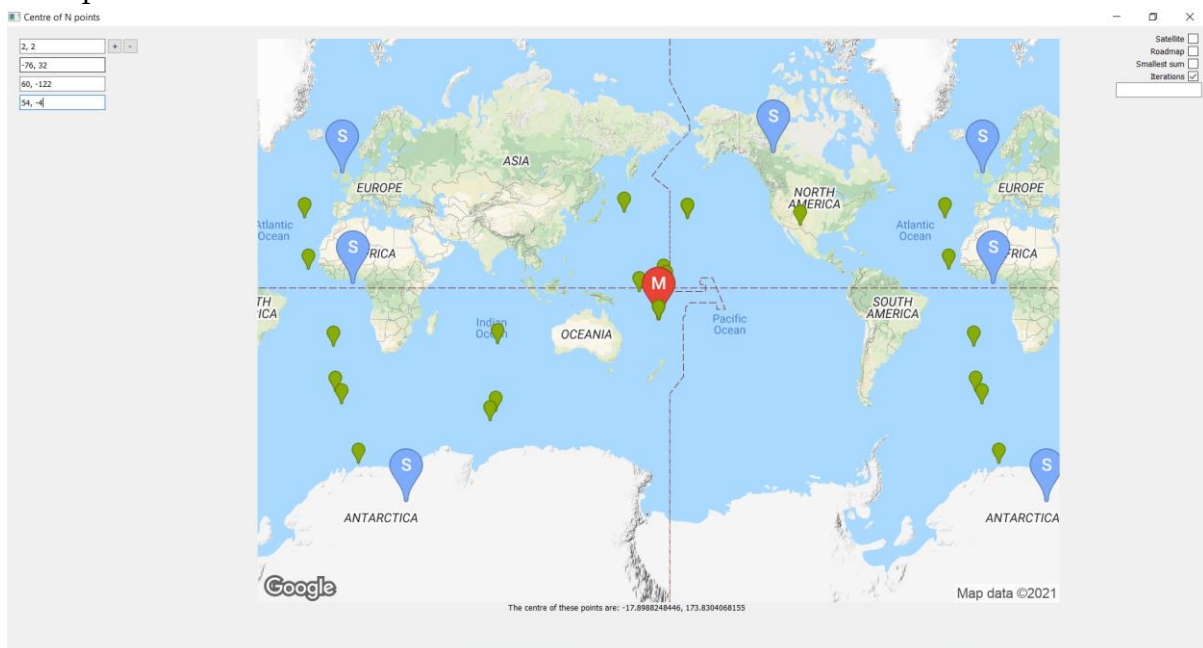
Example 3:





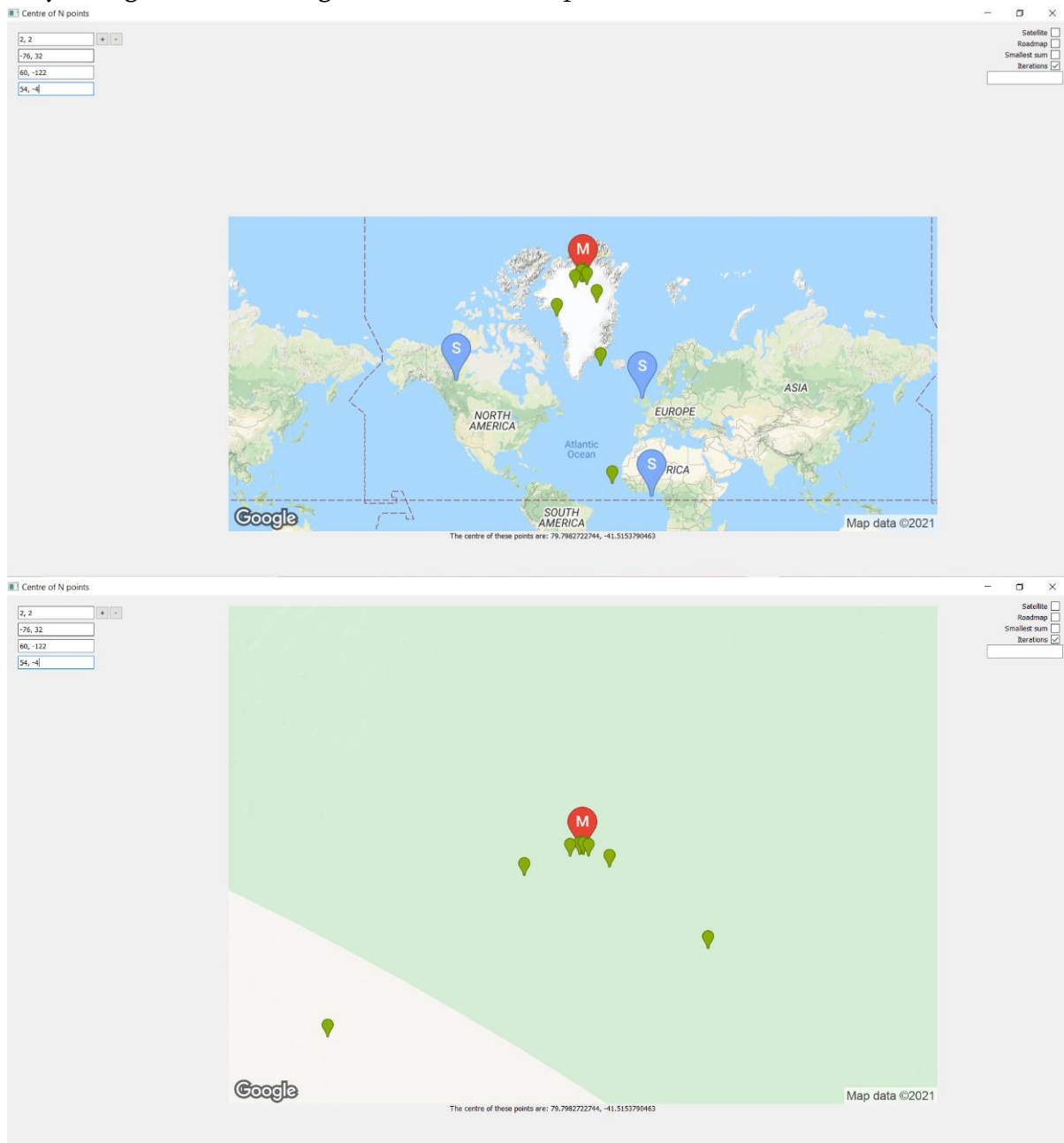
This example began to showcase some of the weirdness of the program, showing how the centre looked like it was closing in on a single point, only to race off again and begin to close in on another point (which when left running does actually tend to that point still, ending up being [23.421256764673974, -1.7378211097831022]).

Example 4:



This example helps to show the limitations in this program, but also a unique phenomenon that seems to repeatedly appear across these cases, where 2 or 3 points repeatedly repeat over and over. Seemingly it's a cycle where inputting one point, gives another point out of it, which ends up giving up the first point put in, meaning its trapped into an infinite cycle, which really isn't good, especially for any future improvements that may run indefinitely until a common pattern is found. However, it gets even more

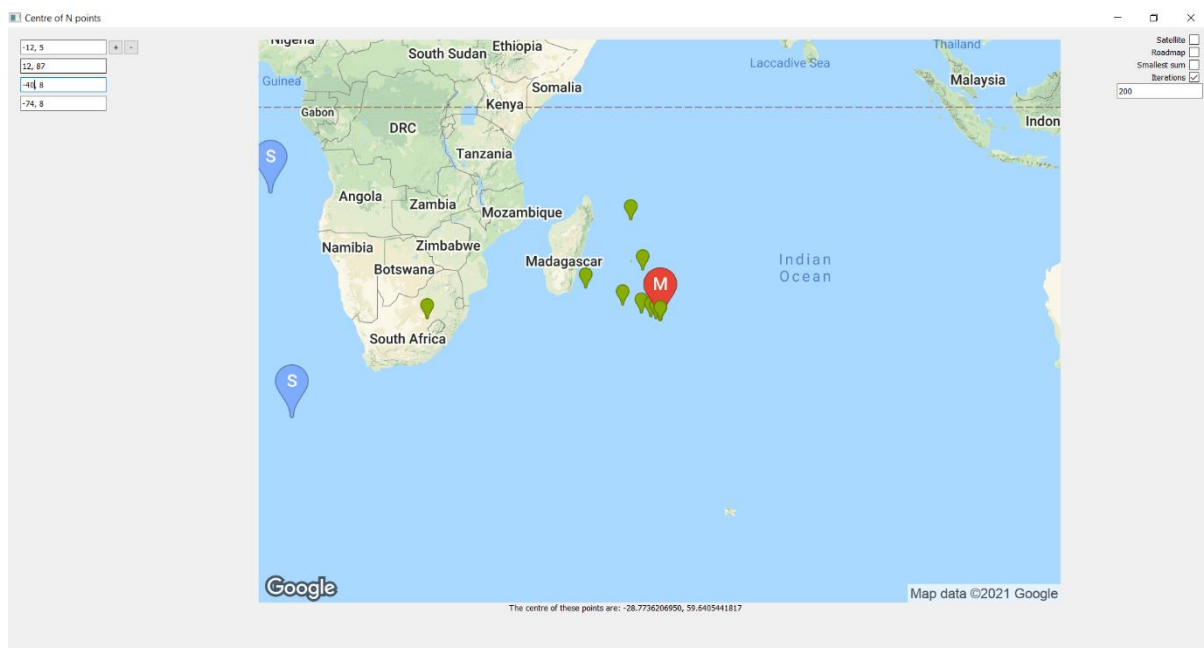
interesting when we observe what happens if we use the other method, we find that it eventually gets into a loop of 2 (aka a double loop), but it happens at a point that variance only changes when looking at the 13th decimal place.



Example 5:

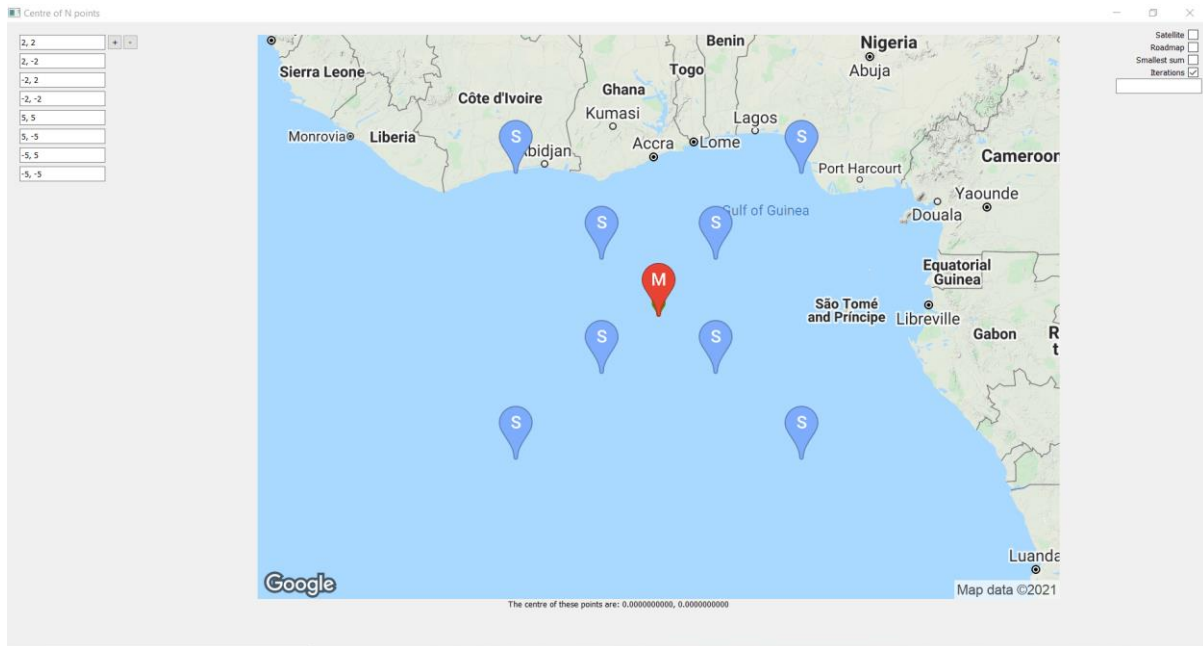


This next one is strange simply for the nature at how it approaches its target, as we see it stringing out in two, closing in together, before spreading out once more to then approach to the target. The strange part is how this comes from two locations splitting off like two tails- or the points oscillating up and down like a wave along the equator.



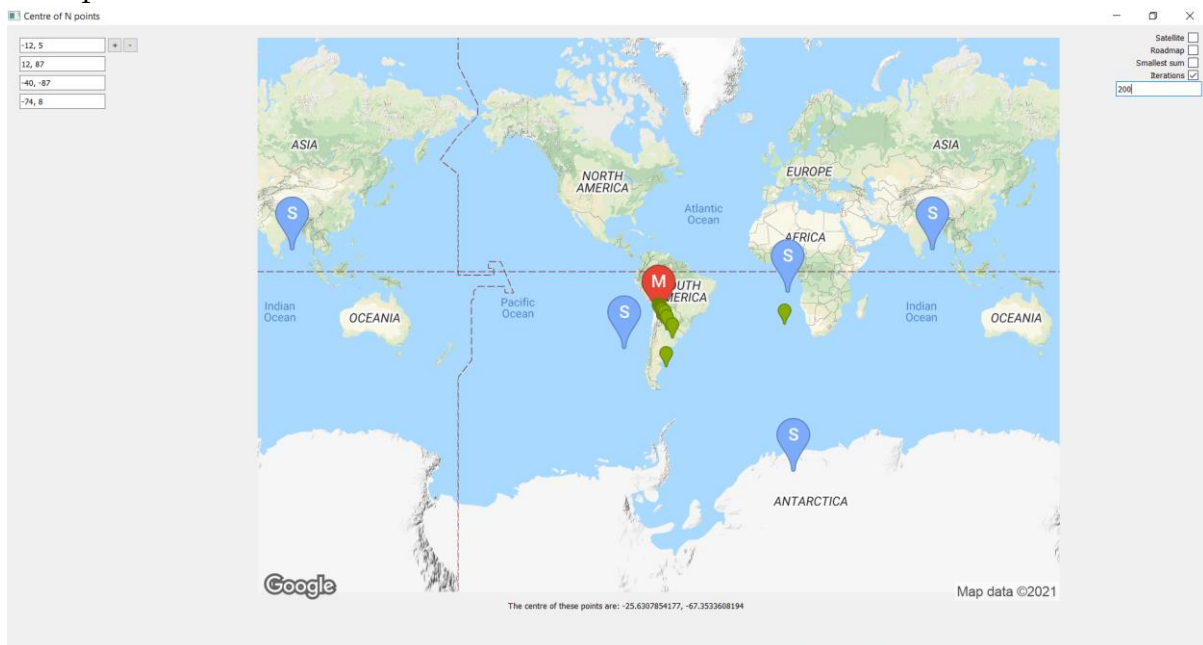
We see it happen again here, in an extremely random and almost nonsensical way. My guess is that rather than the solution coming from two directions, it's rather the point is bouncing between planes that are guiding it toward the solution, which is why it looks like two paths- when it is actually oscillations of sorts, with a damping effect.

Example 6:

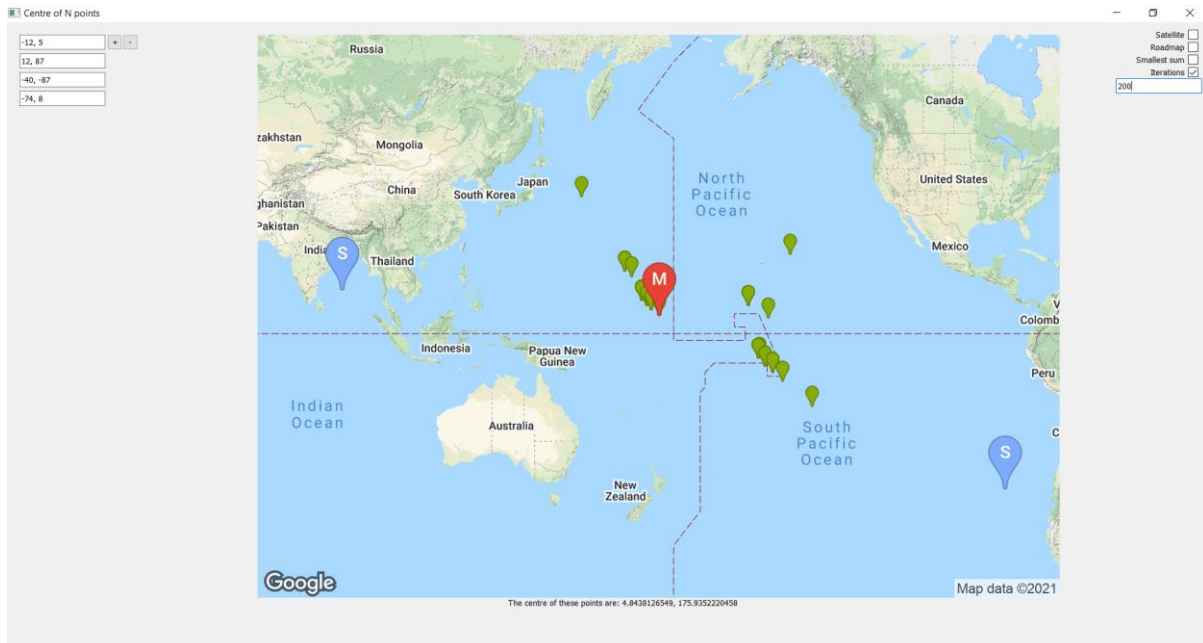


This example is one that is useful to help demonstrate the prowess of this variance minimizer. Given 8 points, it takes into account all of them and sees quite obviously that 0,0 is the point that minimizes the variances in distances. If we were to view this in a data set, we would see two sets of 4 equal distances, which makes sense then that that is the point of minimum variance, as effectively there is a single change for two data points.

Example 7:



This next example is one where we aren't fully sure if it's allowed to work or not. See this one actually has both variance formulas decreasing, which means that it we don't know how to decide which formula. Well, above is using method 1, and when using method 2...



Not only is the map ugly and the points all over the place, but so is the variance, as if we look at the console, we see that variance goes up down up down over and over.

```

Object-test.py
Run: Object-test
55492784.18528841
[-28.896918188388076, -141.11567023576632]
41539685.18185452
[7.631635363461949, -159.0863393928189]
45861955.21848337
[17.077796565448708, 166.31498157865832]
39811684.89346493
[-6.944166186119208, -156.25349626783728]
41945457.37262373
[9.188755195989574, 171.03184517141418]
48185426.22359482
[-9.273422347477652, -154.26473849623645]
48599158.73088398
[7.667587488947212, 172.3913636228782]
58348662.25777581
[-11.01987338561337, -152.26834588275666]
39398893.96591961
[6.421915517123895, 173.65239674337226]
52384268.485458914
[-13.48788274828898, -149.40625861540638]
37898564.85879786
[4.843812654857846, 175.93522204577144]
55492784.18328785
[-28.896918188385785, -141.11567023576941]
41539685.181858346
[7.631635363465864, -159.08633939283887]
45861955.21849278
[17.077796565448704, 166.31498157865442]
39811684.89346355
[-6.944166186119652, -156.25349626783796]
41945457.37262413
[9.188755195989498, 171.03184517141426]

```

This helps show that the program isn't perfect, it has certain flaws and errors that it can't deal with, and currently I am unsure as to how to tackle them. The best way forward is to see what my stakeholder says and refine the program in the testing stage.

V2.4 Third Iteration Review:

Upon having tested and experimented with the results of the third iterations as shown with the examples above, I am now confident in returning to my stakeholder to showcase the proof and efforts I have taken to find the centre of n points. With a dedicated period after class, I demonstrated my findings by using the large white board as I did with the proof of the first iteration. This saw me to derive my formula, finding the derivatives of

the cross and dot product, using the quotient rule on them, and then rearranging to the formula we all know and love. It was very cathartic to finally show off all I had learnt, as it was a real culmination of all of the mathematics, I had learnt throughout the project. My computer science skills were also put on display as I discussed the data structures I used to define and abstract the problem to a solvable form.

It was particularly nice to discuss the problem with someone that understood the mathematics, and my stakeholder divulged a great deal of interest in the mathematics and reasoning behind everything I did. It was very satisfying to reach this point in the project, where I could confidently discuss the high undergraduate degree level maths that I had poured hours into researching.

He was also very appreciative of the updated and intuitive UI, finding the ability to add in any number of textboxes and inputs to be very accessible and easy to do for the user, alongside the new additions of the summation mode and the accuracy textbox. It made it easier for the two of us to look at the results and outcomes and discuss the potential meaning behind the results. I showed him several examples, all of which being the ones disclosed in the previous segment in which each one elicited an interesting conversation about the nature of the formulae.

Example 1 was interesting to discuss as it was my way of showing off the project and its functionality. It made a lot of sense to see how the program initially found the point 0, 0 as it showed when there is an actual defined centre, it tends toward it.

Example 2 exemplified this by having that small change show how despite there being now defined centre point, there is a point that minimises the change. It was also a good introduction as to how the iterations work, showing it curve and tend toward a straight line toward the actual point.

Example 3 was where the discussion got interesting as we saw how the program's points curved away from the solution, before turning and then heading in a straight line toward the solution. This was something we were both confused by as most iteration formulas have the difference to the solution get smaller and smaller each time, however, when looking at this one its movements go against that idea, despite it always tending toward the correct answer. Frustratingly it is unlikely that I will be able to interpret why, however one suspicion I have is that it could be a result of the approximation to the Earth's radius. At the end of the function I find a unit vector and then multiply it by the radius of the Earth, which means that it can be re-entered as a point object as otherwise there would be errors, and in theory this shouldn't affect the outcome as the directions of the vectors are what are important with respect to each other, but there is a chance that that is the reason it heads in a specific direction until turning rapidly to another one, as

potentially the radius changes made it really sensitive to small changes. However, that can't be confirmed, and will be explored more in deeper testing.

Example 4 was an important one then to showcase the limitations of the program, as it showcased a scenario where both formulas has decreasing variance, but when trying it, it resulted in the constant switching of two points. This is another unexplained phenomenon which basically gives two results that it constantly switches between, something that he admitted he didn't know how that would work in an iteration formula.

Example 5 was one he was really interested in trying to understand when considering its weird nature, and how it definitely did converge, but not in a way that looked like it made much sense. Considering all of these examples, it felt like results could be grouped into classes, but so far, I can't find justifications for each class. My stakeholder thought similarly, thinking that if they can be classified it would be a definitive way to find out if there is a point of convergence for those weird examples.

Example 6 was a nicer one though that made my stakeholder smile. He found it strong evidence to show that the program was doing exactly what it was supposed to be doing, regardless of the number of points inputted, being strong visual evidence of the program's success.

Finally, Example 7 was another weird one that Mr Banks found to be something that can be explained by divergent results, saying how the position it started in could make the iteration tend to one side or the other of diverging or converging, which means that the best improvements could be to find a way to try a variety of starting points. Frustratingly it is difficult to pinpoint how to determine a starting point when we can't view what the graph of the iteration formula looks like. It's so hard because it's so abstract and almost nonsensical.

After all of the examples though, the main feeling from my stakeholder was that he was very happy with the resulting product. It was a success, even if there were refinements that could be made. He specified that the time it took for longer approximations definitely needed some form of improvement, which perhaps could be best optimised with a better starting point. Next, some form of greater classification system would be useful in determining the course of action the program should take, the current method only accounting for basic changes in variance. Other than those small areas that could be optimized, the program in its third iteration is highly successful, and with the deadline coming up, it is optimal time to try and make these quality-of-life improvements, after extensive testing.

Testing:

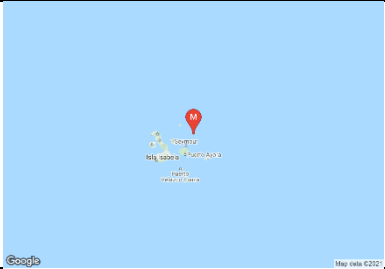
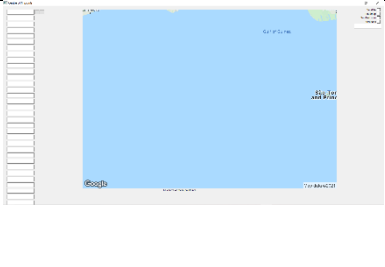
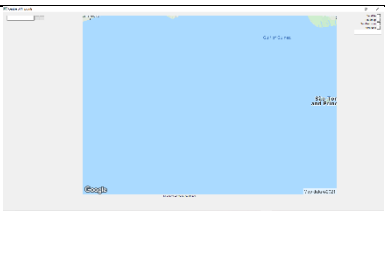
With the project in its final stages, the main thing to do is to complete some final testing of all of the functions testing boundary conditions and additional more questionable inputs, ensuring that the program responds accordingly, I firstly need to test all of the functions myself, testing the result when certain unexpected inputs are taken, and creating an appropriate contingency for each example, likely just adding in an error message for when no centre point can be found. These will be planned out and a very direct attempt to exploit the potential problems in the program, being my whitebox testing of the program with full knowledge of its internal workings. Then I shall be going back to a couple of friends, and letting them try and use the program, as a way to perform blackbox testing, finding any errors I may not have accounted for as they would have no knowledge of the internal system within. All of the functions have had some level of base testing, in which I have just been entering random values and seeing if it gibes the result, it should, as should be seen throughout all of my development, so to specify these tests are for ones that *can* be entered (for example fromPolar will never have its input of polar not be in the form that is required due to how the program is built), and these inputs need to be stuff that is testing the boundaries of conditions as well as potential program breaking values (ones which result in dividing by 0).

Function	Test values	Test results
fromPolar(polar) This is primarily testing the boundaries of 90 and 180, the limits of latitude and longitude.	polar = [90, 180]	cartesian = [-3.90111238e-13 4.77748479e-29 6.37100000e+03] ≈ [0, 0, 6371]
	polar = [-90, 180]	cartesian ≈ [0, 0, -6371]
	polar = [90, -180]	cartesian ≈ [0, 0, 6371]
	polar = [-90, -180]	cartesian ≈ [0, 0, -6371]
fromCartesian(cartesian) Whenever something is entered into fromCartesian it is first changed to make the vector on the sphere. This means the arcsin function won't ever return an error. Therefore no boundary conditions present.	NA	NA
FromAddress(address) In the case of an address that doesn't exist it deems it false so that the app knows to return an error.	address = "sksksksksksk"	P = False (atrrributes don't exist)
	address = "Zzyzx Road" (I just thought this road name was absurd and I love it's a real address)	polar = [35.14278, -116.10389]

centre2(P1, P2) This is simply the test of antinodes that are 180 degrees apart from each other, meaning the centre can fall either direction of the P1 P2 line. What this implies is that they there should be two points equally distanced from these, meaning 180, 0 should be a viable point, the only reason it takes 0, 0 is because it takes the initial value when doing sin of 0, not the +180k.	P1 = fromCartesian([0, 0, 6371]) P2 = fromCartesian([0, 0, -6371])	Centre.polar = [0,0]
centre3() This poses a huge issue, that the division by zero is caused by the perpendicular plane having values equal to 0. When figuring out why this is, we can think about it in a 2d cartesian way first. The way that we first found the centre was using the fact a triangle will always be cyclic, however a line isn't a triangle, and that holds true in spherical geometry.	P1 = P2 = fromPolar(0, -5) P3 = fromPolar(0, 5)	[nan, nan] This returned an error... which is evidently a problem. When looking into it more this is a result when there is a division by zero.
	P1 = fromPolar(0, -5) P2 = fromPolar(0, 5) P3 = fromPolar(0, 10)	[nan, nan] This is another division by zero, and seems to be true whenever it happens in a line.
	P1 = fromPolar(-5, 0) P2 = fromPolar(5, 0) P3 = fromPolar(10, 0)	[nan, nan] Another division by zero, happening in a line vertically this time.
createPerpendicular(P1, P2)	No inputs cause this to error (Effective extension of the centre of 3 function).	NA
arcdistance(P1, P2) Arcdistance is found with cross/dot, which means that when dot product = 0 then this should return an error. Dot product = 0 when two vectors are perpendicular.	P1 = fromCartesian([6371, 0, 0]) P2 = fromCartesian([0, 6371, 0])	RuntimeWarning: divide by zero encountered in double_scalars temp = np.linalg.norm(cross) / dot 10007.543398010286
	P1 = fromCartesian([0, -6371, 0])	-0.0

Interestingly when inputting two perpendicular vectors, we see it produces an error, but also an answer. This is because when dividing something by zero it effectively becomes infinity, and the limit of $\text{atan}(\text{infinity})$ is $\pi/2$, and dividing 10007.54... by $\pi/2 = 6371$. Additionally we have to deal with when the cross product is 0, which happens when the angle between the two vectors is either 0 or 180. In the case of it being 180 this is a problem as you aren't travelling 0 distance when going half way around the world.	$P2 = \text{fromCartesian}([0, 6371, 0])$	
<code>varianceCheck(C)</code> Only possible errors are when length of pointlist = 0, which wouldn't need to be done anyway so the function is safe.	NA	NA
<code>distanceCheck(C)</code> Extension of <code>arcdistance</code>.	NA	NA
<code>summationCheck(C)</code> Extension of <code>arcdistance</code>.	NA	NA
<code>s1(V, C)</code> This is $1/(V C)^2$ which means when V or $C = 0$ we have a problem, fortunately this can't happen as it will only occur if centred at O.	NA	NA
<code>s2(V, C)</code> Defined as $V \cdot C / V \times C$, this is a problem when $V \times C = 0$, which happens when in the same direction or opposite direction, hence it isn't surprising when putting in two opposite	$C = \text{fromCartesian}([0, -6371, 0])$ $V = \text{fromCartesian}([0, 6371, 0])$	RuntimeWarning: divide by zero encountered in <code>true_divide</code> <pre>s2 = np.dot(V.cartesian, C.cartesian) / np.linalg.norm(np.cross(V.cartesian, C.cartesian))</pre> -inf

direction it gives positive or negative infinity		
s3(V, C) Defined as $ V \times C $ no potential errors possible.	NA	NA
K(V, C) Defined as a matrix with no divisions so no room for errors.	NA	NA
createOmega1(C)/ createOmega2(C) Dependent on l, s1, s2, and K, meaning this doesn't need to be checked.	NA	NA
createDelta1(C)/ createDelta2(C) Dependent on l, s1, and s3, meaning this doesn't need to be checked.	NA	NA
centreN1(C)/centreN2(C) When doing the inverse of certain matrices, it returns an error as there is a division by the determinant. The reciprocal of the determinant is multiplied by the matrix formed, so if the determinant = 0 then it will return an error of being a singular matrix	omega1/omega2 = null matrix	numpy.linalg.LinAlgError: Singular matrix
minimumDistanceTravelled(C) Dependent on s1, s2, K and s3, meaning it doesn't need to be checked (only potential logic errors to be investigated).	NA	NA
centreMethod() This will be tested with logic tests of the whole program, since it's really dependent on most other programs.	NA	NA

returnImage() This works as expected even for points on the edge, which makes sense as this is primarily GoogleMaps at work.	Map(6, "terrain", Point.fromCartesian([0, -6371, 0]), [P1, P2])	
addbox() This was a simple test to see what would happen if the number of textboxes extended past the screen, and fortunately it doesn't crash.	Pressing the add button	
removebox() This was another simple test with the aim of seeing what would happen if a user tried to go less than 1 textbox, fortunately nothing happens.	Pressing the minus button	
newImgLoad() This has worked for every example I have, since its just posting an image with the UI.	NA	NA
iterations(method, detail, C) Comprised primarily of other functions so its hard for this to fail.	NA	NA
on_returnPressed() This test just checks if the inputs are valid of invalid, with specialised error messages specified if invalid.	Textbox = "sksksksksk"	Formatting Error
stage_changed() No other potential inputs not tested.	NA	NA
wheelEvent(event: QWheelEvent) When testing the scroll feature it was found it could actually go out to zoom = 0 by scrolling fast	Scrolling really fast	

enough, and because it changes by frame of scroll it means leaving it like this without touching the scroll will cause it to remain like this.		
--	--	--

When looking at the testing of these functions we find that many of them later on are near fully dependent on the earlier functions, which means that there is little area for errors now once the main problems have been solved, which means there are roughly 4 functions that need amendments and improvements.

3D shapes with a volume of zero:

Firstly, is the centre of three function, which has the huge oversight that it doesn't work when they are in a line. This seems to hold true when it lies on a great circle, which makes sense as it no longer is a triangle of points, a line passing through all of them, I.e., the angle between them not being greater than, which is impossible to fit a circle around. So, this is where we use the minimizing arclength formula once more, though interestingly when performing the maths proof of the one-dimensional minimization of variance is actually impossible, the proof being shown below.

$$\sigma^2 = \frac{\sum_{i=0}^n (C - V_i)^2}{n} - \frac{(\sum_{i=0}^n (C - V_i))^2}{n^2}$$

$$\frac{\partial \sigma^2}{\partial C} = \frac{\sum_{i=0}^n 2(C - V_i)}{n} - \frac{2(\sum_{i=0}^n (C - V_i))(n)}{n^2}$$

$$\frac{\partial \sigma^2}{\partial C} = \frac{2 \sum_{i=0}^n (C - V_i)}{n} - \frac{2 \sum_{i=0}^n (C - V_i)}{n} = 0$$

This made me worry that for a spherical space that it wouldn't work out nicely, though interestingly enough it does work for spherical geometry, which implies that the natural number line is not spherical, which is always nice to know. Hence, we need to implement an if statement where if all 3 points lie on a great circle, it needs to use the approximation formula over the linear algebra one. The solution to this is the volume of a tetrahedron. See if three points lie on a great circle it means that they are all coplanar with the origin, hence we can test this by finding the volume of 4 points, as the volume of a plane should equal 0, which can be done with a very nice formula:

$$V = \frac{|(\mathbf{a} - \mathbf{d}) \cdot ((\mathbf{b} - \mathbf{d}) \times (\mathbf{c} - \mathbf{d}))|}{6}.$$

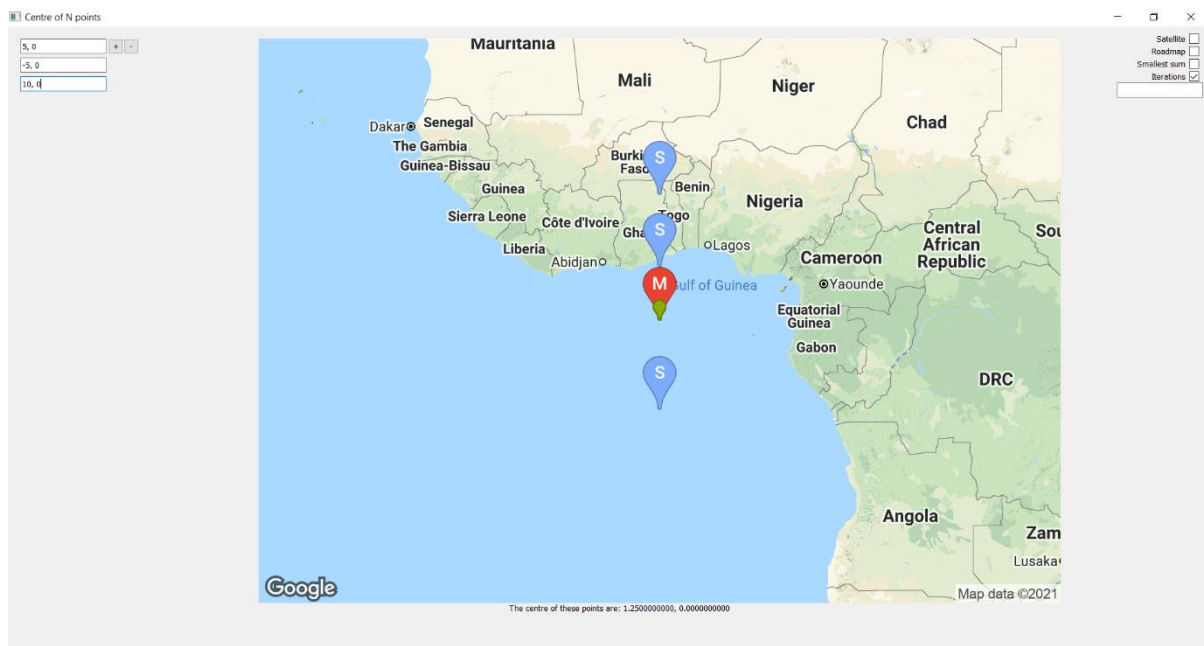
Fortunately, this can be written even nicer as a determinant of a 4x4 matrix, even better though is that one of these points will be a constant of the origin, which would mean that it reduces down into the 3x3 matrix seen below.

$$V = \begin{vmatrix} V_{11} & V_{21} & V_{31} \\ V_{12} & V_{22} & V_{32} \\ V_{13} & V_{23} & V_{33} \end{vmatrix}$$

This is very nice for the computer to calculate, and it means that we can create an if statement saying that if the volume of three given points and the origin should equal zero, and if it does, we can conclude we need to use the minimum variance formula, the code for this being seen below:

```
1. def planeVolume(self, P1, P2, P3):
2.     matrix = np.array([[P1.cartesian[0],P2.cartesian[0],P3.cartesian[0]],
3.                        [P1.cartesian[1],P2.cartesian[1],P3.cartesian[1]],
4.                        [P1.cartesian[2],P2.cartesian[2],P3.cartesian[2]]] )
5.     volume = np.linalg.det(matrix)
6.     return volume
```

As intended this returns zero when they all share a plane, and works to remove any potential division by zero errors, the if statement implemented showing it worked below:



Next function to tackle is the arcdistance function, and being sure to put in a simple set of conditions: if the solution = 0 but the same points aren't entered, then it will change the value to a pre-existing one of $6371 * \pi$:

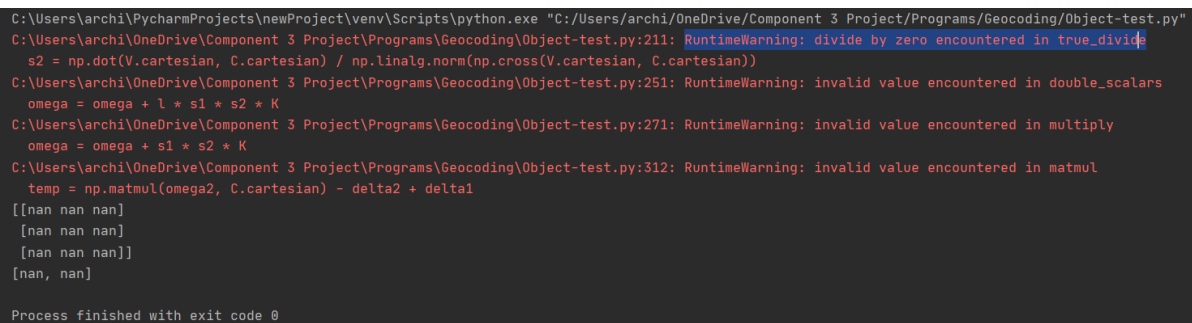
```

1. def arcdistance(self, point1, point2):
2.     cross = np.cross(point1.cartesian, point2.cartesian)
3.     dot = np.dot(point1.cartesian, point2.cartesian)
4.     temp = np.linalg.norm(cross) / dot
5.
6.     sigma = math.atan(temp)
7.     if sigma == 0 and (point1.cartesian != point2.cartesian).all():
8.         distance = R * np.pi
9.     else:
10.        distance = R * sigma
11.
12.    return distance

```

The s2 crisis:

Looking at the function of s2, then we have the problem that if this occurs, then it means that points are on opposite ends of the Earth, which is a problem as the program just doesn't know how to handle that and it will all collapse rather dramatically, which can be seen below:



```

C:\Users\archi\PycharmProjects\newProject\venv\Scripts\python.exe "C:/Users/archi/OneDrive/Component 3 Project/Programs/Geocoding/Object-test.py"
C:\Users\archi\OneDrive\Component 3 Project\Programs\Geocoding\Object-test.py:211: RuntimeWarning: divide by zero encountered in true_divide
  s2 = np.dot(V.cartesian, C.cartesian) / np.linalg.norm(np.cross(V.cartesian, C.cartesian))
C:\Users\archi\OneDrive\Component 3 Project\Programs\Geocoding\Object-test.py:251: RuntimeWarning: invalid value encountered in double_scalars
  omega = omega + 1 * s1 * s2 * K
C:\Users\archi\OneDrive\Component 3 Project\Programs\Geocoding\Object-test.py:271: RuntimeWarning: invalid value encountered in multiply
  omega = omega + s1 * s2 * K
C:\Users\archi\OneDrive\Component 3 Project\Programs\Geocoding\Object-test.py:312: RuntimeWarning: invalid value encountered in matmul
  temp = np.matmul(omega2, C.cartesian) - delta2 + delta1
[[nan nan nan]
 [nan nan nan]
 [nan nan nan]]
[nan, nan]

Process finished with exit code 0

```

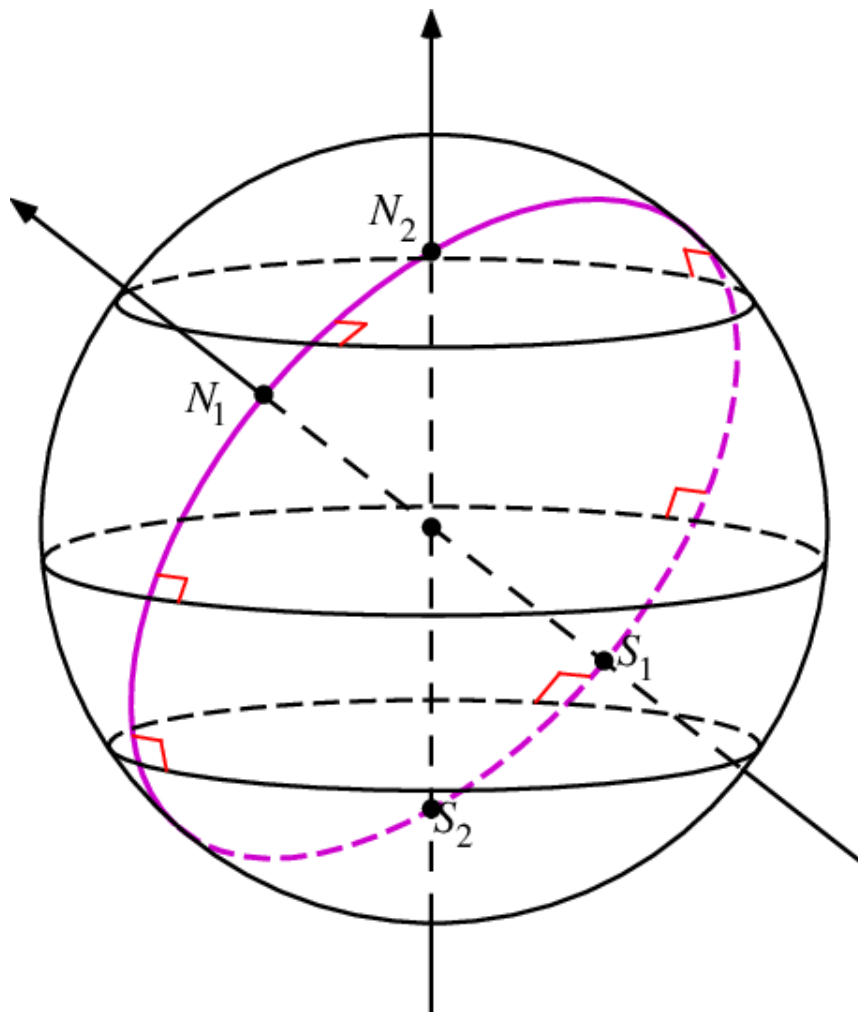
The two things printed at the end are omega and the polar coordinates of the centre it found, or more accurately didn't find. Unfortunately, this is a mathematical problem without a solution as what is effectively happening is that omega has entered an indeterminate form, and to decipher what that means in terms of the program is quite difficult in the geometrical sense. Therefore, we shall implement counter measures as to avoid it ever happening in the first place (probably). We know that it will occur when the approximated centre is equal to 0 (meaning they are opposite directions), as that is when s2 is indeterminate, and so we need to make sure that won't happen. At the moment, we have it so the starting approximation always is [0, 0], but that means if [0, 90], or [90, 0] are entered (and negative counter parts) we will see this error. Therefore, we need to make the starting centre variable, and the best way to do that is to take the Euclidean average of polar points. This will not only reduce the number of iterations taken toward finding the true centre, but will also mean that the antinode of one of the points will never come out as a centre; and for number of points greater than 2, it won't occur naturally, therefore the s2 crisis is averted.

Indeterminate determinant:

One other area an error may come up is when taking the inverse of the omega matrix. The reason this happens is because an inverse matrix is defined as the reciprocal of its

determinant, multiplied by a transformation of its matrix. The problem comes with it being a reciprocal of the determinant, meaning if the determinant is 0, that means that the output in polar form will be [nan, nan] basically saying that it tends off to infinity. A matrix's determinant is only equal to zero when it is a singular matrix, which if we were to view it in a geometrical sense, a series of 3 points with coordinates in each column would describe the vertices of a tetrahedron; if the volume is equal to zero, this means that it will error and not output any result to the user. In the example used in the table above, it started with a matrix being a null matrix, meaning it is all zeros, which has a determinant of zero. The question is if this matrix is ever possible to gather from the previous stages of the program, and sadly it is quite difficult to tell.

If we view a matrix as a transformation, where each column alters the x, y and z components independently, we can say that a singular matrix has no inverse to its transformation, meaning the transformation is noninvertible. Hence, I think we can argue that the single matrix problem will not actually come up aside for one example. Previously we saw that if three points lie on the same great circle, you can't use the linear algebra method to find the point of least variance, however you also cannot use the calculus method in the following case where the angles between each point is the same. We can think about this methodically, as if we have a pure 2D circle, and we place a point on the outside of the circle, no matter how you approach one point, you will be getting much further away from other points; the point of minimum variance is the centre, yet we can't access that in 2D, we can in 3D. It is hard to visualise in words but if we went to the peak of a great circle that is perpendicular to the great circle where all of the points lie, we will have a point where every one of them is equal distance away from. This holds true for more than just points that are equally spaced though once you start visualising it.



The image above showcases my point well, as we know that we want a line perpendicular to the great circle, and passing through the origin, and when it hits the sphere once again, that will mean every point lying on the great circle will be at an angle of $\pi/2$ radians to the centre point, meaning that we can now create a program to this solution.

Firstly, we need to quickly establish I am going to have the program take the positive cross product vector for the pole, simply because its slightly easier to work with, but I understand that there are two solutions to the problem shown by its symmetry. Next, we need to find a way to determine if points are coplanar with the origin, which will be an effective remake of the volume function. If the volume contained between $n+1$ number of points is $= 0$, then we know that they lie on a great circle. Alternatively, we need another way to see if every point lies on the same plane, so we could form a plane using 3 points, and then test if the remaining points are consistent with the plane. Initially, I looked into the process of finding the volume of a n sided shape, and it is possible using triangulation. https://en.wikipedia.org/wiki/Polygon_triangulation This Wikipedia page showed that any polygon in any space can be used to triangulate it, by converting each face into a triangle, and then by doing the sum of the area of the faces, and then the dot product of the perpendicular unit vector and any position on the face, all multiplied by a third. Yes.

Not worth doing. The problem with triangulation is determining what faces do and don't overlap, which is a project within itself. Therefore, I have decided to go down the far easier route, of taking the cross product of two points (both technically subtracting the origin), to find a plane that encapsulates the origin and two other points. Then I go through every point in the pointlist and see if the dot product of the normal vector and the point are the same for each. If they are, then that means we can return true, and if not, we can return false.

```

1. def coplanar(self):
2.     P1 = self.pointlist[0]
3.     P2 = self.pointlist[1]
4.     equal = 0
5.     perpendicular = np.cross(P1.cartesian, P2.cartesian)
6.     checkval = np.dot(P1.cartesian, perpendicular)
7.     for V in self.pointlist:
8.         if np.dot(V.cartesian, perpendicular) != checkval:
9.             equal = equal + 1
10.
11.     if equal != 0:
12.         return False
13.     else:
14.         return True

```

Now, we need to consider the case when this is true. We can work out the perpendicular vector, which describes the direction from any point that is perpendicular to the plane, which means if we use this and centre it at the origin, we can then extend the vector to the plane by finding the $t_coefficient$ again.

```

1. else: #all points lie on a great circle
2.     normal = np.cross(self.map.pointlist[0].cartesian,
3.                       self.map.pointlist[1].cartesian)
4.     t_coefficient = R / np.linalg.norm(normal)
5.     self.map.centre = Point.fromCartesian(t_coefficient * normal)

```

When using an example of 4 points all along the great circle at the equator, we find that the variance is equal to zero, which is a nice confirmation of my method:

```

643 P1 = Point.fromPolar([0, 3])
644 P2 = Point.fromPolar([0, 5])
645 P3 = Point.fromPolar([0, -8])
646 P4 = Point.fromPolar([0, 10])
647
648 map = Map(6, "terrain", Point.fromPolar([0,0]), [P1, P2, P3, P4])
649 normal = np.cross(map.pointlist[0].cartesian, map.pointlist[1].cartesian)
650 t_coefficient = R / np.linalg.norm(normal)
651 C = Point.fromCartesian(t_coefficient * normal)
652 print(map.varianceCheck(C))
653
654
655
656

```

Map > arcdistance() > else

Object-test x

C:\Users\archi\PycharmProjects\newProject\venv\Scripts\python.exe "C:/Users/archi/OneDrive/Component 3 Project/Programs/Geocoding/Object-test.py"

0.0

C:\Users\archi\OneDrive\Component 3 Project\Programs\Geocoding\Object-test.py:181: RuntimeWarning: divide by zero encountered in double_scalars

temp = np.linalg.norm(cross) / dot

Process finished with exit code 0

With this, finally I am rather confident in dealing with the devil, aka dealing with great circles. This helps showcase the benefits of testing, as by testing all of these features thoroughly for boundary conditions it helped get the optimal solution for the problem I have at hand.

Evaluation:

The evaluation. The beginning of the end. Whilst each iteration of the way through the project I have been evaluating and testing iteratively, ensuring that I know the best way to go forward with the project, now that I have reached the final product – as in the project deadline – I can now look back at the project as a whole and reflect. This project was a vast undertaking getting me to explore far outside my current knowledge, providing me with a breadth of experiences that will help in any other projects I take part in.

Evaluating the analysis:

Starting where all good stories start - the start – the early stages of the project was a lot of research, and a lot of it not as helpful as I thought it would be originally. When first beginning this project, I was working under several false pretences, first of which thinking that I was working in two dimensions! My initial response to my problem was to abstract away much of the detail, and that meant bringing it down to its base components of finding a point that was equidistant from n other points. However, this is an example of abstraction that went horribly wrong, as it abstracted away the detail that we do not live on a flat earth. This meant for a good portion in the early stages of the project I was looking at cyclic shapes in 2D, which wasn't the worst use of time, but there were certainly better areas to research.

Looking at the conditions for cyclic polygons was my first approach, but that was quickly limited to how often the formulae came up as too complicated, and more than that not helpful. Topics like Bretschneiders formula, monic polygon, the shoelace formula, all of it was extremely interesting, yet they didn't contribute toward the project in nearly any way. The only area I was truly on topic for was looking at circumcentre of a triangle. From there I spent a gross amount of time drawing quadrilaterals and figuring out where an approximate centre would lie, even trying to find code with poor and lousy approximations. Now, the problem really came from my lack of understanding in mathematics, which happened because I was still in the process of learning in my maths and further maths A level. This meant a series of areas like vectors and calculus was still completely unknown to me, and truly I didn't know I could do those things with them. This lack of knowledge is a reoccurring reason my design took so long to figure out, and why programming in certain aspects took so long, definitely showing a negative side to the project, the complex understanding of mathematics required.

This stagnation I was facing only changed because of the advice I was given by my dad, who told me to approach the problem in a less abstracted and more useful manner. It was then I began to look into various other models online... to find there weren't really any.

The few that attempted to do what I did are listed in my bibliography, however most stopped with 2 points, or most other solutions said to just take the average of points which also isn't the answer. It took looking at the example: <http://www.movable-type.co.uk/scripts/latlong.html?from=49.243824,-121.887340&to=49.235347,-121.92532> for me to understand the complexity of the problem, and how I was actually working around a sphere. It was around this time though; I began learning vector maths in my further maths lessons. This was important as it let me figure out how everything worked in 3 dimensions and is the sole reason, I was able to generate a method to find the centre of 3 points on the surface of a sphere. The key to my real start to the project was looking at previous models, albeit not doing what I wanted them to do on first glance.

Learning how to work with certain vectors, and linear algebra was an incredibly enriching experience, simply for the fact it heightened my project to a level above most previous projects by accuracy alone. This is something that was very successful about the project, as it was a formula that didn't require much more than a few cross and dot product methods.

Despite the success, I then had reached a dead end. As oppose to what I had set out to do, I was quite unconfident that I would be able to go beyond 3 points. All of my initial research into cyclic shapes told me that there wouldn't essentially be a defined point for shapes with more than 3 vertices, yet this all changed, when I had a lesson on statistics. This maths lesson introduced a new idea to me, the idea of variance. Once again now armed with new knowledge, I began the devastating crusade to try and figure how I could use this in the program. This was both a low and high of the project, as on one hand it was the potential for me to do what for a long time, I was considering to be impossible. The downside was this is where I had to go far and wide to research the topic. My teachers were unable to help me, many of them not knowing much about vector calculus aside from the basics; and I didn't know any university maths students to get in contact with; nor where to start researching. But, by diligently working through problems online, and simpler ideas of vector calculus, guided through by help by others on a mathematics server on discord, I eventually was able to figure out and understand how to find the derivative that I needed to. The process was gruelling and took a substantial amount of time to do, especially since I was learning it all while coding the first iteration of the project, but in the end the result was worth it to find the solution to the problem I set myself at the very beginning.

The analysis of the problem is one of the strongest parts of this project, as not only is it able to showcase the depth and detail I went into for each program, but also it is a really strong demonstration of how I came to the solution of the problem. The downside is the

amount of time it took, limiting how much time I could put into other aspects of the project, as well as how much of the analysis wasn't actually used.

Evaluating the design:

The design of my project wasn't very successful. The design area covered a few aspects, covering the first two iterations, yet quickly whilst programming certain sections were found to be entirely redundant and didn't work algorithmically calling for change.

Firstly, addressing the positives of the design is that it made anything mathematical in my project far easier to do and pursue. The centre of 2 and the centre of 3 having the pseudocode written for it was incredibly useful as it became a simple act of replacing pseudocode language for pythonic language and aided in ensuring I inputted all of the variables correctly within the right parentheses. Additionally, the planned use of points as objects was to me the best part of the project algorithmically, as not only did it enable me to allow for a variety of user inputs, but it also allowed for the conversion of cartesian coordinates to ones the user would understand, something essential for the program.

Beyond those positives though, there were several flaws. The largest flaw was my initial plan in the class diagram which showed how the classes were to interact with each other. Most glaringly is how I ended up doing the user interface, rather using two objects - one to represent the points and one to represent the map - I instead used a single class called app where all the information was contained. This was a result of a lack of research into the google maps API static images, which meant I didn't know the option of markers. Regardless, this made the creation of the interface far harder as there was little basis in my design that worked accurately, therefore it made learning each step far harder, and also stopped me from being able to implement certain features that would have been useful (as discussed in product evaluation).

The one area of design that aided in the creation of the user interface was the assignment of attributes to the Map class, starting off with centre and pointlist, however that soon expanded into the various details of the static map API. This planning made it far easier to implement features surrounding variable zoom levels, and alternate terrains. Overall whilst much of my far more ambitious and detailed design was scrapped and redesigned, the general plan of using object-oriented programming helped significantly. The decision between procedural and object-oriented was difficult for a long while, as I wasn't familiar with OOP, and could definitely write code more fluently in procedural, however in my design phase I realised the value of OOP, and how it lent itself to this problem, and made the solution far more efficient and of value to the user.

Whilst the Point object was certainly the most successful thing out of the design stage, it wasn't originally designed the best as we can see from the flow charts from the design phase. In those flow charts it shows a different way that it filled in information, and that was to update dependent on if the attribute was empty or not. When learning about class methods though that changed my initial plan, and I designed it around only one set of the information being input, as to ensure no clashes in data. This change was a necessary one and was bound to happen simply because I didn't know about the possibilities of class methods until I began designing the pseudocode.

The design also had one other additional aspect which was useful and that was the planned test data. Throughout the development these were tested, and it allowed for any small errors and issues to be amended throughout, for example when testing the midpoint of three locations using only addresses, it found the distances were different drastically, which ended up me finding that the degrees hadn't been converted to radians in the fromZip function. The test data uses came to ahead at the end of each cycle, each time, I tested the values and saw what came from them, as well as often times following the math and ensuring that each calculation it did was correct.

Evaluating the development:

The development of my project was an interesting one in how it followed its different sprints of each iteration. Each sprint had its own share of advantages and complications, which made them take longer or much shorter. The first sprint was the most successful, with a tightly planned design and pseudocode, making the point object with its various retrieval methods made it easy to make and execute, with most facets having little hiccups aside from forgetting to convert between radians and degrees – quickly found out via testing. One of the biggest changes though in this first sprint was how I retrieved the geographical information from the google maps API; at first using .geocode from the library, only to change to json data accessed on a website, which turned out to be very successful. This was important as it allowed me to gather the status of the location, knowing if an address existed or didn't exist.

The second sprint surrounded the Map class, and similarly this faced little difficulty due to the tight design, but in retrospect there were a few areas of it that weren't the best in terms of coding. When delving into the more mathematical side of things, the variable names lost a little bit of meaning, namely what was meant by perpendicular and what was meant by the t_coefficient, making the comments have to be more essential and descriptive as to what the program was doing. This was also the sprint where I encountered the many uses of NumPy, a library that became very essential later on.

The other major area of the first iteration was surrounding the 'lost programs', as in the programs I didn't think I could get to work. These weren't planned thoroughly with pseudocode, nor were the variables used that appropriate, and throughout there was a poor use of code, not using the various pythonic commands I had at my disposal to improve the quality. However, during the time it felt more essential to try and get a piece of working code to come out. As I didn't know if the algorithm would work, since it was based off of maths that not only, I wasn't familiar with, but also maths I couldn't test without programming it. Therefore, the code quality and development of this sprint was incredibly poor, and the third iteration helped massively in improving it.

The second iteration was most likely the most involved development process, as a lot of the coding was figuring out small details of the GUI, and how best to implement the static maps. Learning how to use PyQt made up most of the first sprint, making sure that I could display a window, textboxes and images. Doing this part of the development was really reassuring, as up to this point it had felt there was little algorithmic or computational thought behind what I was doing, just maths. The GUI and the complex demands of the third iteration though enabled for the complexities of algorithmic thinking this project needed.

The second iteration also posed me with many new opportunities to learn new coding skills. This iteration saw me working with regex which was excellent for string manipulation, as well as layouts of GUIs and retrieving images off of the internet using urllibs. Learning the best way to manipulate the look of the static maps with markers was incredibly insightful and interesting, even if at times it was frustrating with the guide having somewhat unclear syntax. What was satisfying though was letting it update in real time to the user's intended scroll, as this made me feel like the user now had so much more control over the program. Similarly, maptypes added in a whole layer of depth that hadn't been achieved before, with the satellite mode letting me see actual locations like the actual google maps. Throughout, google maps was a massive source of inspiration for how I designed my product, but this will be more explored in the product evaluation.

The third and final iteration was an important one, as it is the one that completes the goal that I set out to do originally. There was a large amount of catharsis and satisfaction in seeing the program work, knowing that the maths behind it all was correct. There are a few elements though of the coding that may not have been ideal, in particular when we look at how the functions were written, they may not have been done in the most optimized way, what I mean by that is there are some bits of repeated code, and perhaps the equations could have been rearranged in a way that would be simpler to compute, making the program faster as a whole. Moreover, when looking at the names of variables it may be difficult to understand what they mean, and that's because I don't fully know

what they mean. Many were given references such as scalar1, scalar2, etc. in the original proofs so that the formulae may actually fit on my page to write. Despite the flaws in the first write up of the program, it was successful, and a massive improvement from the lost programs version which saw poor implementation of the sums, this version using the for element in array feature. Although the improvements were massive, it took several attempts for me to make the function follow the method I desired, and even then, it isn't in its optimised form. There are chunks of repeated code which I am unsure on how to mitigate, mostly looking at when we have two versions of each function that creates delta, omega and the centre of N. There should be some way to change these functions to work dependent on the input without excessive use of repeated code, yet whenever I tried to implement this there would be a variety of errors, mostly in determining which scalar values to use when. It was a difficult trade-off between an enormous amount of if statements, or two specially designated functions. From here, there was a lot more complex decision making that the computer had to. This involved how it decided which method it was going to follow, and this needed effectively 4 options. This used the changing variance of the program to identify which was the best method to go down and will actually tend toward minimum variance. Unfortunately, when the other two cases arise, a good method for what the computer should do hasn't been figured out yet.

The bits of the GUI that were left to implement to accommodate the now larger amount of data it needed to handle, was arguably the most unnecessarily challenging bit of the development. This saw five different attempts at trying to add in and remove textboxes, originally starting with an exceptionally complicated method of only removing empty boxes and retaining the data within the non-empty boxes. This was incredibly stupid and a waste of time; and then I tried to implement a textbox which would have a number entered into it which would then create that many textboxes.... Yes. It was another stupid approach. Fortunately, I realised sense soon after and created a simple system that saw the easy addition and removal of textboxes. This isn't perfect however, as there wasn't an implementation accounting for more than 33 textboxes, making the algorithm (that the user can access) limited to up to 33 points, not n points.

Despite that complication, there was a real success with how the iterations and accuracy system was made, where an iteration list would have every nth point calculated each iteration, and this would then add on this point to the URL as a marker. Additionally, the implementation of the minimum total distance was also useful as not only was it something had been calculated prior, but it was arguably more useful in an applied context, so it was something that helped enhance the program and its uses.

To conclude, my development was very successful despite certain hiccups, as not only did the development see improvements in my code algorithmically, but also allowed for the

development of features that aided toward the program being more successful in me and my stakeholder observing the phenomena at play. Additions like the iterations, and the accuracy input was a great debugging tool for testing and investigating the various outputs and results.

Evaluating the final product:

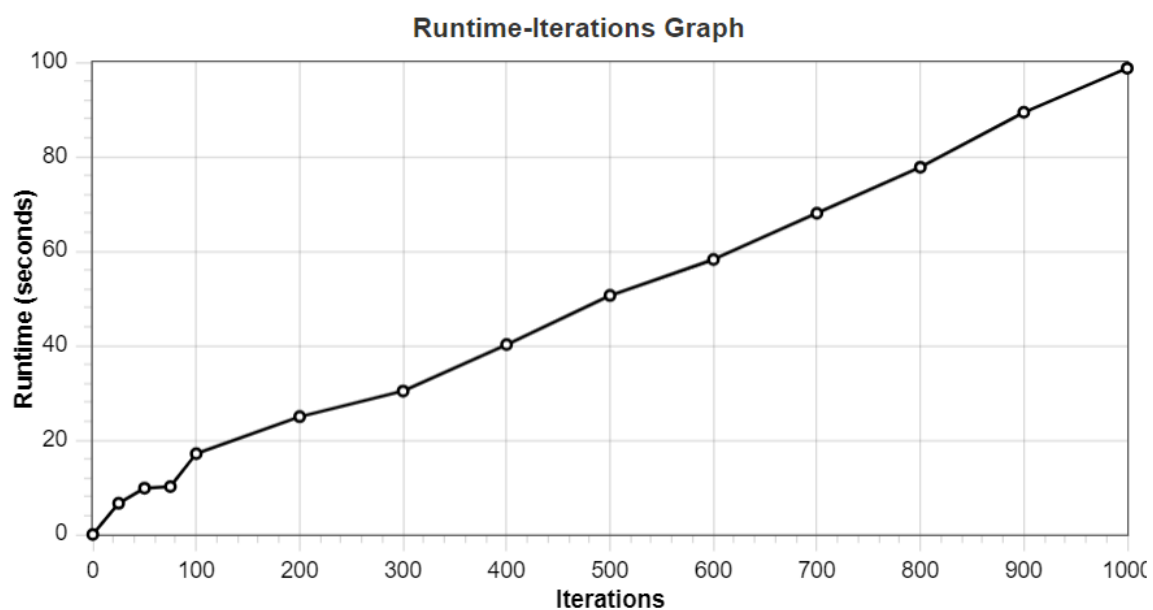
My final product that came out of my project is not perfect, having many various areas that weren't completed due to the time frame, and the priorities of the stakeholder; the product having many strengths and weaknesses for what it achieved. In a base sense, the product succeeded in being able to find the centre of n number of points, with an algorithm figured out and programmed, with an interactive graphical interface that is easy for the user/stakeholder to understand and follow. In a purely mathematical sense, the project every area was achieved for what was wanted, as we can see in the examples that the 'centre' is found and often is converged too, especially succeeding in shapes that are near cyclic. Additionally, the other mode of the smallest total sum is a useful feature that fits with the project and suffers little of any errors or exceptions. My stakeholder found the project and the implementation of the code extremely fascinating, citing this to be the equivalent of an end of year undergraduate maths project. In setting out to make something mathematically sound, the program is lacking in certain areas, in particular in how certain features are implemented, and features that have been neglected entirely.

Unmet features and improvements:

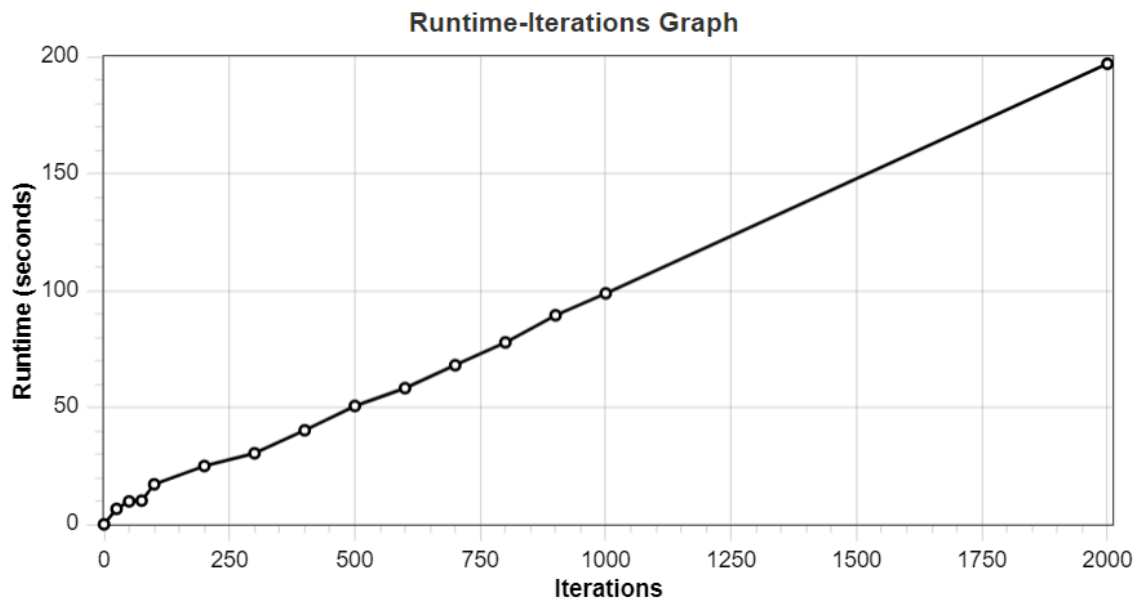
When considering the pre-existing functions that aren't implemented as well as they could have been, the iterative functions are the biggest offenders. As we saw back with example 4 and 7, there are still cases where the program cannot find the point of minimum variance, and there isn't a secure method of finding out which centre method to use. The program cannot react properly when both methods cause a decrease in variance, nor can they react properly when there is an increase in variance in both. The case of them both increasing variance, it is safe enough to say there is no point of minimum variance, however this isn't certain as iteration formulae are somewhat dependent on their input, and we are supposed to start with a point close to the root. Since we cannot graph the function, nor can we use any other methods to make a decent approximation of where point of minimum variance is, we go with the average of latitude and longitude, yet this still often leads to lengthy processes. What is worse though is the case where both methods cause variance to decrease, yet only one of them converges. This is bad because it means there should be some other condition to determine which method to use, but due to the limited time frame I was unable to investigate what that condition should be. The solution I have thought of is through a convergence test, which effectively would remove the accuracy feature and have the program run until a string of 10 decimal places matches with the previous iteration, and variance is decreasing, then if

its found that variance isn't decreasing more, then it could discard one method, and do the other one until it converges. This is dependent on the assumption that if both methods have decreasing variance, at least one will converge. This approach though brings to light the biggest problem in the current version of the program: time taken.

The program does a lot of maths. It performs multiple calculations that result in cross products, summations, matrix multiplications, inverting of matrices, and more; all of which when conducted 100, 200, 500 times cause a massive amount of time taken. Obviously not all problems need 100 iterations, many that are close to a cyclic shape need only 50, but some problems take upwards of 750 iterations. To get a better look at how time scales, I measured the time it took for the program to complete once enter had been pressed for 4 points: $[-10, 4]$, $[21, -5]$, $[1, 4]$, $[3, -12]$, for different numbers of accuracy (I.e. the number of iterations it performs).

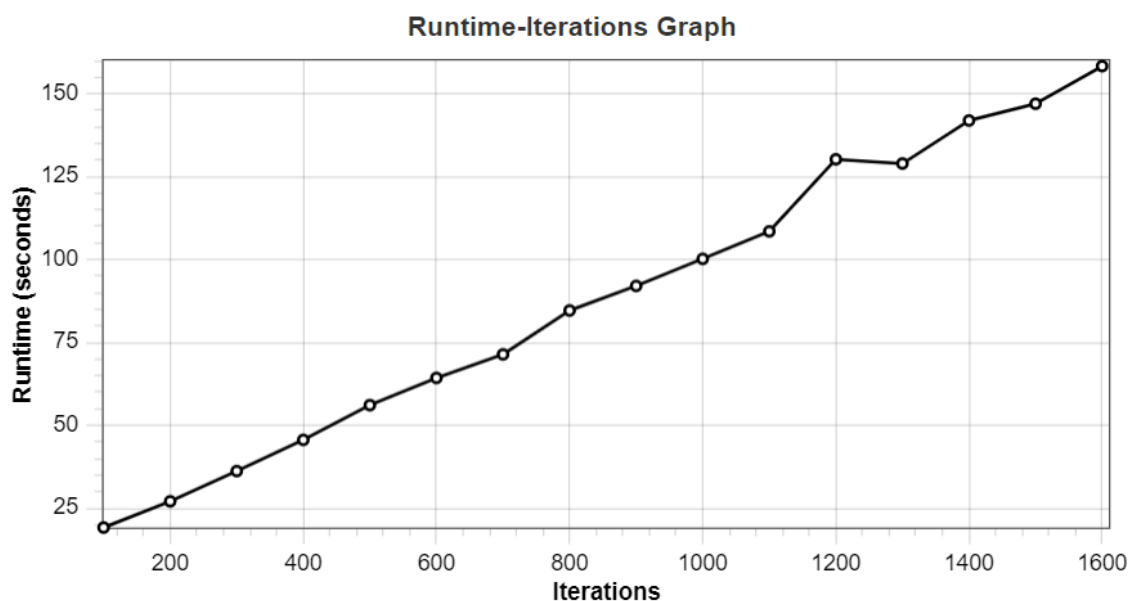


As we can see from the graph above, past 100 iterations the runtime is directly proportional to the number of iterations, to the point I was so confident I let it run for 2000 iterations and unsurprisingly, it gave this graph.



Initially I wanted to conclude that this meant that iterations is directly proportional to runtime, however we don't know when it converges, so when looking at the actual answers we see that it actually converges, at 147 iterations (to 10 decimal places). As the shape of the graph at convergence actually seems to look like its gradient is decreasing, I can't say with certainty from this one case if time taken for convergence is directly proportional, but I can say post convergence runtime is directly proportional, where post convergence, 1 iteration takes 0.1 second.

So, to rigorously test it I chose another 4 set of points that takes much longer to converge, not actually converging until about 1500 iterations: [4, 3], [-2, 1], [5, -2], [19, 27].



This graph shows the result of this difference where it takes 1500 iterations to converge, meaning I can completely conclude that runtime and the number of iterations is directly proportional. Moreover, this shows me that to really improve the program, to have it run

until fully convergent is the way to go, as not only would it mean it would drastically reduce the time of sequences that converge quickly, but also let the program do the number of iterations that are necessary. This could be done with a while loop where until a temporary variable which is set to be the previous iteration, and if it is near exactly equal to the new iteration, we will conclude it is convergent and stop.

Interestingly, when measuring runtime against the number of points (greater than 3), remains roughly constant, averaging out at about 15 seconds (with about 150 iterations), which shows that the number of points does not affect the runtime anywhere near as much as the number of iterations it has to perform.

There are a few quality-of-life areas lacking in the final product as well, in particular regarding the user interface. First and foremost, the zoom feature isn't perfect. Whilst there isn't anything I can do to change it, the zoom feature looks and feels jolty because it is, as it can only zoom with varying integer values, not the spectrum of decimal values between each interval, which causes it to jump unnaturally between certain zoom levels, especially for lower values. This can't be fixed without using a different static image Api, as the URL for static images requires integers. Additionally, the map is lacking as well, with certain points like 90, 0 not appearing on the map, and there is little to no camera control. The camera control in particular is a feature that couldn't be explored, and is one that would have greatly improved the final outcome.

Google Maps greatly inspired my GUI, as can be seen with the effort to create a map you can move around and zoom in and out. However, my map lacks the interactivity that Google Maps has because I was unsure how to associate pixel on the screen to latitude/longitude. In the case I could, then I would make it so the map could be realigned so that it would centre on the point of the mouse when clicked, rather than always staying on the calculated centre point. For this there could be two main solutions. Either to figure out a way to relate the x and y axis pixels to a certain latitude and longitude, and then use that to recentre the static image when clicked, where it would retain its previous zoom level and markers due to the constant URL attribute. Alternatively, another API could have been used for the GUI, one that involves an interactive map. In my search for how to create an image of a map I found several GUI's that had the potential of interactivity, but many were limited by my lack of understanding of them and so I steered clear, but perhaps if I explored them more back then, a better product could have been produced.

This brings me on to a couple other features that were unmet in the final version of the program, and that's because over the course of the project my stakeholder's wants grew more toward the mathematical and abstract side of the project, rather than the real-world

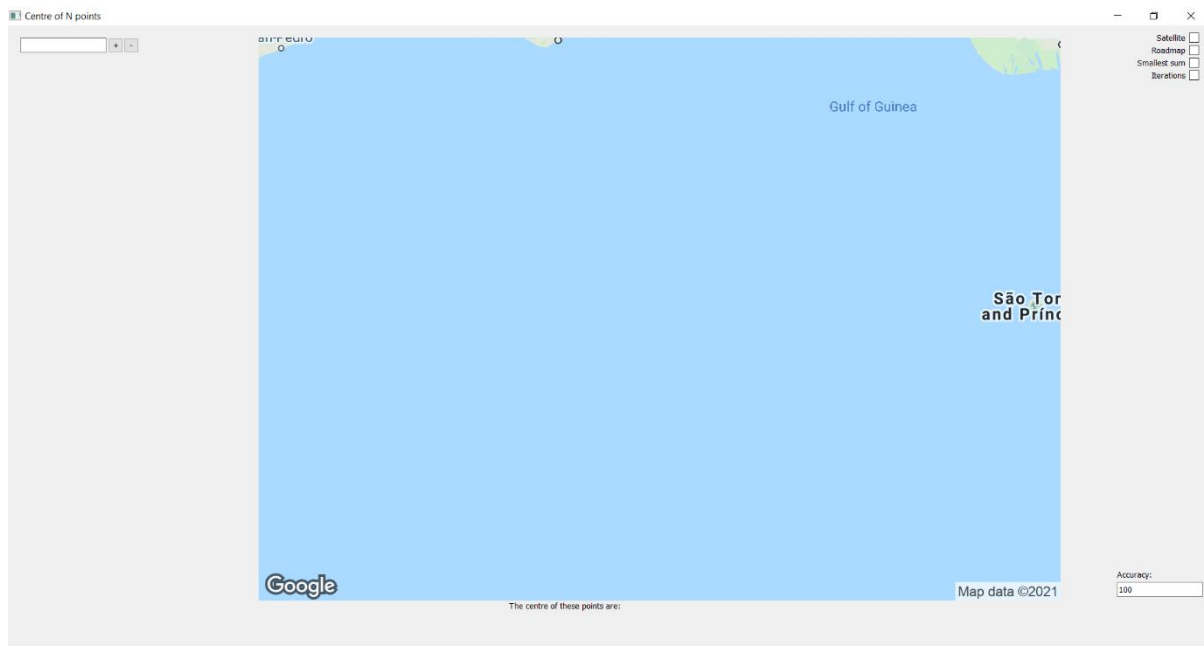
application. However, originally, I intended to add in some extra features that would have made it more usable. These would have included, nearest address finding methods, which would have used the google maps to locate the nearest address that would be reachable, as to have the result not be in the middle of the ocean. Alternatively, it may have directed to the nearest reachable location, and then implement the feature of searching for nearby tourist/gathering destinations, such as parks, cafes, attractions, etc. using websites or other APIs, such as tripadvisor, so the program would be able to recommend locals for meeting points. Furthermore, I considered implementing a version of this that actually resembled Dijkstra's algorithm, finding the point that was the best meeting place between n number of points, using a node graph system. Interestingly this would have taken a similar approach, finding the node that would have the minimum variance in distance from all of the other points, however the problem with this is it would require many heuristics for it to be useful and not take an extortionate amount of time. Unfortunately, all of these interesting and likely useful features weren't implemented because of timing, and the priorities of the stakeholder, and this puts a rather large limitation on the program's usability, limiting its use to everyday people in the real world.

Usability:

Whilst perhaps the day to day uses of the program are somewhat limited, usability has been shown to be quite high, with my stakeholder, and two other testers found the GUI easy to maneuver and figure out. To each tester (my stakeholder, Mr Banks, my Dad, and my friend) I gave them a series of instructions and saw if they were able to complete them without help or aid. The instructions given to them were:

1. Find the centre of the points (34, 12) and (-2, 5)
2. Find the centre between the locations of London, Birmingham and Manchester
3. Find the centre of the 4 points (10, 10), (10, -10), (-10, 10), (-10, -11)
4. Do the same above with an accuracy of 200
5. Find the minimum total distance of London, Birmingham and Manchester
6. See the iterations of instruction 5

These instructions got across the main functions and features of the project, and for the most part they could be figured out quite easily. The second one wasn't done perfectly, as two testers were unable to figure out that Birmingham needed to be specified to be the UK's Birmingham, and that accuracy wasn't actually specified anywhere, so I needed to correct that (even though every tester gathered the number on screen was the accuracy value). Regardless, what can be concluded is that the GUI is intuitive enough for usage, which is likely a result from its similarity to its Google Maps inspiration.



The UI above showcases the strengths of the program's usability, as everything is clearly labelled, with the points on the left-hand side of the screen similar to Google Maps, and all other details surround the image on screen. However, when looking at the UI above, there are a few areas that need improvement. Firstly, the buttons, textboxes and text all need to be much larger, as anyone that has poorer eyesight may strain their eyes in using this program. Especially the bottom text of "The centre of these points are:" as that is so easy to miss it could mean the user could miss the entire point of the program (ironically). Fortunately, this can be fixed by making use of PyQt's simple font change function, it just didn't occur to me in the second iteration to make the interface more user friendly, even one font size larger than the default being enough to make a large difference.

Maintenance:

The program I have produced has also been designed to be able to be maintained in the future, with debugging and improvements to be added. When designing the program, it was split up into various functions that would be easy to implement other features. For example, when doing other improvements such as to implement a feature that would direct the point to the nearest tourist location, it could be written as a function in the Map class and called with an if statement inside the on_returnPressed function. Similarly, with debugging because of the way that the different functions are separated, especially when looking at the iteration functions split into s1, s2, s3... etc. Therefore, features and debugging are fairly easy to implement due to the design of the program, and hence it is a strength of my final product. However, there are some areas of my program that could have been improved upon regarding the maintainability. In particular, the variable names. Whilst the comments explain the purpose of each of those variables and functions, it doesn't change how they were named terms that aren't that descriptive or insightful, and only really are understood if the programmer is familiar with the proof of the

formulae. Additionally, there is the problem this program is dependent on an external API, which requires an API key, which can and does expire. When only one user makes use of the API the number of requests don't exceed the given amount and the billing key for the Google Maps API holds, but if this was to be distributed and used by multiple people on multiple devices, then it would be likely the API key would become invalid simply because it would then start to cost money. Furthermore, to use it would require additional security as currently it just retrieves it from an external file; hence posing risk to my details and API key if it isn't appropriately protected.

Success Criteria:

Criteria	Met?
Display an image of a map pulled from Google Maps API	Yes
Display the location of points on that image	Yes
Zoom in to the image a suitable amount dependent on what is wanted	Yes
To be able to enter any number of points to the program	Yes
Using the number of points to find the point equidistant from each point	Yes
Checking equidistance and outputting the distance to the centre	Yes
Suggesting tourist/meeting locations near that centre point	No
An option to adjust the centre to remove unreachable points	No

Conclusion:

To conclude, this project has had many successes, and many limitations, however it completed what it set out to do: find the centre of n number of points on a map. From starting off from barely understanding the basic mathematics that is needed to be understood to complete this, to eventually getting to the point vector calculus could be performed. At times, the project felt more of a maths project rather than a programming one, but the abilities necessary for computer science appeared throughout, abstracting away details, making use of approximations and heuristics, following an iterative agile methodology, constantly making the goals suit the needs of the stakeholder. The program has its fair share of limitations, as well as its potential improvements that simply didn't get enough focus or priority to be completed by the deadline, but many of those improvements surrounded commercial usage, as opposed to the very mathematical and abstracted facets of this project.

Code Appendix:

2D Centre of n Attempt 1:

```
1. class point:
2.     x = 0
3.     y = 0
4.     i = 0
5.     area = 0
6.     Cx = 0
7.     Cy = 0
8.
9.     points = []
10.
11.     def __init__(self, x, y, i):
12.         self.x = x
13.         self.y = y
14.         self.i = i
15.
16.     def display_coords(self):
17.         print(self.x, self.y)
18.
19.     def list_append(self):
20.         self.points.append([self.x, self.y, self.i])
21.
22.     def signed_area(self):
23.         for i in range(0, n-1):
24.             self.area = self.area + 0.5*((self.points[i][0] * self.points[i+1][1])
- (self.points[i+1][0] * self.points[i][1]))
25.             print(self.area)
26.         return abs(self.area)
27.
28.     def center_x(self):
29.         for i in range(0, n-1):
30.             #print("x: " + str(self.Cx))
31.             self.Cx = self.Cx + ((self.points[i][0] +
self.points[i+1][0])*((self.points[i][0] * self.points[i+1][1]) -
(self.points[i+1][0] * self.points[i][1])))
32.             self.Cx = self.Cx*(1/(6*full_area))
33.         return abs(self.Cx)
34.
35.     def center_y(self):
36.         for i in range(0, n-1):
37.             #print("y: " + str(self.Cy))
38.             self.Cy = self.Cy + ((self.points[i][1] +
self.points[i+1][1])*((self.points[i][0] * self.points[i+1][1]) -
(self.points[i+1][0] * self.points[i][1])))
39.             self.Cy = self.Cy*(1/(6*full_area))
40.         return abs(self.Cy)
41.
42.     def distance_check(self):
43.         for z in range(0, n):
44.             print("From vertex " + str(z) + ": " + str(((self.points[z][0]-
abs(self.Cx))**2 + (self.points[z][1]-abs(self.Cy))**2)**0.5))
45.
46. n = int(input("Enter how many vertices the shape has: "))
47. for i in range(0, n):
48.     input_x = int(input("Enter the x coordinate of the vertex: "))
49.     input_y = int(input("Enter the y coordinate of the vertex: "))
```

```

50.
51.     append_point = point(input_x, input_y, i)
52.     point.list_append(append_point)
53.
54. final_point = point(point.points[0][0], point.points[0][1], point.points[0][2]+n)
55. point.list_append(final_point)
56.
57. full_area = point.signed_area(append_point)
58. center_coords = [point.center_x(append_point), point.center_y(append_point)]
59. print(center_coords)
60. point.distance_check(append_point)

```

2D Centre of n Attempt 2:

```

1. import math
2.
3. v = []
4. n = int(input("Enter how many vertices the shape has: "))
5.
6. def point_distance(p1, p2):
7.     return ((p2[0] - p1[0])**2 + (p2[1] - p1[1])**2)**0.5
8.
9. def input_points(v, n):
10.     for i in range(0, n):
11.         input_x = int(input("Enter the x coordinate of the vertex: "))
12.         input_y = int(input("Enter the y coordinate of the vertex: "))
13.         v.append([input_x, input_y])
14.
15. def signed_area(v, n):
16.     area = 0
17.     for i in range(0, n-1):
18.         area += ((v[i][0]*v[i+1][1]) - (v[i+1][0]*v[i][1]))
19.         print(area)
20.     area += ((v[n-1][0]*v[0][1])-(v[0][0]*v[n-1][1]))
21.     area = abs(area*0.5)
22.     return area
23.
24. def cyclic_area(v, n):
25.     a = []
26.     d = []
27.     w = [point_distance(v[1], v[3]), point_distance(v[0], v[2])]
28.     area = 0
29.
30.     for i in range(0, n-1):
31.         d.append(point_distance(v[i], v[i+1]))
32.     d.append(point_distance(v[n-1], v[0]))
33.     print(d)
34.
35.     for i in range(0, n-1):
36.         x = int(((((-1)**i) + 1)/2))
37.         a.append(math.acos((d[i+1]**2 - d[i]**2 - w[x]**2)/(-2*d[i]*w[x])))
38.     a.append(math.acos((d[0]**2 - d[3]**2 - w[1]**2)/(-2*d[3]*w[1])))
39.
40.     for i in range(0, n):
41.         area += ((d[i]**2)*math.tan(a[i]))
42.     print(area)
43.
44. input_points(v, n)
45. cyclic_area(v, n)
46. print(point_distance(v[0], v[1]))

```

Original Point extraction method:

```
1. import gmaps
2. import requests
3. import json
4. from gmaps import Geocoding
5.
6. with open('API-KEY.txt') as f:
7.     Key = f.readline()
8.     f.close()
9.
10.
11. def extract_lat_long_via_address(address_or_zipcode):
12.     lat, lng = None, None
13.     api_key = Key
14.     base_url = "https://maps.googleapis.com/maps/api/geocode/json"
15.     endpoint = f"{base_url}?address={address_or_zipcode}&key={api_key}"
16.     r = requests.get(endpoint)
17.     json_data = json.loads(r.text)
18.     return json_data
19.
20. def extract_address_via_lat_long(lat, lng):
21.     api_key = Key
22.     base_url = "https://maps.googleapis.com/maps/api/geocode/json"
23.     endpoint = f"{base_url}?latlng={lat}, {lng}&key={api_key}"
24.     r = requests.get(endpoint)
25.     json_data = json.loads(r.text)
26.     return json_data['status']
```

2D Centre of 4:

```
1. import numpy as np
2.
3. def perpendicularBisector(p1, p2):
4.     midpoint = [(p1[0] + p2[0]) / 2, (p1[1] + p2[1]) / 2]
5.     if p2[1] - p1[1] == 0:
6.         gradient = "infinity"
7.         c_value = midpoint[0]
8.     else:
9.         gradient = -(p2[0] - p1[0]) / (p2[1] - p1[1])
10.        c_value = midpoint[1] - gradient*midpoint[0]
11.
12.    return [gradient, c_value]
13.
14. def intersect(l1, l2):
15.    if l1[0] == l2[0]:
16.        return "No intersects"
17.    else:
18.        if l1[0] == "infinity":
19.            x = l1[1]
20.            y = l2[0]*x + l2[1]
21.        elif l2[0] == "infinity":
22.            x = l2[1]
23.            y = l1[0] * x + l1[1]
24.        else:
25.            x = (l2[1] - l1[1]) / (l1[0] - l2[0])
26.            y = l1[0]*x + l1[1]
27.
28.        return [x,y]
29.
30. def centre4(pointlist):
```

```

31.     linelist = []
32.     newpointlist = []
33.     for i in range(3):
34.         linelist.append(perpendicularBisector(pointlist[i], pointlist[i+1]))
35.     linelist.append(perpendicularBisector(pointlist[3], pointlist[0]))
36.
37.     for i in range(3):
38.         newpointlist.append(intersect(linelist[i], linelist[i+1]))
39.     newpointlist.append(intersect(linelist[3], linelist[0]))
40.
41.
42.     roundedlist = np.around(newpointlist, -4)
43.     if all(np.all(elem == roundedlist[0]) for elem in roundedlist) == True:
44.         return newpointlist
45.     else:
46.         print("yes")
47.         return centre4(newpointlist)
48.
49. def distance(p1, p2):
50.     x = p2[0] - p1[0]
51.     y = p2[1] - p1[1]
52.     dist = (x**2 + y**2)**0.5
53.     return dist
54.
55. pointlist = [[-6,0], [10,5], [1,9], [5, 4]]
56. cntr = centre4(pointlist)
57. print(cntr)
58.
59. for i in range(4):
60.     print(distance(cntr[i], pointlist[i]))

```

Greatcircle intersect:

```

1. import math
2. R = 6371
3.
4. def geodetic_to_cartesian(lat, long):
5.     rlat = math.radians(lat)
6.     rlong = math.radians(long)
7.     x = R*math.cos(rlat)*math.cos(rlong)
8.     y = R*math.cos(rlat)*math.sin(rlong)
9.     z = R*math.sin(rlat)
10.    return x, y, z
11.
12. def cartesian_to_geodetic(x, y, z):
13.    rlat = math.asin(z/R)
14.    rlong = math.atan2(y,x)
15.    lat = math.degrees(rlat)
16.    long = math.degrees(rlong)
17.    return lat, long
18.
19. def greatcircle_definition(x1, y1, z1, x2, y2, z2):
20.    a = y1*z1 - z1*y2
21.    b = z1*x2 - x1*z2
22.    c = x1*y2 - y1*x2
23.    return a, b, c
24.
25. def greatcircle_intersect(a1, b1, c1, a2, b2, c2):
26.    h = (a2*c1 - c2*a1)/(b2*a1 - a2*b1)
27.    g = (-b1*h - c1)/(a1)
28.    k = math.sqrt((R**2)/(g**2 + h**2 + 1))

```



```

29.
30.     x = g*k
31.     y = h*k
32.     z = k
33.
34.     return cartesian_to_geodetic(x, y, z)

```

Centre of n Attempt 1:

```

1. def createOmega(self, pointlist, C):
2.     omega = 0
3.     for i in range(0, len(pointlist)):
4.         V = pointlist[i].cartesian
5.         K = np.matrix([[V[1] ** 2 + V[2] ** 2), -1 * V[0] * V[1], -1 * V[0] *
V[2]],
6.                         [-1 * V[0] * V[1], (V[0] ** 2 + V[2] ** 2), -1 * V[1] *
V[2]],
7.                         [-1 * V[0] * V[2], -1 * V[1] * V[2], (V[0] ** 2 + V[1] **
2)]]))
8.         scalar1 = 1 / ((np.linalg.norm(C) ** 2) * (np.linalg.norm(V) ** 2))
9.         scalar2 = np.dot(V, C) / np.linalg.norm(np.cross(V, C))
10.        omega = omega + scalar1 * scalar2 * K
11.
12.    return omega
13.
14.
15. def createDelta(self, pointlist, C):
16.     delta = 0
17.     for i in range(0, len(pointlist)):
18.         V = pointlist[i].cartesian
19.         scalar1 = np.linalg.norm(np.cross(V, C)) / ((np.linalg.norm(C) ** 2) *
(np.linalg.norm(V) ** 2))
20.         delta = delta + scalar1 * V
21.
22.    return delta
23.
24.
25. def centreN(self, pointlist, C):
26.     omega = self.createOmega(pointlist, C)
27.     delta = self.createDelta(pointlist, C)
28.
29.     invomega = np.linalg.inv(omega)
30.     centre = np.dot(invomega, delta).tolist()[0]
31.     t_coeffecient = R / np.linalg.norm(centre)
32.     midpoint = t_coeffecient*centre
33.
34.    return Point.fromCartesian(centre)

```

Centre of n Attempt 2:

```

1. def osum1(self, pointlist, centrepnt):
2.     C = centrepnt.cartesian
3.     sum = 0
4.     for i in range(0, len(pointlist)):
5.         V = pointlist[i].cartesian
6.         K = np.matrix([[V[1] ** 2 + V[2] ** 2), -1 * V[0] * V[1], -1 * V[0] *
* V[2]],
7.                         [-1 * V[0] * V[1], (V[0] ** 2 + V[2] ** 2), -1 * V[1] *
* V[2]],
8.                         [-1 * V[0] * V[2], -1 * V[1] * V[2], (V[0] ** 2 +
V[1] ** 2)]]))

```

```

9.         scalar1 = (1/R) * self.arcdistance(pointlist[i], centrepoint)
10.        scalar2 = 1 / np.dot(V, C)**2
11.        scalar3 = np.dot(V, C) / np.linalg.norm(np.cross(V, C))
12.        vector1 = scalar3 * np.dot(K, C)
13.        vector2 = np.linalg.norm(np.cross(V, C)) * V
14.        vector3 = (vector1 - vector2)
15.        total = scalar1 * scalar2 * vector3
16.        sum = sum + total
17.    sum = sum * len(pointlist)
18.
19.    return sum
20.
21.    def osum2(self, pointlist, centrepoint):
22.        C = centrepoint.cartesian
23.        sum = 0
24.        for i in range(0, len(pointlist)):
25.            V = pointlist[i].cartesian
26.            total = (1/R)*self.arcdistance(pointlist[i], centrepoint)
27.            sum = sum + total
28.
29.        return sum
30.
31.    def osum3(self, pointlist, centrepoint):
32.        C = centrepoint.cartesian
33.        sum = 0
34.        for i in range(0, len(pointlist)):
35.            V = pointlist[i].cartesian
36.            scalar1 = np.linalg.norm(np.cross(V, C)) / ((np.linalg.norm(V) ** 2)
37. * (np.linalg.norm(C) ** 2))
38.            total = scalar1 * V
39.            sum = sum + total
40.
41.        return sum
42.
43.    def createOmega(self, pointlist, centrepoint):
44.        C = centrepoint.cartesian
45.        omega = 0
46.        for i in range(0, len(pointlist)):
47.            V = pointlist[i].cartesian
48.            K = np.matrix([[V[1] ** 2 + V[2] ** 2), -1 * V[0] * V[1], -1 * V[0]
49. * V[2]],
50. * V[2]],
51. * V[2]],
52. * V[2]],
53. * V[2]],
54. * V[2]],
55. * V[2]],
56. * V[2]],
57. * V[2]],
58. * V[2]],
59. * V[2]],
60. * V[2]],
61. * V[2]],
62. * V[2]],
63. * V[2]],
64. * V[2]],
65. * V[2]])
66.            scalar1 = 1 / ((np.linalg.norm(C) ** 2) * (np.linalg.norm(V) ** 2))
67.            scalar2 = np.dot(V, C) / np.linalg.norm(np.cross(V, C))
68.            total = scalar1 * scalar2 * K
69.            omega = omega + total
70.
71.        return omega
72.
73.    def oldcentreN(self, pointlist, C):
74.        sum1 = self.sum1(pointlist, C)
75.        sum2 = self.sum2(pointlist, C)
76.        sum3 = self.sum3(pointlist, C)
77.        omega = self.createOmega(pointlist, C)
78.
79.        inverse = np.linalg.inv(omega)
80.        vector1 = ((1 / sum2) * sum1).tolist()[0] + sum3
81.        centre = np.dot(inverse, vector1)

```

```

66.
67.         return centre

```

Centre of n Attempt 3:

```

1. def s1(self, V, C):
2.     s1 = 1 / ((np.linalg.norm(V.cartesian) ** 2) * (np.linalg.norm(C.cartesian)
   ** 2))
3.     return s1
4.
5.     def s2(self, V, C):
6.         s2 = np.dot(V.cartesian, C.cartesian) /
   np.linalg.norm(np.cross(V.cartesian, C.cartesian))
7.         return s2
8.
9.     def s3(self, V, C):
10.        s3 = np.linalg.norm(np.cross(V.cartesian, C.cartesian))
11.        return s3
12.
13.    def K(self, point):
14.        X = point.cartesian
15.        K = np.matrix([[X[1] ** 2 + X[2] ** 2), -1 * X[0] * X[1], -1 * X[0] *
   X[2]],
16.                        [-1 * X[0] * X[1], (X[0] ** 2 + X[2] ** 2), -1 * X[1] *
   X[2]],
17.                        [-1 * X[0] * X[2], -1 * X[1] * X[2], (X[0] ** 2 + X[1] **
   2)]]])
18.        return K
19.
20.    def l(self, V, C):
21.        cross = np.cross(V.cartesian, C.cartesian)
22.        dot = np.dot(V.cartesian, C.cartesian)
23.        temp = np.linalg.norm(cross) / dot
24.
25.        l = math.atan(temp)
26.        return l
27.
28.    def sum1(self, pointlist, C):
29.        sum = 0
30.        for i in range(0, len(pointlist)):
31.            V = pointlist[i]
32.            temp = self.l(V, C) * self.s1(V, C) * self.s3(V, C) * V.cartesian
33.            sum = sum + temp
34.
35.        sum = sum * len(pointlist)
36.        return sum
37.
38.    def sum2(self, pointlist, C):
39.        sum1 = 0
40.        sum2 = 0
41.        for i in range(0, len(pointlist)):
42.            V = pointlist[i]
43.            temp1 = self.l(V, C)
44.            sum1 = sum1 + temp1
45.
46.        for i in range(0, len(pointlist)):
47.            V = pointlist[i]
48.            temp2 = self.s1(V, C) * self.s3(V, C) * V.cartesian
49.            sum2 = sum2 + temp2
50.
51.        sum = sum1 * sum2

```

```

52.         return sum
53.
54.     def sum3(self, pointlist, C):
55.         sum = 0
56.         for i in range(0, len(pointlist)):
57.             V = pointlist[i]
58.             K = self.K(V)
59.             temp = self.l(V, C) * self.s1(V, C) * self.s2(V, C) * K
60.             sum = sum + temp
61.
62.         sum = sum * len(pointlist)
63.         return sum
64.
65.     def sum4(self, pointlist, C):
66.         sum1 = 0
67.         sum2 = 0
68.         for i in range(0, len(pointlist)):
69.             V = pointlist[i]
70.             K = self.K(V)
71.             temp1 = self.l(V, C)
72.             temp2 = self.s1(V, C) * self.s2(V, C) * K
73.             sum1 = sum1 + temp1
74.             sum2 = sum2 + temp2
75.
76.         sum = sum1 * sum2
77.         return sum
78.
79.     def centreN(self, pointlist, C):
80.         #print("s1", self.sum1(pointlist, C))
81.         #print("s2", self.sum2(pointlist, C))
82.         print("s3", self.sum3(pointlist, C))
83.         print("s4", self.sum4(pointlist, C))
84.         vector = self.sum1(pointlist, C) - self.sum2(pointlist, C)
85.         omega = self.sum3(pointlist, C) - self.sum4(pointlist, C)
86.         inverse = np.linalg.inv(omega)
87.
88.         centre = np.dot(inverse, vector)
89.         return Point.fromCartesian(centre.tolist())

```

Final Code:

```

1. import math #for square roots and trigonometric functions
2. import numpy as np #for matrix and vector calculations
3. import requests #for accessing google api data
4. import json #for accessing the json data from the api
5. import urllib #for accessing the image from the google maps static images
6. import sys
7. import re #for the use of regex to regulate inputs
8. from PyQt5.QtWidgets import * #for the GUI
9. from PyQt5.QtGui import * #for the GUI
10. from PyQt5.Qt import QWheelEvent #for the zoom function
11. from PyQt5 import QtCore #for PyQt5
12. from PyQt5.QtCore import *
13. from gmaps import Geocoding #for the geocoding function
14. import time #for debugging time taken
15.
16. with open('API-KEY.txt') as f:
17.     Key = f.readline()
18.     f.close()
19. #This just keeps my API Key in a seperate folder so it can't be seen when I upload
    this for the project

```

```

20.
21. api = Geocoding(api_key = Key)
22. R = 6371
23. base_url = "https://maps.googleapis.com/maps/api/geocode/json"
24. #Main constants of the program, the api, the radius of the Earth, and a base
    endpoint url
25.
26. class Point:
27.     cartesian = np.array([]) #[x, y, z]
28.     polar = [] #[latitude, longitude]
29.     postcode = str() #address string
30.     #the three ways location can be described
31.
32.     def __init__(self, cartesian, polar, postcode):
33.         self.cartesian = cartesian
34.         self.polar = polar
35.         self.postcode = postcode
36.
37.     @classmethod
38.     def fromPolar(cls, polar):
39.         lat = math.radians(polar[0])
40.         lng = math.radians(polar[1])
41.
42.         x = R * math.cos(lat)*math.cos(lng)
43.         y = R * math.cos(lat)*math.sin(lng)
44.         z = R * math.sin(lat)
45.         #finds the x, y and z coordinates from polar coordinates
46.
47.         endpoint = f"{base_url}?latlng={polar[0]}, {polar[1]}&key={Key}"
48.         r = requests.get(endpoint)
49.         json_data = json.loads(r.text)
50.         if json_data['status'] == "OK":
51.             postcode =
                json_data['results'][0]['address_components'][0]['long_name']
52.             #retrieves the json data and ensures an address exists, if it defines
                this variable
53.             else:
54.                 postcode = "None"
55.                 #the status isn't considered OK, meaning there is no address available
                to retrieve
56.
57.
58.         return cls(np.array([x, y, z]), polar, postcode)
59.
60.     @classmethod
61.     def fromCartesian(cls, cartesian):
62.         lat = math.degrees(math.asin(cartesian[2] / R))
63.         lng = math.degrees(math.atan2(cartesian[1], cartesian[0]))
64.         #finds the lat and lng from cartesian coordinates
65.
66.         endpoint = f"{base_url}?latlng={lat}, {lng}&key={Key}"
67.         r = requests.get(endpoint)
68.         json_data = json.loads(r.text)
69.         if json_data['status'] == "OK":
70.             postcode =
                json_data['results'][0]['address_components'][0]['long_name']
71.             #retrieves the json data and ensures an address exists, if it defines
                this variable
72.             else:
73.                 postcode = "None"

```

```

74.         #the status isn't considered OK, meaning there is no address available
to retrieve
75.
76.         return cls(np.array(cartesian), [lat, lng], postcode)
77.
78.     @classmethod
79.     def fromAddress(cls, postcode):
80.
81.         endpoint = f"{base_url}?address={postcode}&key={Key}"
82.         r = requests.get(endpoint)
83.         json_data = json.loads(r.text)
84.         #gathers json data for an input
85.
86.         if json_data['status'] == "OK":
87.             lat = json_data['results'][0]['geometry']['location']['lat']
88.             lng = json_data['results'][0]['geometry']['location']['lng']
89.             polar = [lat, lng]
90.
91.             x = R * math.cos(math.radians(lat)) * math.cos(math.radians(lng))
92.             y = R * math.cos(math.radians(lat)) * math.sin(math.radians(lng))
93.             z = R * math.sin(math.radians(lat))
94.             #finds the cartesian coordinates and the polar coordinates
95.
96.             return cls(np.array([x, y, z]), polar, postcode)
97.         else:
98.             return False
99.         #returns false to signify there was some error occurred from the address
input
100.
101.     class Map:
102.         zoom = int() #refers to the zoom of the image
103.         maptype = str() #refers to the map type of the image: satellite,
roadmap, hybrid, terrain
104.         pointlist = [] #a list of the point objects on the map
105.         url = str() #the url to retrieve an image from
106.         iterationlist = [] #list of markers to iterate
107.
108.         def __init__(self, zoom, maptype, centre, pointlist):
109.             self.zoom = zoom
110.             self.maptype = maptype
111.             self.centre = centre
112.             self.pointlist = pointlist
113.
114.         def centre2(self, P1, P2):
115.             lat1 = math.radians(P1.polar[0])
116.             lng1 = math.radians(P1.polar[1])
117.             lat2 = math.radians(P2.polar[0])
118.             lng2 = math.radians(P2.polar[1])
119.             #defines latitude and longitude separately so it doesn't need to
find it in radians very time
120.
121.             Bx = math.cos(lat2) * math.cos(lng2 - lng1)
122.             By = math.cos(lat2) * math.sin(lng2 - lng1)
123.
124.             midlat = math.atan2(math.sin(lat1) + math.sin(lat2),
math.sqrt((math.cos(lat1) + Bx) ** 2 + By ** 2))
125.             midlng = lng1 + math.atan2(By, math.cos(lat1) + Bx)
126.             #performs the calculations necessary to find the middle latitude and
middle longitude
127.

```

```

128.         midpoint = Point.fromPolar([math.degrees(midlat),
    math.degrees(midlng)])
129.         #returns the midpoint which is a point object, as to keep the
    standard of everything being a point whenever possible
130.
131.         return midpoint
132.
133.     def centre3(self):
134.         P1 = self.pointlist[0]
135.         P2 = self.pointlist[1]
136.         P3 = self.pointlist[2]
137.         #defines the points as shorter to write objects
138.
139.         perpendicular1 = self.createPerpendicular(P1, P2)
140.         perpendicular2 = self.createPerpendicular(P1, P3)
141.         #finds the perpendicular planes between P1 and P2, and P1 and P3
142.
143.         y_coefficient = -((perpendicular1[0]*perpendicular2[2] -
    perpendicular1[2]*perpendicular2[0]) / (perpendicular1[1]*perpendicular2[2] -
    perpendicular1[2]*perpendicular2[1]))
144.         z_coeffecient = -((perpendicular1[0]*perpendicular2[1] -
    perpendicular1[1]*perpendicular2[0]) / (perpendicular1[2]*perpendicular2[1] -
    perpendicular1[1]*perpendicular2[2]))
145.         #parameterizes the line equation and rearranges them to solve them
    with  $x^2 + y^2 + z^2 = R$ 
146.
147.         t_coeffecient = 1 + y_coefficient**2 + z_coeffecient**2
148.         t = math.sqrt((R**2)*(1/t_coeffecient))
149.         midpoint = Point.fromCartesian([t, t*y_coefficient,
    t*z_coeffecient])
150.         # finds the parameter that brings the coordinates to the sphere to
    return them as the standard output
151.
152.         return midpoint
153.
154.     def createPerpendicular(self, point1, point2):
155.         plane = np.cross(point1.cartesian, point2.cartesian)
156.         #plane where point1, point2 and the origin are coplanar
157.         centre = self.centre2(point1, point2)
158.         #the midpoint between point 1 and point2
159.
160.         perpendicular = np.cross(plane, centre.cartesian)
161.         #finds a vector perpendicular to the plane's normal vector, at the
    midpoint
162.         return perpendicular
163.
164.     def coplanar(self):
165.         P1 = self.pointlist[0]
166.         P2 = self.pointlist[1]
167.         equal = 0
168.         perpendicular = np.cross(P1.cartesian, P2.cartesian) #the normal
    vector
169.         checkval = np.dot(P1.cartesian, perpendicular) #constant value to
    define the plane
170.         for V in self.pointlist:
171.             if np.dot(V.cartesian, perpendicular) != checkval: #r.n = d
172.                 equal = equal + 1
173.
174.         if equal != 0:
175.             return False #not coplanar
176.         else:

```

```

177.             return True #coplanar
178.
179.         def arcdistance(self, point1, point2):
180.             cross = np.cross(point1.cartesian, point2.cartesian)
181.             dot = np.dot(point1.cartesian, point2.cartesian)
182.             temp = np.linalg.norm(cross) / dot
183.             sigma = math.atan(temp)
184.             #formula of sigma = atan(|VxC|/V.C) using the numpy library with
linalg which I still don't know what it stands for
185.
186.             if sigma == 0 and (point1.cartesian[0] != point2.cartesian[0] or
point1.cartesian[1] != point2.cartesian[1] or point1.cartesian[2] !=
point2.cartesian[2]):
187.                 distance = R * np.pi
188.                 #in the case where point1 is parallel to point2 but point1 !=
point2, as specified by comparing cartesian coords
189.             else:
190.                 distance = R * sigma
191.                 #in every other case, even including when dot = 0, as atan(inf)
= pi/2
192.
193.             return distance
194.
195.         def varianceCheck(self, C):
196.             mean = 0
197.             variance = 0
198.             for V in self.pointlist:
199.                 mean = mean + self.arcdistance(C, V)
200.             mean = mean/len(self.pointlist)
201.             for V in self.pointlist:
202.                 variance = variance + (self.arcdistance(C, V) - mean)**2
203.             variance = variance/len(self.pointlist)
204.             #using the standard definition of variance = sum of data - mean
squared divided by number of data points squared
205.
206.             return variance
207.
208.         def distanceCheck(self, C):
209.             for V in self.pointlist:
210.                 print("Distance: " + str(self.arcdistance(C, V)))
211.             #an easier way of seeing all of the distances to a point from a map
object's pointlist
212.
213.         def summationCheck(self, C):
214.             total = 0
215.             for V in self.pointlist:
216.                 total = total + abs(self.arcdistance(C, V))
217.             #takes the absolute of arcdistance as to avoid negative distances
when finding the total
218.             return total
219.
220.         def s1(self, V, C): #Scalar
221.             s1 = 1 / ((np.linalg.norm(V.cartesian) ** 2) *
(np.linalg.norm(C.cartesian) ** 2))
222.             #s1=1/(|V||C|)^2
223.             #its own function as its reused so much
224.             return s1
225.
226.         def s2(self, V, C): #Scalar
227.             s2 = np.dot(V.cartesian, C.cartesian) /
np.linalg.norm(np.cross(V.cartesian, C.cartesian))

```



```

228.         #s2=V.C/|VxC|
229.         #its own function as its reused so much
230.         return s2
231.
232.     def s3(self, V, C): #Scalar
233.         s3 = np.linalg.norm(np.cross(V.cartesian, C.cartesian))
234.         #s3=|VxC|
235.         #its own function as its reused so much
236.         return s3
237.
238.     def l(self, V, C): #Scalar
239.         cross = np.cross(V.cartesian, C.cartesian)
240.         dot = np.dot(V.cartesian, C.cartesian)
241.         temp = np.linalg.norm(cross) / dot
242.
243.         l = math.atan(temp)
244.         return l
245.
246.     def K(self, V): #Matrix
247.         X = V.cartesian
248.         K = np.array([(X[1] ** 2 + X[2] ** 2), -1 * X[0] * X[1], -1 * X[0]
249.             * X[2]],
250.             [ -1 * X[0] * X[1], (X[0] ** 2 + X[2] ** 2), -1 * X[1]
251.             * X[2]],
252.             [ -1 * X[0] * X[2], -1 * X[1] * X[2], (X[0] ** 2 +
253.             X[1] ** 2)])
254.         #uses a numpy 3x3 matrix as derived in formula
255.         return K
256.
257.     #all of these functions are representative of how I made the problem I
258.     #was solving simpler
259.     #when performing the proof of the derivative of the variance of
260.     #arclength I represented a series of scalars and matrices in a more concise form so
261.     #I could solve for C
262.     #as there was room for error for scribing the formulae to computational
263.     #form
264.
265.     def createOmega1(self, C): #Matrix
266.         omega = np.array([[0, 0, 0],
267.             [0, 0, 0],
268.             [0, 0, 0]])
269.         for V in self.pointlist:
270.             l = self.l(V, C)
271.             s1 = self.s1(V, C)
272.             s2 = self.s2(V, C)
273.             K = self.K(V)
274.             omega = omega + l * s1 * s2 * K
275.         #summation of matrices with the previously established
276.         #functions
277.
278.         return omega
279.
280.     def createOmega2(self, C): #Matrix
281.         omega = np.array([[0, 0, 0],
282.             [0, 0, 0],
283.             [0, 0, 0]])
284.         n = len(self.pointlist)
285.         lsum = 0
286.         for V in self.pointlist:
287.             l = self.l(V, C)
288.             lsum = lsum + l

```

```

281.             #summation of scalars of the arclength to be multiplied at the
end
282.
283.         for V in self.pointlist:
284.             s1 = self.s1(V, C)
285.             s2 = self.s2(V, C)
286.             K = self.K(V)
287.             omega = omega + s1 * s2 * K
288.             #summation of matrices with the previously established
functions
289.
290.         return (lsum / n) * omega
291.
292.     def createDelta1(self, C): #Vector
293.         delta = np.array([0, 0, 0])
294.         for V in self.pointlist:
295.             l = self.l(V, C)
296.             s1 = self.s1(V, C)
297.             s3 = self.s3(V, C)
298.             delta = delta + l * s1 * s3 * V.cartesian
299.             #summation of vectors with the previously established functions
300.
301.         return delta
302.
303.     def createDelta2(self, C): #Vector
304.         delta = np.array([0, 0, 0])
305.         n = len(self.pointlist)
306.         lsum = 0
307.         for V in self.pointlist:
308.             l = self.l(V, C)
309.             lsum = lsum + l
310.             #summation of scalars of the arclength to be multiplied at the
end
311.
312.         for V in self.pointlist:
313.             s1 = self.s1(V, C)
314.             s3 = self.s3(V, C)
315.             delta = delta + s1 * s3 * V.cartesian
316.             #summation of vectors with the previously established functions
317.
318.         return (lsum / n) * delta
319.
320.     def centreN1(self, C):
321.         omega1 = self.createOmega1(C)
322.         omega2 = self.createOmega2(C)
323.         delta1 = self.createDelta1(C)
324.         delta2 = self.createDelta2(C)
325.         #once again sets up key variables as to avoid confusion
326.
327.         invomega = np.linalg.inv(omega1)
328.         temp = np.matmul(omega2, C.cartesian) - delta2 + delta1
329.         M = np.matmul(invomega, temp)
330.         #seperates the C value and uses the fact  $M \cdot M^{-1} = I$ , to find a
vector
331.
332.         t_coeffecient = R / np.linalg.norm(M)
333.         C = t_coeffecient * M
334.         #returns the vector to the surface of the sphere
335.
336.         return Point.fromCartesian(C)
337.
338.     def centreN2(self, C):

```

```

338.         omega1 = self.createOmega1(C)
339.         omega2 = self.createOmega2(C)
340.         delta1 = self.createDelta1(C)
341.         delta2 = self.createDelta2(C)
342.         #once again sets up key variables as to avoid confusion
343.
344.         invomega = np.linalg.inv(omega2)
345.         temp = np.matmul(omega1, C.cartesian) - delta1 + delta2
346.         M = np.matmul(invomega, temp)
347.         #seperates the C value and uses the fact  $M \cdot M^{-1} = I$ , to find a
vector
348.
349.         t_coeffecient = R / np.linalg.norm(M)
350.         C = t_coeffecient * M
351.         #returns the vector to the surface of the sphere
352.
353.         return Point.fromCartesian(C)
354.
355.     def minimumDistanceTravelled(self, C):
356.         omega = np.array([[0, 0, 0],
357.                             [0, 0, 0],
358.                             [0, 0, 0]])
359.         delta = np.array([0, 0, 0])
360.         for V in self.pointlist:
361.             s1 = self.s1(V, C)
362.             s2 = self.s2(V, C)
363.             K = self.K(V)
364.             s3 = self.s3(V, C)
365.
366.             omega = omega + s1 * s2 * K #Matrix
367.             delta = delta + s3 * V.cartesian #Vector
368.
369.             invomega = np.linalg.inv(omega)
370.             M = np.matmul(invomega, delta)
371.             #similar seperation process just simplified to two terms of delta
and omega
372.
373.             t_coeffecient = R / np.linalg.norm(M)
374.             C = t_coeffecient * M
375.             #returns the vector to the surface of the sphere
376.
377.             return Point.fromCartesian(C)
378.
379.     def polarAverage(self):
380.         lataverage = 0
381.         lngaverage = 0
382.         for V in self.pointlist:
383.             lataverage = lataverage + V.polar[0]
384.             lngaverage = lngaverage + V.polar[1]
385.
386.         #uses linear method of averaging the points to return a polar array
387.
388.         return [lataverage/len(self.pointlist),
lngaverage/len(self.pointlist)]
389.
390.     def centreMethod(self):
391.         original = 0
392.         temp1 = 0
393.         temp2 = 0
394.
395.         M1 = []

```

```

396.         M2 = []
397.         #instantiates the arrays and variables to store iterations and tests
398.
399.         C = Point.fromPolar(self.polarAverage()) #starting approximation
400.         original = self.varianceCheck(C)
401.         for i in range(5):
402.             C = self.centreN1(C)
403.             temp1 = self.varianceCheck(C)
404.
405.         C = Point.fromPolar(self.polarAverage())
406.         for i in range(5):
407.             C = self.centreN2(C)
408.             temp2 = self.varianceCheck(C)
409.         #for both methods it sees if it decreases or inceases the variance
         as it does in an iterative formula tending toward the maximum or minimum
410.
411.         C = Point.fromPolar(self.polarAverage())
412.
413.         if temp1 <= original and temp2 >= original: #case where method 1
         variance is decreasing and method 2 is increasing
414.             return 1
415.         elif temp2 <= original and temp1 >= original: #case where method 2
         variance is decreasing and method 1 is increasing
416.             return 2
417.         elif temp1 <= original and temp2 <= original: #case where variance
         is decreasing for both methods
418.             print("Unknown")
419.             return 1
420.         else: #variance is increasing for both methods
421.             print("Non existent point")
422.
423.     def returnImage(self):
424.         if len(self.pointlist) == 0: #no markers
425.             self.url =
         f"http://maps.googleapis.com/maps/api/staticmap?center={str(self.centre.polar[0])},
         {str(self.centre.polar[1])}&scale=2&size=800x450&key={Key}&zoom={str(self.zoom)}&ma
         ptype={self.maptypes}&sensor=false"
426.         elif len(self.pointlist) > 3 or len(self.iterationlist) > 0: #if
         there needs to be iterations added to the final image or not
427.             self.url =
         f"http://maps.googleapis.com/maps/api/staticmap?center={str(self.centre.polar[0])},
         {str(self.centre.polar[1])}&scale=2&size=800x450&key={Key}&zoom={str(self.zoom)}&ma
         ptype={self.maptypes}&markers=size:large%color:red%7Clabel:M%7C{str(self.centre.pola
         r[0])},{str(self.centre.polar[1])}&sensor=false"
428.             for I in self.iterationlist:
429.                 self.url = self.url +
         f"&markers=size:tiny%7Ccolor:green%7C{str(I.polar[0])},{str(I.polar[1])}" #why url
         is an attribute as it can constantly be updated and called
430.             else:
431.                 self.url =
         f"http://maps.googleapis.com/maps/api/staticmap?center={str(self.centre.polar[0])},
         {str(self.centre.polar[1])}&scale=2&size=800x450&key={Key}&zoom={str(self.zoom)}&ma
         ptype={self.maptypes}&markers=size:mid%color:red%7Clabel:M%7C{str(self.centre.polar[
         0])},{str(self.centre.polar[1])}&sensor=false"
432.
433.         for V in self.pointlist:
434.             self.url = self.url +
         f"&markers=color:blue%7Clabel:S%7C{str(V.polar[0])},{str(V.polar[1])}"
435.
436.         urllib.request.urlretrieve(self.url, "map1.jpg") #uses urllib to
         retrieve the image under the name map.jpg

```

```

437.
438.     class App(QWidget):
439.
440.         def __init__(self):
441.             super().__init__()
442.             self.title = 'Centre of N points'
443.             self.left = 0
444.             self.top = 0
445.             self.width = 1920
446.             self.height = 1080 #dimensions of most normal computer screens
447.             self.map = ""
448.             self.n = 0
449.             self.accuracy = 100 #default accuracy the program uses
450.             self.textboxarray = []
451.             self.initUI()
452.
453.         def initUI(self):
454.             self.setWindowTitle(self.title)
455.             self.setGeometry(self.left, self.top, self.width, self.height)
456.
457.             self.map = Map(6, "terrain", Point.fromPolar([0,0]), [])
458.             #the instance of the map object the program uses
459.
460.             self.map.returnImage()
461.             self.label = QLabel(self)
462.             pixmap = QPixmap('map1.jpg')
463.             self.label.setPixmap(pixmap)
464.             self.label.move(400, 20)
465.             #displays the initial image
466.
467.             self.centreText = QLabel("The centre of these points are:
", self)
468.             self.centreText.move(800, 920)
469.             #simple display of text beneath the image
470.
471.             self.satellite = QCheckBox('Satellite', self)
472.             self.satellite.move(1828, 10)
473.             self.satellite.stateChanged.connect(self.state_changed)
474.             self.satellite.setLayoutDirection(Qt.RightToLeft)
475.             self.roadmap = QCheckBox('Roadmap', self)
476.             self.roadmap.move(1820, 30)
477.             self.roadmap.stateChanged.connect(self.state_changed)
478.             self.roadmap.setLayoutDirection(Qt.RightToLeft)
479.             #sets up the two checkboxes of satellite imagery and roadmap imagery
480.
481.             self.iteration = QCheckBox('Iterations', self)
482.             self.iteration.move(1820, 70)
483.             self.iteration.setLayoutDirection(Qt.RightToLeft)
484.             self.minimization = QCheckBox('Smallest sum', self)
485.             self.minimization.move(1797, 50)
486.             self.minimization.setLayoutDirection(Qt.RightToLeft)
487.             #sets up the two checkboxes of iteration mode and the smallest sum
mode
488.
489.             self.widget = QWidget(self)
490.             self.addb = QPushButton(self.widget)
491.             self.addb.setText("+")
492.             self.addb.setGeometry(160, 20, 24, 24)
493.             self.addb.clicked.connect(self.addbox)
494.             self.subb = QPushButton(self.widget)
495.             self.subb.setText("-")

```

```

496.         self.subb.setGeometry(185, 20, 24, 24)
497.         self.subb.clicked.connect(self.removebox)
498.         #sets up the two buttons to add and remove textboxes
499.
500.         self.accuracytext = QLabel("Accuracy:", self)
501.         self.accuracytext.move(1770, 870)
502.
503.         self.accuracybox = QLineEdit("100", self)
504.         self.accuracybox.move(1770, 890)
505.         self.accuracybox.returnPressed.connect(self.on_returnPressed)
506.         self.accuracybox.show()
507.         #sets up the textbox to input the new accuracy
508.
509.         self.addbox()
510.         #adds in the first textbox
511.
512.         self.show()
513.
514.     def addbox(self):
515.         self.textbox = QLineEdit(self)
516.         self.textbox.move(20, 50 + (self.n - 1)*30) #adds in text boxes
always 30 pixels apart using an nth term
517.         self.textbox.returnPressed.connect(self.on_returnPressed)
518.         self.textbox.show()
519.         self.textboxarray.append(self.textbox)
520.         self.n = self.n + 1 #the number of textboxes
521.
522.     def removebox(self):
523.         if self.n > 1: #checking there is always at least one textbox
524.             self.textboxarray[self.n - 1].deleteLater() #deletes a widget
525.             del self.textboxarray[self.n - 1]
526.             self.n = self.n - 1
527.             self.show()
528.
529.     def newImgLoad(self):
530.         self.map.returnImage()
531.         pixmap = QPixmap('map1.jpg')
532.         self.label.setPixmap(pixmap)
533.         self.label.update()
534.         #updates the current image with the new image retrieved from
returnImage()
535.
536.     def iterations(self, method, detail, C):
537.         if self.iteration.isChecked():
538.             self.map.iterationlist.append(C)
539.             #this adds in the initial starting point marker
540.             for i in range(self.accuracy):
541.                 #print(C.polar)
542.                 #print(detail(C))
543.                 C = method(C)
544.                 #this does the repeated method specified in the function,
repeating accuracy number of times
545.             if self.iteration.isChecked():
546.                 if i < 10:
547.                     self.map.iterationlist.append(C)
548.                 else:
549.                     if (i / 10).is_integer():
550.                         self.map.iterationlist.append(C)
551.                     #this takes an iteration marker every 10 iterations so that
it doesn't take as much time to process the image

```

```

552.                #it will show the first 10 iterations as that is usually
                    when the most change happens
553.                return C
554.
555.                def on_returnPressed(self):
556.                    start = time.time()
557.                    self.map.iterationlist.clear()
558.                    self.map.pointlist.clear()
559.                    #empties the pointlist and iterationlist from any previous sessions
560.                    polarCheck = re.compile("^[-+]?([1-8]?\\d(\\.\\d+)?)|90(\\.0+)?),\\s*[-+]?
                    (180(\\.0+)?|((1[0-7]\\d)|([1-9]?\\d))(\\.\\d+)?)$") #the regex for polar coordinates
561.                    if self.accuracybox.text() != "": #only changes accuracy if the
                        textbox is altered
562.                        self.accuracy = int(self.accuracybox.text())
563.
564.                    for box in self.textboxarray:
565.                        text = box.text()
566.                        if text != "": #only considers it a point to add if the textbox
                            isnt empty
567.                            if polarCheck.match(text) != None:
568.                                polar = text.split(", ") #splits it up into the polar
                                    form
569.                                P = Point.fromPolar([int(polar[0]), int(polar[1])])
                                    #makes the point from polar coords
570.                                self.map.pointlist.append(P)
571.                                elif Point.fromAddress(text) != False: #if a valid address
                                    is inputted it will not return false
572.                                self.map.pointlist.append(Point.fromAddress(text))
                                    #makes the point from an address
573.                                else:
574.                                    print("Formatting error")
575.
576.                                if self.minimization.isChecked(): #if it is selected to do the
                                    total minimum distance formula
577.                                    self.map.centre =
                                        self.iterations(self.map.minimumDistanceTravelled, self.map.summationCheck,
                                        Point.fromPolar(self.map.polarAverage()))
578.
579.                                elif not self.minimization.isChecked():
580.                                    if len(self.map.pointlist) >= 3:
581.                                        if self.map.coplanar() == False: #the points don't lie on
                                            the same great circle
582.
583.                                            if len(self.map.pointlist) == 3: #makes sure points are
                                                not coplanar
584.                                                self.map.centre = self.map.centre3()
585.                                                print(self.map.varianceCheck(self.map.centre))
586.                                                print(self.map.summationCheck(self.map.centre))
587.
588.                                            else:
589.                                                method = self.map.centreMethod()
590.                                                if method == 1: #method 1 has decreasing variance
591.                                                    self.map.centre =
                                                        self.iterations(self.map.centreN1, self.map.varianceCheck,
                                                        Point.fromPolar(self.map.polarAverage()))
592.                                                    elif method == 2: #method 2 has decreasing variance
593.                                                        self.map.centre =
                                                            self.iterations(self.map.centreN2, self.map.varianceCheck,
                                                            Point.fromPolar(self.map.polarAverage()))
594.                                                    else:
595.                                                        print("Invalid Point") #Discloses point doesn't exist

```

```

596.                 else: #all points lie on a great circle
597.                     print("normal")
598.                     normal = np.cross(self.map.pointlist[0].cartesian,
self.map.pointlist[1].cartesian)
599.                     t_coeffecient = R / np.linalg.norm(normal)
600.                     self.map.centre = Point.fromCartesian(t_coeffecient *
normal)
601.
602.                 else:
603.                     if len(self.map.pointlist) == 1:
604.                         self.map.centre = self.map.pointlist[0] # the centre of
1 point is itself
605.
606.                     if len(self.map.pointlist) == 2:
607.                         self.map.centre =
self.map.centre2(self.map.pointlist[0],
608. self.map.pointlist[1]) # centre of 2 points
609.                         print(self.map.varianceCheck(self.map.centre))
610.
611.
612.                 self.newImgLoad() #loads in the new image
613.                 roundedLat = "{:.10f}".format(self.map.centre.polar[0])
614.                 roundedLog = "{:.10f}".format(self.map.centre.polar[1])
615.                 self.centreText.setText(f"The centre of these points are:
{roundedLat}, {roundedLog}")
616.                 #takes the first 10 values of the string of the polar coords of the
point it has found
617.
618.                 end = time.time()
619.
620.                 print(f"Runtime of the program is {end - start}")
621.
622.                 self.update()
623.
624.             def state_changed(self):
625.                 if self.satellite.isChecked() and not self.roadmap.isChecked():
626.                     self.map.maptype = "satellite" #satellite image
627.                 elif self.roadmap.isChecked() and not self.satellite.isChecked():
628.                     self.map.maptype = "roadmap" #roadmap image
629.                 elif self.roadmap.isChecked() and self.satellite.isChecked():
630.                     self.map.maptype = "hybrid" #hybrid image
631.                 elif not self.roadmap.isChecked() and not
self.satellite.isChecked():
632.                     self.map.maptype = "terrain" #terrain image
633.
634.                 self.newImgLoad() #loads in the new image with updated maptype
635.
636.             def wheelEvent(self, event: QWheelEvent):
637.                 delta = event.angleDelta().y()/120 #can only be 1 or -1
638.                 if self.map.zoom > 0: #lets user increase zoom
639.                     self.map.zoom = int(self.map.zoom + delta)
640.                 elif self.map.zoom == 0 and delta == 1: #lets user increase when
theyre at minimum zoom
641.                     self.map.zoom = int(self.map.zoom + delta)
642.                 elif self.map.zoom < 0:
643.                     self.map.zoom = 0 #ensures zoom can never be less than 0
644.                 self.newImgLoad() #loads in the new image with updated zoom
645.
646.         if __name__ == '__main__':
647.             app = QApplication(sys.argv)

```



```
648.         ex = App()
649.         sys.exit(app.exec_())
```

Bibliography:

Bertels, Dirk. *Intersections of Great Circles on a Sphere*. Jan. 2005, **Error! Hyperlink reference not valid..**

Bourke, Paul. "Circle, Cylinder, Sphere." *Paulbourke.net*, Apr. 1992,
paulbourke.net/geometry/circlesphere/#:~:text=A%20line%20can%20intersect%20a. Accessed 12 Apr. 2021.

Davis, Robert. "Geocoding with Google Maps API in Python | RapidAPI." *The Last Call - RapidAPI Blog*, 27 Mar. 2020, rapidapi.com/blog/google-maps-api-python/.

Dib, Firas. "Regex101: Build, Test, and Debug Regex." *Regex101*,
regex101.com/r/cO8lqs/4.

Fitzpatrick, Martin. "PyQt5 Tutorial 2021 — Create GUI Applications with Python and Qt." *Learn PyQt*, **Error! Hyperlink reference not valid..**

Fox, Jonny. "Regex Tutorial — a Quick Cheatsheet by Examples." *Medium*, 23 June 2017,
medium.com/factory-mind/regex-tutorial-a-simple-cheatsheet-by-examples-649dc1c3f285.

Google. "Get Started | Maps Static API." *Google Developers*, 18 Mar. 2021,
developers.google.com/maps/documentation/maps-static/start.

---. "Google Maps Platform | Google Developers." *Google Developers*, 2019,
developers.google.com/maps/documentation.

---. "Googlemaps: Python Client Library for Google Maps Platform." *PyPI*, 15 Mar. 2020,
pypi.org/project/googlemaps/.

"Googlemaps/Google-Maps-Services-Python." *GitHub*, github.com/googlemaps/google-maps-services-python.

Herrmann, Michael. "PyQt5 Tutorial 2020: Create a GUI with Python and Qt." *Build-System.fman.io*, build-system.fman.io/pyqt5-tutorial.

Kareem, Hafeezul. "Get a Google Map Image of Specified Location Using Google Static Maps API in Python." **Error! Hyperlink reference not valid.**, 12 Feb. 2020, **Error! Hyperlink reference not valid.**

LA_. "Javascript - How to Find a Point in the Middle of N Points?" *Stack Overflow*, 10 Dec. 2015, stackoverflow.com/questions/34210831/how-to-find-a-point-in-the-middle-of-n-points.

Maley, F. Miller, et al. *On the Areas of Cyclic and Semicyclic Polygons*. July 2004, arxiv.org/pdf/math/0407300v1.pdf.

Mapbox. "A New Algorithm for Finding a Visual Center of a Polygon." *Medium*, 29 June 2017, blog.mapbox.com/a-new-algorithm-for-finding-a-visual-center-of-a-polygon-7c77e6492fbc.

Marianne. "Lost but Lovely: The Haversine." *Plus.maths.org*, 4 July 2014, plus.maths.org/content/lost-lovely-haversine.

"Math.atan2() - JavaScript | MDN." *Developer.mozilla.org*, developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/atan2.

Parwiz. "PyQt5 How to Add Image in PyQt Window." *CodeLoop.org*, 2020, codeloop.org/pyqt5-how-to-add-image-in-pyqt-window/.

Patel, Zain. "Calculus - Minimizing a Function - Sum of Squares." *Mathematics Stack Exchange*, 7 July 2015, math.stackexchange.com/questions/1352726/minimizing-a-function-sum-of-squares.

Petroules, Jake. "Regex - Regular Expression for Matching Latitude/Longitude Coordinates?" *Stack Overflow*, 19 Aug. 2010, stackoverflow.com/questions/3518504/regular-expression-for-matching-latitude-longitude-coordinates.

Pretty Printed. "An Example of Using the Google Maps API with Python." **Error! Hyperlink reference not valid.**, 7 Dec. 2015, **Error! Hyperlink reference not valid.**

"PyQt5 – Check If Layout Direction of Radio Button Is Right to Left." *GeeksforGeeks*, 7 Apr. 2020, **Error! Hyperlink reference not valid.**

"PyQt5 Image." *Pythonspot*, pythonspot.com/pyqt5-image/.

"PyQt5 Textbox Example." *Pythonspot*, pythonspot.com/pyqt5-textbox-example/.

"Python | Get a Google Map Image of Specified Location Using Google Static Maps API." *GeeksforGeeks*, 18 May 2018, **Error! Hyperlink reference not valid.**

"Python | Plotting Google Map Using Folium Package." *GeeksforGeeks*, 7 June 2018, **Error! Hyperlink reference not valid.**

"Python Delete File." **Error! Hyperlink reference not valid.**, **Error! Hyperlink reference not valid.**

"QGuiApplication Class | Qt GUI 5.15.3." *Doc.qt.io*, doc.qt.io/qt-5/qguiapplication.html#layoutDirection-prop.

Ray. "Calculating the Centroid of Compound Shapes Using the Method of Geometric Decomposition." *Owlcation - Education*, 31 Dec. 2019, owlcation.com/stem/How-to-Solve-Centroids-of-Compound-Shapes.

Robinson, Scott. "Reading and Writing JSON to a File in Python." *Stack Abuse*, stackabuse.com/reading-and-writing-json-to-a-file-in-python/.

Saslow, Elliott. "Getting Started with Google Maps in Python." *Medium*, 2 Aug. 2019, medium.com/future-vision/google-maps-in-python-part-2-393f96196eaf.

Segal, Daniel. "Python - How Can I Know When the Enter Key Was Pressed on QTextEdit." *Stack Overflow*, 28 Aug. 2019, stackoverflow.com/questions/57698744/how-can-i-know-when-the-enter-key-was-pressed-on-qtextedit.

Song, Kenny. "Geometry - given an Arbitrary Number of Points, How Do You Find an Equidistant Center?" *Mathematics Stack Exchange*, 11 Apr. 2012, math.stackexchange.com/questions/130534/given-an-arbitrary-number-of-points-how-do-you-find-an-equidistant-center.

Strot, Joshua. "Python 2.7 - How to Check If a Checkbox Is Checked in Pyqt." *Stack Overflow*, 8 Dec. 2013, stackoverflow.com/questions/20455499/how-to-check-if-a-checkbox-is-checked-in-pyqt.

Thagana, Kimaru. "Map Visualizations in Python Using Folium | Pluralsight." **Error! Hyperlink reference not valid.**, 17 Sept. 2020, **Error! Hyperlink reference not valid.**

"The Great Circle Distance | Trigonometry: Compound Angles | Underground Mathematics." *Undergroundmathematics.org*, 27 Aug. 2017, undergroundmathematics.org/trigonometry-compound-angles/the-great-circle-distance#:~:text=We%20can%20work%20out%20the.

"Urllib.request — Extensible Library for Opening URLs — Python 3.8.6rc1 Documentation." *Docs.python.org*, docs.python.org/3/library/urllib.request.html.

Veness, Chris. "Calculate Distance and Bearing between Two Latitude/Longitude Points Using Haversine Formula in JavaScript." **Error! Hyperlink reference not valid.**, 23 Dec. 2020, **Error! Hyperlink reference not valid.**

Virgiliu. "Qt - Change Checkbox Position for a QCheckBox." *Stack Overflow*, 6 June 2010, stackoverflow.com/questions/3189951/change-checkbox-position-for-a-qcheckbox.

Weisstein, Eric W. "Bretschneider's Formula." *Mathworld.wolfram.com*, mathworld.wolfram.com/BretschneidersFormula.html.

---. "Cyclic Hexagon." *Mathworld.wolfram.com*, mathworld.wolfram.com/CyclicHexagon.html.

- . "Cyclic Pentagon." *Mathworld.wolfram.com*,
mathworld.wolfram.com/CyclicPentagon.html.
- . "Cyclic Polygon." *Mathworld.wolfram.com*,
mathworld.wolfram.com/CyclicPolygon.html.
- . "Cyclic Quadrilateral." *Mathworld.wolfram.com*,
mathworld.wolfram.com/CyclicQuadrilateral.html.
- . "Delaunay Triangulation." *Mathworld.wolfram.com*,
mathworld.wolfram.com/DelaunayTriangulation.html.
- . "Monic Polynomial." *Mathworld.wolfram.com*,
mathworld.wolfram.com/MonicPolynomial.html.
- . "Triangulation." *Mathworld.wolfram.com*,
mathworld.wolfram.com/Triangulation.html.
- Wikipedia Contributors. "Arc Length." *Wikipedia*, 4 Mar. 2021,
en.wikipedia.org/wiki/Arc_length#Arcs_of_great_circles_on_the_Earth.
- . "Bicentric Polygon." *Wikipedia*, 3 Nov. 2020,
en.wikipedia.org/wiki/Bicentric_polygon.
- . "Brahmagupta's Formula." *Wikipedia*, Wikimedia Foundation, 24 Mar. 2019,
en.wikipedia.org/wiki/Brahmagupta%27s_formula.
- . "Centroid." *Wikipedia*, 11 Feb. 2020, en.wikipedia.org/wiki/Centroid.
- . "Circumscribed Circle." *Wikipedia*, 23 Nov. 2019,
en.wikipedia.org/wiki/Circumscribed_circle.
- . "Delaunay Triangulation Wikipedia." *Wikipedia*, 8 Apr. 2019,
en.wikipedia.org/wiki/Delaunay_triangulation#Properties.
- . "Geographic Coordinate System." *Wikipedia*, 9 Apr. 2021,
en.wikipedia.org/wiki/Geographic_coordinate_system#cite_note-GISS-14.

- . "Geometric Median." *Wikipedia*, 19 Jan. 2019,
en.wikipedia.org/wiki/Geometric_median.
- . "Great-Circle Distance." *Wikipedia*, Wikimedia Foundation, 30 Apr. 2019,
en.wikipedia.org/wiki/Great-circle_distance.
- . "Haversine Formula." *Wikipedia*, Wikimedia Foundation, 7 Aug. 2019,
en.wikipedia.org/wiki/Haversine_formula.
- . "Incircle and Excircles of a Triangle." *Wikipedia*, 21 Dec. 2020,
en.wikipedia.org/wiki/Incircle_and_excircles_of_a_triangle.
- . "Pole of Inaccessibility." *Wikipedia*, 6 Apr. 2019,
en.wikipedia.org/wiki/Pole_of_inaccessibility.
- . "Shoelace Formula." *Wikipedia*, 6 Feb. 2021,
en.wikipedia.org/wiki/Shoelace_formula.
- . "Spherical Law of Cosines." *Wikipedia*, 11 Mar. 2019,
en.wikipedia.org/wiki/Spherical_law_of_cosines.
- . "Symmetric Polynomial." *Wikipedia*, 19 Oct. 2019,
en.wikipedia.org/wiki/Symmetric_polynomial.